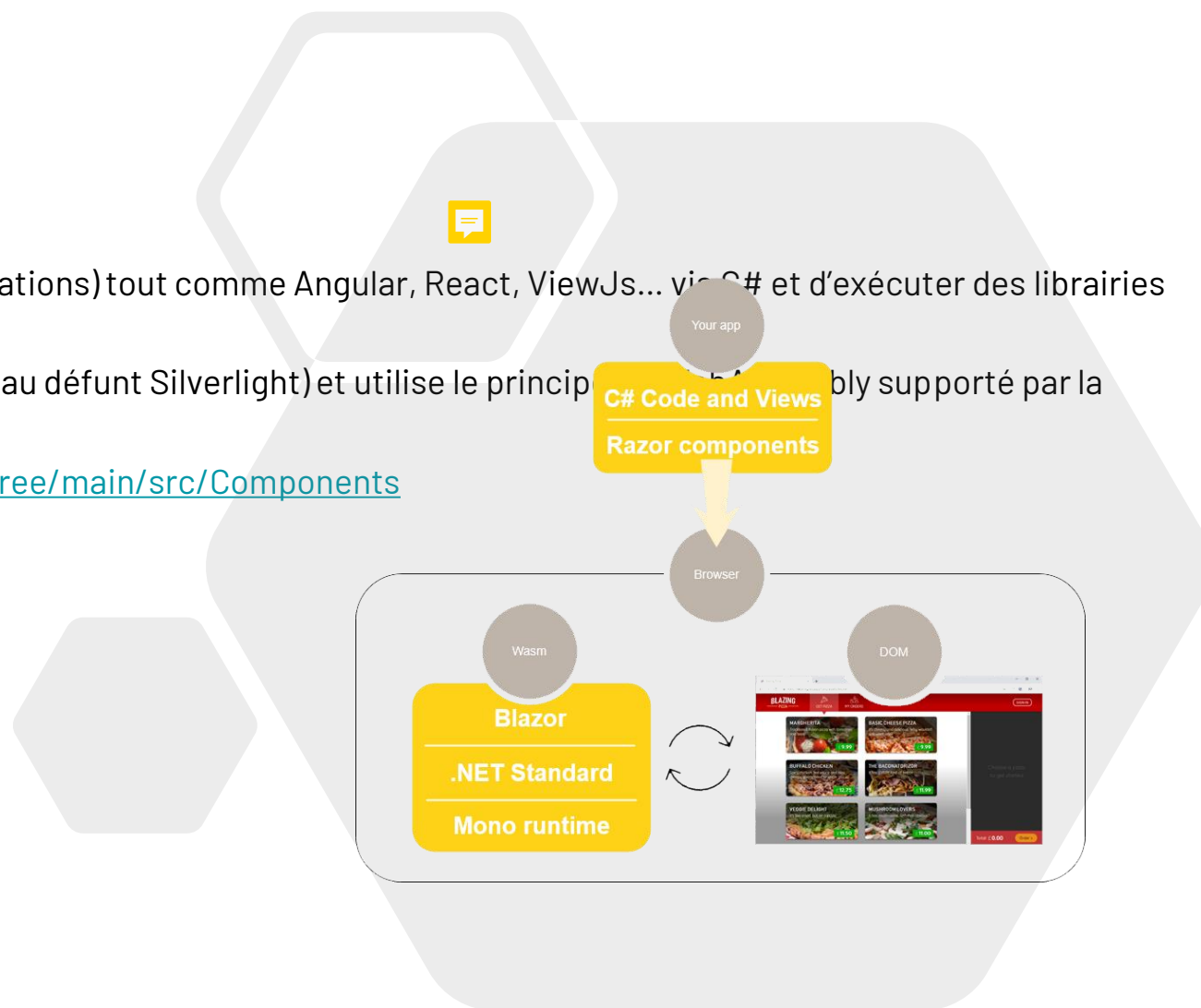


Blazor

SPA Framework

Introduction

- Le nom Blazor vient de Browser et Razor.
- Framework permettant de créer des SPAs(Single Page Applications) tout comme Angular, React, ViewJs... via C# et d'exécuter des librairies .NET standard dans notre navigateur.
- Ne requiert aucun plugin sur notre navigateur (contrairement au défunt Silverlight) et utilise le principe de WebAssembly, déjà supporté par la majorité des navigateurs.
- Il est open-source : <https://github.com/dotnet/aspnetcore/tree/main/src/Components>



Introduction : Avantages

- **Générez rapidement l'interface utilisateur web avec des composants réutilisables** : Le modèle de composant flexible de Blazor facilite la création de composants réutilisables
- **Ajouter une interactivité enrichie en C#** : Gérez des événements d'interface utilisateur arbitraires à partir du navigateur et implémentez la logique des composants en C#
- **Une pile de développement** : générez l'ensemble de votre application web à partir du serveur frontal vers le back-end à l'aide d'une pile de développement unique
- **Rendu efficace basé sur les différences** : Lors du rendu des composants, Blazor suit attentivement les parties du DOM qui ont été modifiées, afin que les mises à jour de l'interface utilisateur soient rapides et efficaces.
- **Rendu côté serveur et côté client** : Affichez des composants à partir du serveur et du client pour implémenter diverses architectures d'application web et offrir la meilleure expérience d'application web possible.

Introduction : Avantages

- **Interopérabilité avec JavaScript** : Utilisez l'écosystème des bibliothèques JavaScript et des API de navigateur à partir de votre code C#.
- **Intégrer à des applications existantes** : Intégrez des composants Blazor à des applications MVC, Razor Pages ou JavaScript existantes.
- **Web, mobile et bureau** : les composants Blazor peuvent également être utilisés pour créer des applications mobiles et de bureau natif à l'aide d'un hybride de type natif et web, appelé Blazor Hybrid.

Introduction : Concepts

- **Components**

Les composants constituent la base de l'architecture Blazor. Ce sont des unités d'interface utilisateur réutilisables. Ils peuvent être écrits en C# et en HTML, ou avec la syntaxe Razor, qui permet aux développeurs d'écrire du code C# directement dans le HTML. Ils peuvent comprendre des attributs, des méthodes et des événements, permettant de créer une interface utilisateur plus complexe.

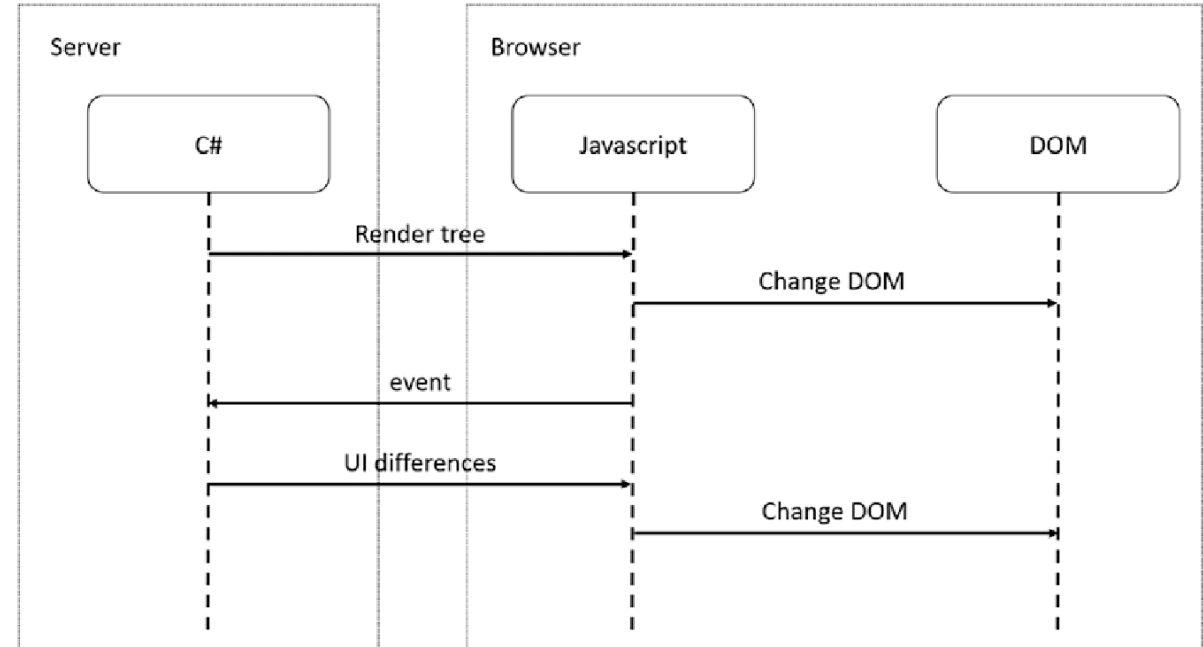
- **Rendering**

Une fois les composants définis, Blazor les restitue dans le DOM virtuel, plus léger que le DOM réel. Lorsque l'état de l'application change, Blazor compare le DOM virtuel précédent à l'état actuel et met à jour uniquement la partie du DOM à modifier. Cela améliore les performances et réduit la quantité de données à transférer entre le serveur et le client.

Introduction : Concepts

- **Hosting**

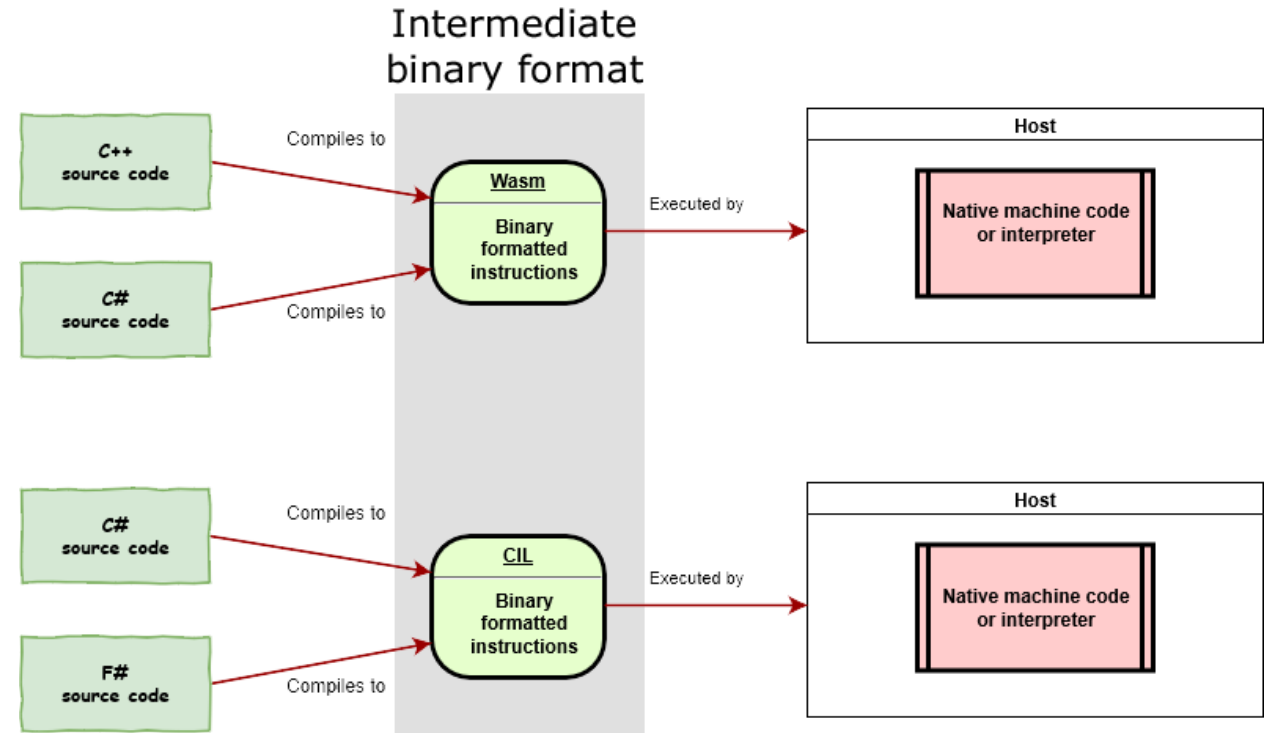
Blazor prend en charge deux modèles d'hébergement : côté serveur et côté client. En hébergement côté serveur, les composants Blazor sont exécutés sur le serveur et l'interface utilisateur est affichée dans le navigateur à l'aide de SignalR pour envoyer les mises à jour au client.



Introduction : Concepts

- Hosting

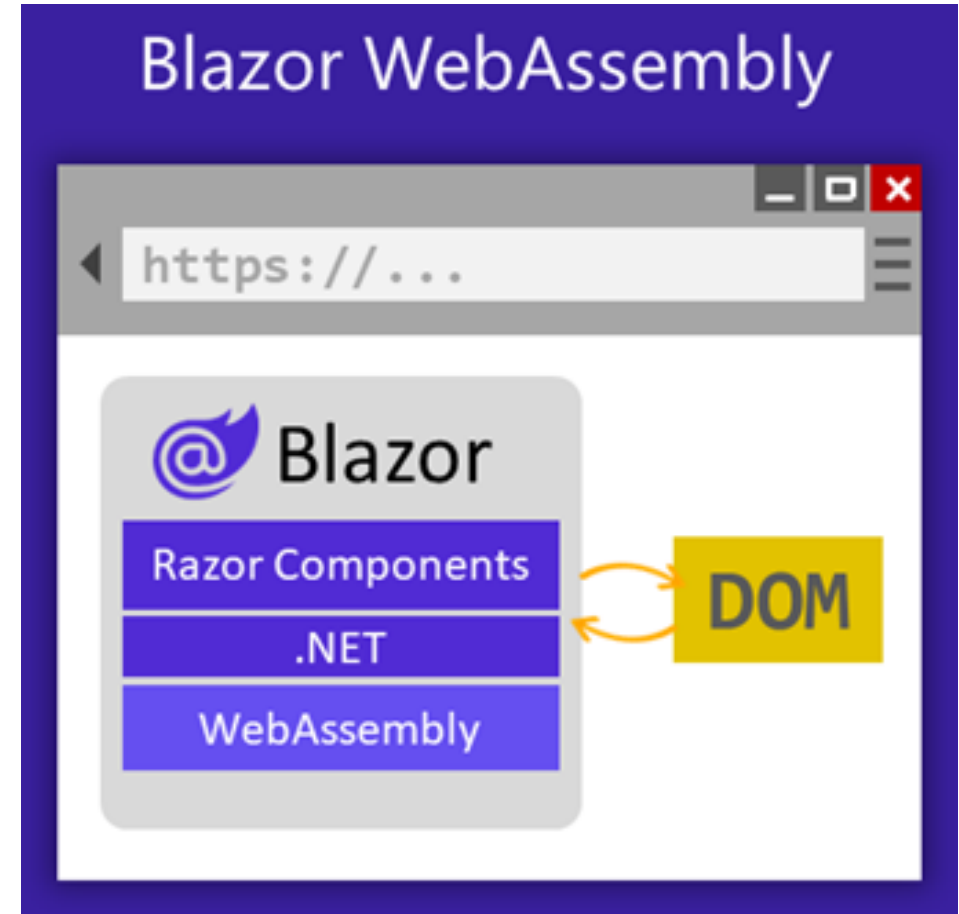
côté client, l'application Blazor, ses dépendances et l'environnement d'exécution .NET sont téléchargés dans le navigateur, et l'application est exécutée directement sur le thread d'interface utilisateur du navigateur.



Les modèles de projets Blazor

Blazor Web Assembly standAlone (WASM)

- framework de développement web côté client permettant de créer des applications web interactives en C# plutôt qu'en Javascript.
- s'exécutent directement dans le navigateur et ne nécessitent donc pas de serveur pour afficher l'interface utilisateur.



Blazor Web Assembly standAlone (WASM)

Avantages

- **Exécution dans le navigateur** : améliore la sécurité en évitant les échanges serveurs constants
- **Langage C# fortement typé** : réduit les erreurs de programmation courantes.
- **Compatibilité multiplateforme** : fonctionne sur ordinateurs, mobiles et objets connectés
- **Réduction des coûts** : un seul développement pour plusieurs plateformes.
- **Autonomie après chargement** : continue de fonctionner même si le serveur est indisponible. 

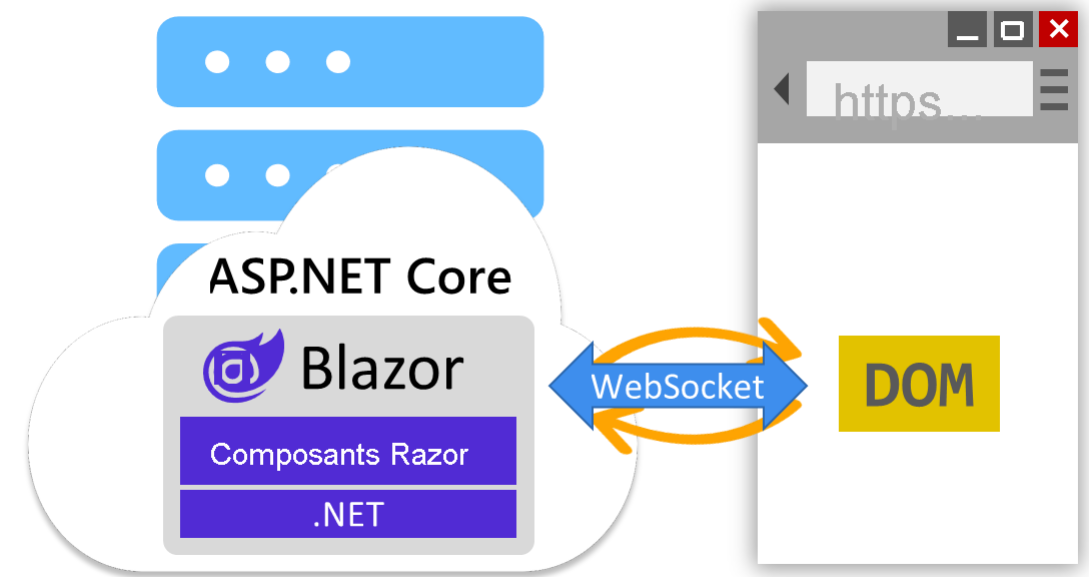
Inconvénients

- **Compatibilité limitée** : les anciens navigateurs ne prennent pas toujours en charge WebAssembly.
- **Temps de chargement initial élevé** : l'application inclut tout le runtime .NET, ce qui rallonge le démarrage (temps de démarrage à froid).
- **Bibliothèques limitées** : certaines bibliothèques .NET ne sont pas encore compatibles avec Blazor WebAssembly

En résumé, Blazor WebAssembly offre un framework de développement adaptable qui vous permet de créer des applications rapides, sécurisées et modernes en utilisant une technologie familière.

Blazor Web App (Support côté client et côté serveur)

- **Accès aux ressources serveur** : les composants côté serveur peuvent interagir directement avec les bases de données et services back-end.
- **Rendu statique par défaut** : les composants génèrent du HTML en réponse aux requêtes sans interactivité.
- **Composants serveur interactifs** : permettent de gérer les événements, conserver l'état et mettre à jour l'interface dynamiquement.
- **Connexion en temps réel** : les interactions sont gérées via une connexion WebSocket entre le serveur et le navigateur.



Blazor Web App (Support côté client et côté serveur)

Avantages

- **Modèle unifié** : Blazor Server + WebAssembly dans une seule app
- **Mode interactif auto** : adaptation dynamique selon le contexte
- **Réutilisation** des composants côté client et serveur
- **Chargement** initial rapide via le serveur

Inconvénients

- **Configuration** plus complexe (gestion des modes interactifs)
- **Taille** du bundle WebAssembly encore importante
- **Performances** variables selon l'appareil client
- **Dépendance** au WebSocket pour Blazor Server 
- Certaines bibliothèques .NET non compatibles avec WebAssembly

En résumé, Blazor WebAssembly offre un framework de développement adaptable qui vous permet de créer des applications rapides, sécurisées et modernes en utilisant une technologie familière.

Éléments clés pour les développeurs Blazor

.Net : Fondamentaux

Open Source & Multiplateforme

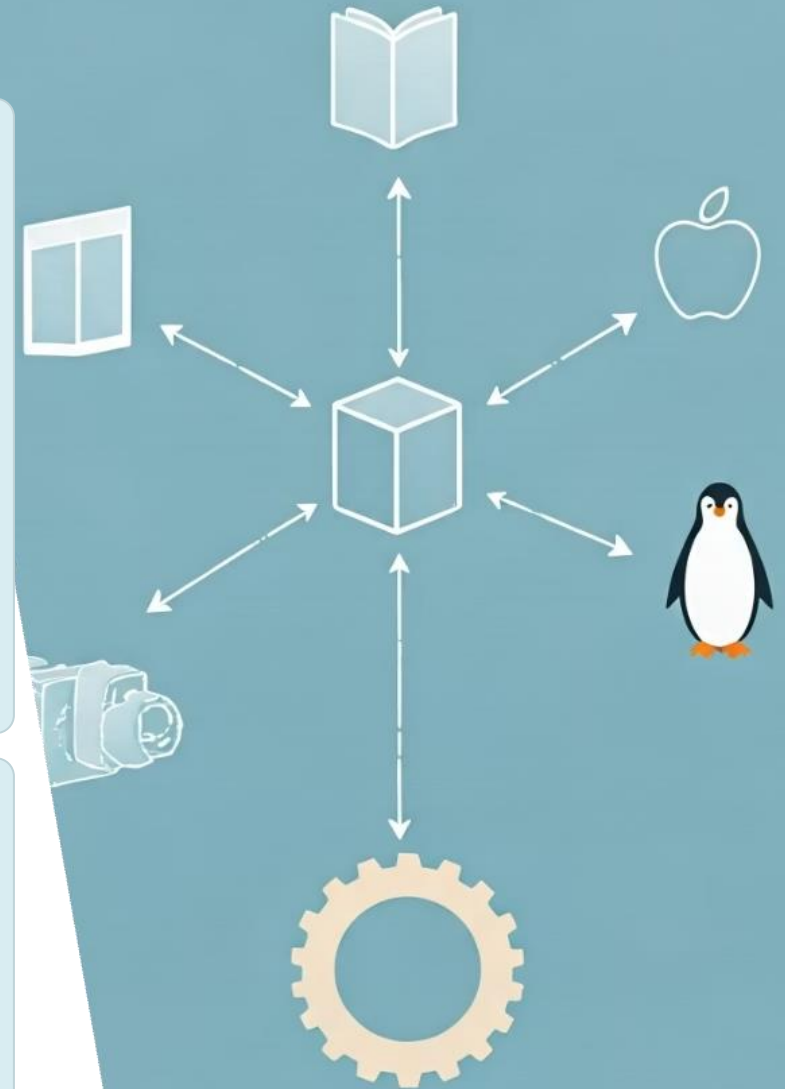
Contrairement à l'ancien .NET Framework, .NET Core fonctionne nativement sur Windows, Linux et macOS, offrant une flexibilité inédite pour les déploiements et l'environnement de développement.

Performance & Modularité

Architecture optimisée pour les performances avec une gestion mémoire améliorée. Le système de packages NuGet permet d'intégrer uniquement les composants nécessaires, réduisant l'empreinte mémoire.

Cloud & Microservices

Conçu dès le départ pour les architectures cloud et microservices, avec des outils natifs pour les conteneurs Docker, l'orchestration Kubernetes et les déploiements Azure.



Le Framework MVC

Model

Représente les données et la logique métier de l'application. Encapsule l'état et fournit des méthodes pour y accéder.



View

Affiche les données du modèle à l'utilisateur. Dans ASP.NET Core, principalement implémenté via le moteur Razor (.cshtml).

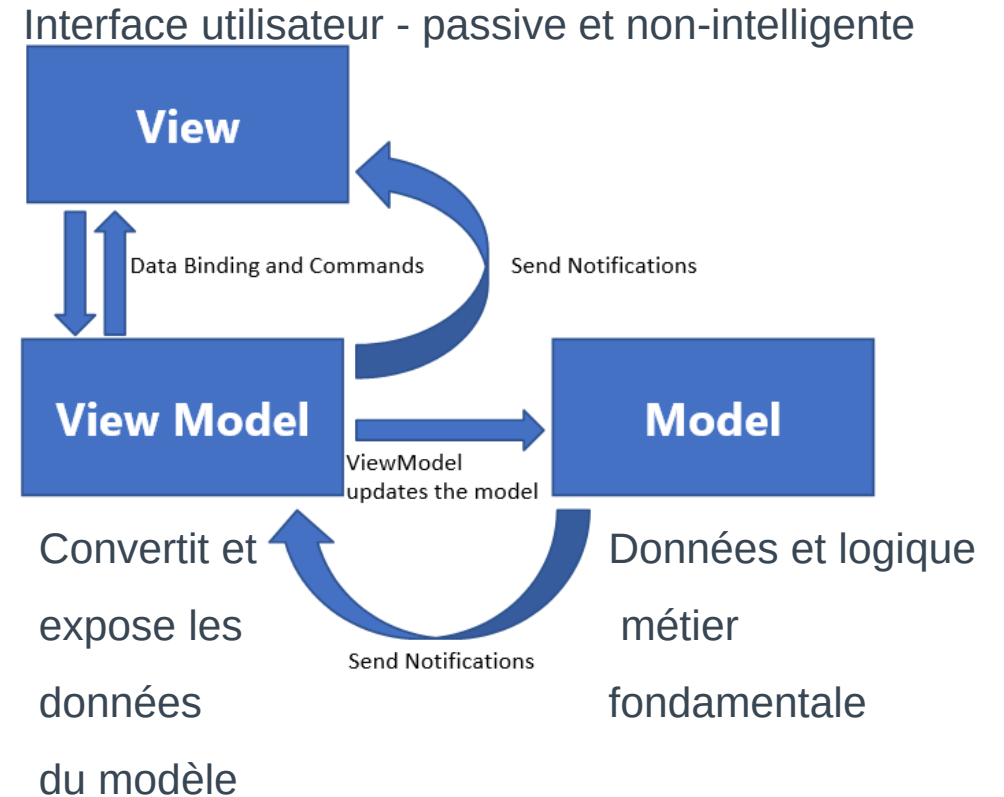
Controller

Gère les interactions utilisateur, manipule le modèle et sélectionne la vue à afficher en fonction des actions utilisateur.

Design Pattern : MVVM

Le pattern MVVM (Model-View-ViewModel) est particulièrement pertinent pour Blazor et les applications avec interfaces riches. Contrairement au MVC, le ViewModel sert de médiateur entre la vue et le modèle, exposant les propriétés et commandes auxquelles la vue peut se lier.

Ce pattern facilite la séparation des préoccupations et permet de tester la logique métier indépendamment de l'interface utilisateur. Les bindings bidirectionnels automatisent la synchronisation des données entre la vue et le ViewModel, réduisant le code nécessaire pour maintenir cette cohérence.



Applications Web



ASP.NET Core

Framework unifié pour créer des applications web et des API. Intègre les fonctionnalités de l'ancien ASP.NET MVC et Web API en un seul modèle de programmation cohérent.



Pipeline de middleware

Chaque requête HTTP traverse une série de composants configurables (middleware) qui peuvent inspecter, modifier ou court-circuiter la requête avant qu'elle n'atteigne le contrôleur.



Hébergement flexible

Serveur Kestrel intégré pour le traitement HTTP direct, avec possibilité de déploiement derrière IIS, Nginx, Apache, ou directement dans des services cloud comme Azure App Service.

Web API



Création d'endpoints



Définition simple de points d'accès REST avec des attributs de routage et de méthodes HTTP (GET, POST, PUT, DELETE).



Sérialisation automatique

Conversion transparente entre les objets C# et les formats JSON ou XML, avec contrôle précis sur le processus de sérialisation.



Architecture découplée

Séparation claire entre le backend (API) et le frontend (SPA, mobile), permettant une évolution indépendante des deux parties.

Les Web API constituent la base de communication entre votre backend .NET et vos clients, qu'ils soient des applications mobiles, des SPA comme Angular, ou Blazor. Elles représentent la façon moderne d'exposer des fonctionnalités via HTTPS.

REST

REpresentational **S**tate **T**ransfer est un style d'architecture logicielle définissant un ensemble de contraintes à utiliser pour créer des services web.

Cette architecture inclus 5 contraintes :

- Uniform UI
- Client-Server
- Stateless
- Cacheable
- Layered System

REST

Uniform Interface

En appliquant le principe de généralité à l'interface des composants, nous pouvons simplifier l'architecture globale du système et améliorer la visibilité des interactions : Separation of concern, Modularity, Abstraction,...

Dans le cadre des API REST uniforme , les 4 contraintes suivantes doivent être respectées :

- **Identification des ressources** – L'interface doit identifier de manière unique chaque ressource impliquée dans l'interaction entre le client et le serveur.
- **Manipulation des ressources par des représentations** – Les ressources doivent avoir des représentations uniformes dans la réponse du serveur. Les consommateurs d'API doivent utiliser ces représentations pour modifier l'état des ressources sur le serveur. 
- **Messages auto-descriptifs** – Chaque représentation de ressource doit contenir suffisamment d'informations pour décrire comment traiter le message. Il doit également fournir des informations sur les actions supplémentaires que le client peut effectuer sur la ressource.
- **Hypermédia comme moteur de l'état de l'application** – Le client ne doit avoir que l'URI initial de l'application. L'application cliente doit piloter dynamiquement toutes les autres ressources et interactions à l'aide d'hyperliens.



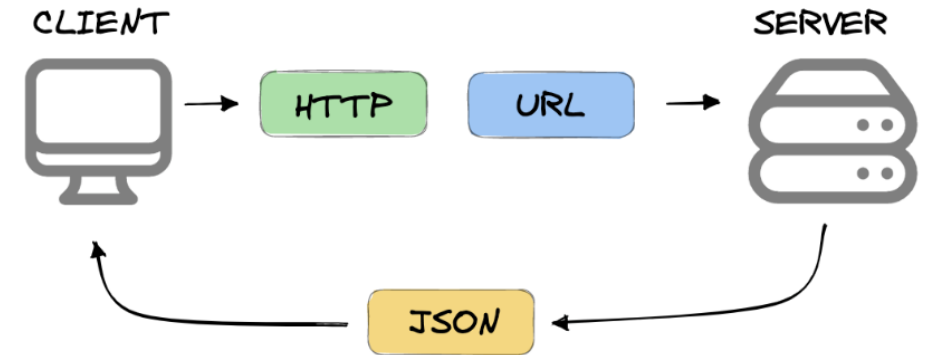
REST

Client-Server

Le modèle de conception client-serveur applique la **séparation des préoccupations (Seperation of concerns)**, ce qui aide les composants client et serveur à évoluer indépendamment.

En séparant les problèmes d'interface utilisateur (client) des problèmes de stockage de données (serveur), nous améliorons la portabilité de l'interface utilisateur sur plusieurs plates-formes et améliorons l'évolutivité en simplifiant les composants du serveur.

Pendant que le client et le serveur évoluent, nous devons nous assurer que l'interface/le contrat entre le client et le serveur ne se rompt pas.

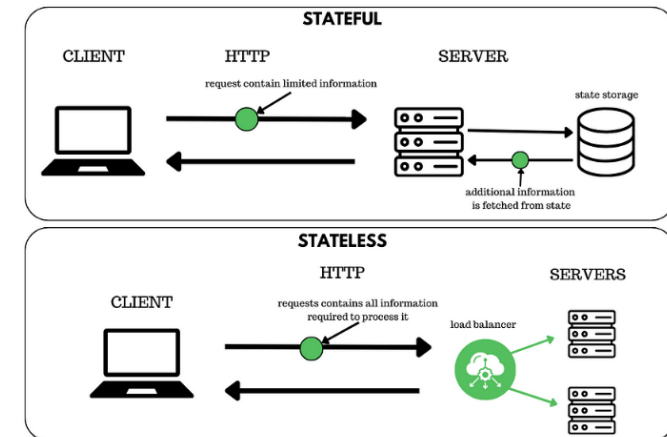


Stateless

- Exige que chaque demande du client au serveur contienne toutes les informations nécessaires pour comprendre et compléter la demande.
- Le serveur ne peut pas tirer parti des informations de contexte précédemment stockées sur le serveur.
- L'application cliente doit conserver entièrement l'état de session.



API REST: STATEFUL VS STATELESS



<https://medium.com/>

REST

Cacheable

La contrainte requiert qu'une réponse spécifie implicitement ou explicitement si elle peut être mise en cache ou non.

Si la réponse peut être mise en cache, l'application cliente obtient le droit de réutiliser les données de réponse ultérieurement pour des demandes équivalentes et une période spécifiée,

Nous devons donc utiliser le header HTTP pour renseigner les informations suivantes :

- **Expires HTTP header** : Expires: <http-date>

<https://developer.mozilla.org/fr/docs/Web/HTTP/Headers/Expires>

- **Cache-Control**

<https://developer.mozilla.org/fr/docs/Web/HTTP/Headers/Cache-Control>

- **Etag** : identifie la ressource de manière unique

<https://developer.mozilla.org/fr/docs/Web/HTTP/Headers/ETag>

- **Last-Modified** : Last-Modified: <nom-jour>, <jour> <mois> <année> <heure>:<minute>:<seconde> GMT

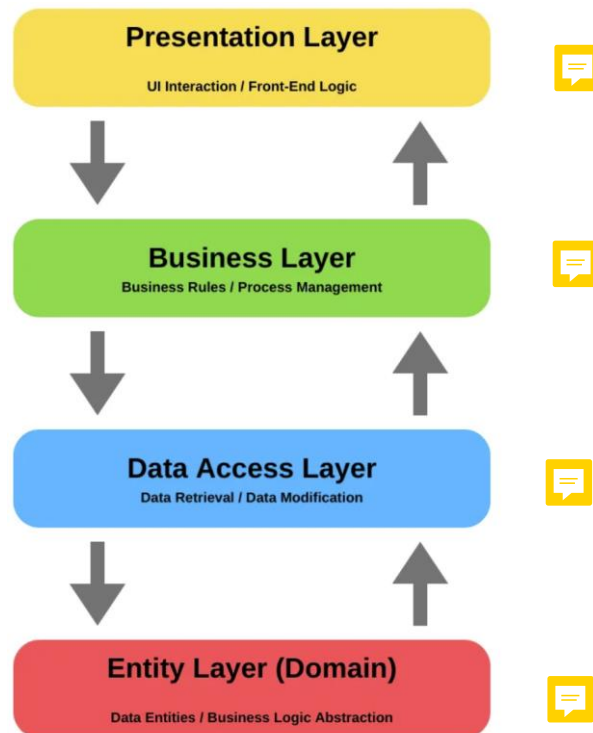
<https://developer.mozilla.org/fr/docs/Web/HTTP/Headers/Last-Modified>

REST

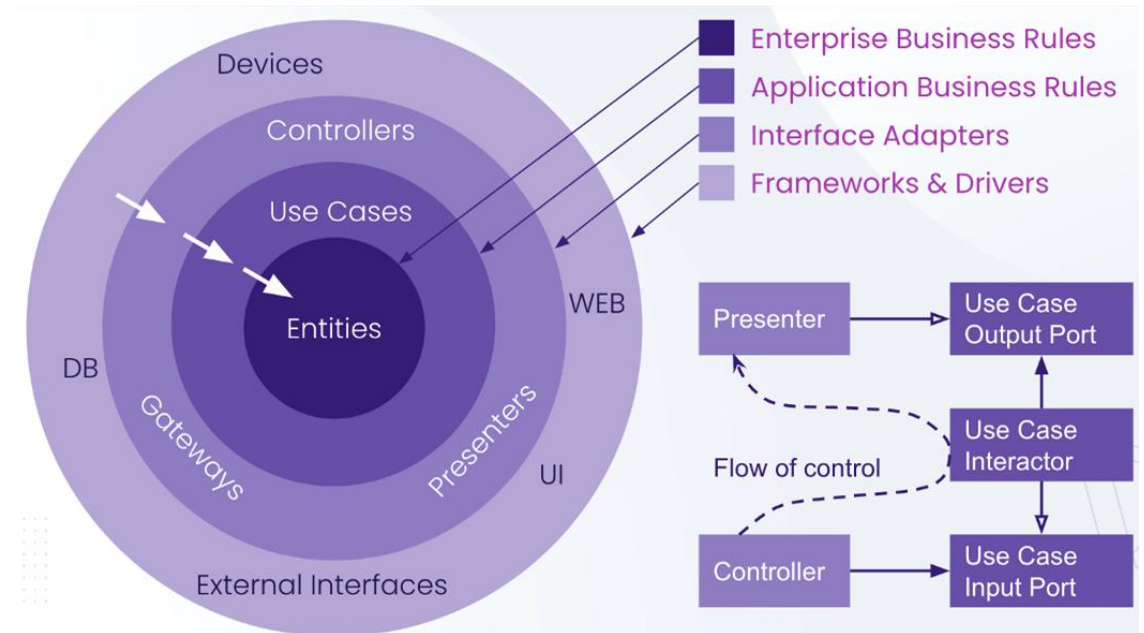
Modèle en couche

Le style de système en couches permet à une architecture d'être composée de couches hiérarchiques en contraignant le comportement des composants.

N-tiers



Clean Architecture



<https://medium.com/>

Sécurité du navigateur côté client

HTTPS obligatoire

ASP.NET encourage l'utilisation de HTTPS même en développement.

Le framework inclut des mécanismes pour rediriger automatiquement le trafic HTTP vers HTTPS et gérer les certificats de développement.

Protection contre les attaques courantes

Protections intégrées contre les attaques XSS (Cross-Site Scripting) via l'encodage automatique des sorties et contre le CSRF (Cross-Site Request Forgery) via les jetons

anti-falsification CORS et cookies sécurisés

Configuration précise des règles CORS (Cross-Origin Resource Sharing) pour contrôler quels domaines peuvent accéder à vos API.

Options avancées pour les cookies (HttpOnly, Secure, SameSite) limitant les risques d'attaques.

La sécurité web doit être une préoccupation constante. ASP.NET offre de nombreux mécanismes pour protéger vos applications, mais il est essentiel de comprendre ces concepts pour les configurer correctement selon vos besoins spécifiques.

WebAssembly

Compilation AOT

Le code C# est compilé en WebAssembly, format binaire compact exécuté directement par le navigateur à des vitesses proches du code natif.

Blazor WebAssembly révolutionne le développement web en permettant d'exécuter du code .NET directement dans le navigateur.

Pour les développeurs .NET, cela signifie pouvoir créer des SPA complètes sans nécessiter de JavaScript, tout en bénéficiant des outils et frameworks .NET familiers.

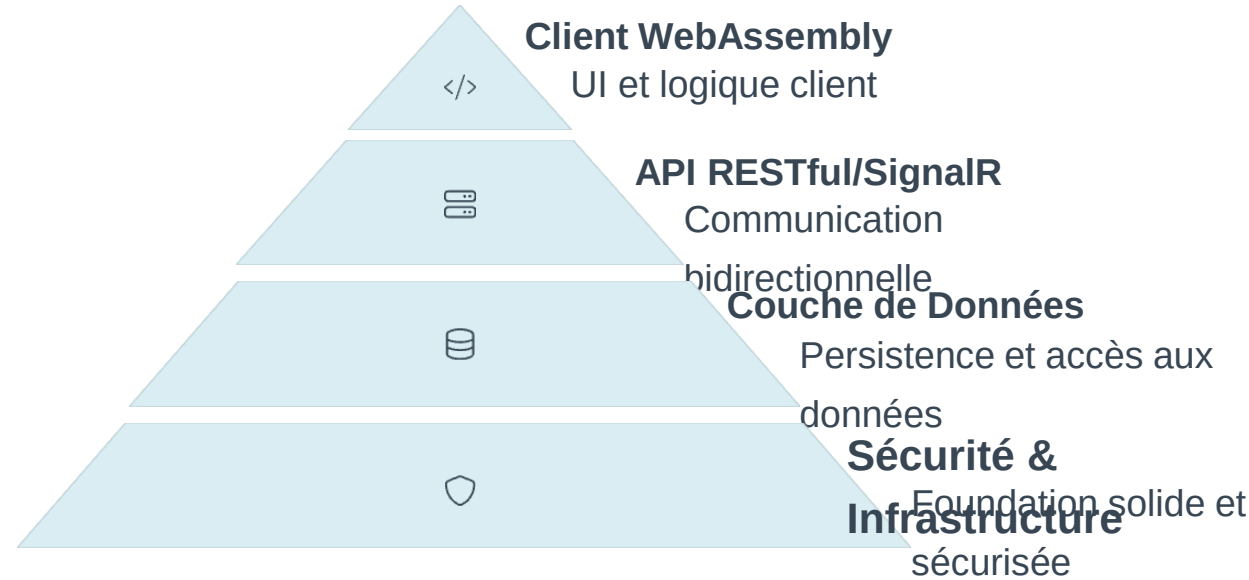
Interopérabilité JavaScript

Communication bidirectionnelle entre C# et JavaScript, permettant d'utiliser les bibliothèques JavaScript existantes tout en écrivant la logique principale en C#.

Réutilisation du code

Même code C# et même logique métier partagés entre le serveur et le client, éliminant la duplication et réduisant les erreurs de synchronisation.

WebAssembly : Architecture et Fondamentaux



Au cœur de WebAssembly pour .NET 8 se trouve le runtime Mono, méticuleusement compilé pour s'exécuter nativement dans l'environnement du navigateur. Cette prouesse technique permet l'amélioration des performances tout en conservant la richesse de l'écosystème .NET.

La disparition du template ASP.NET Core Hosted dans .NET 8 marque un tournant architectural important.

La structure alternative recommandée s'articule désormais autour d'une séparation claire entre API, Client et bibliothèques partagées (Shared), favorisant ainsi une architecture Clean ou Hexagonale (modèle Onion).

Cette organisation modulaire facilite la communication entre le client et le serveur via des API REST traditionnelles ou des connexions temps réel avec SignalR, offrant une flexibilité technique sans précédent.

WebAssembly: Modèles d'architecture et déploiement



Déploiement Continu

Intégration fluide avec GitHub Actions et conteneurisation Docker pour un déploiement automatisé et fiable de vos applications WebAssembly.



Architecture en Couches

Organisation modulaire avec séparation claire des responsabilités selon les principes de la Clean Architecture pour une maintenance optimale.



Sécurité Intégrée

Implémentation robuste de l'authentification JWT et des autorisations basées sur les rôles pour protéger efficacement vos applications.



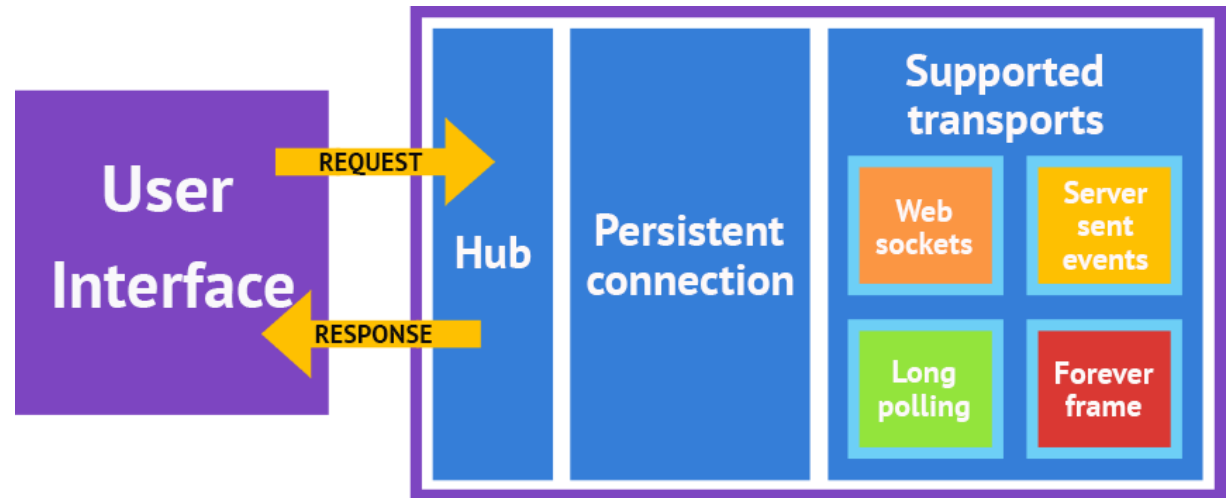
SignalR

- SignalR est une bibliothèque open-source développée par Microsoft qui facilite l'implémentation de fonctionnalités web en temps réel. Elle permet une communication bidirectionnelle instantanée entre le client et le serveur, éliminant le besoin de rafraîchissement manuel des pages.
- Cette technologie s'appuie sur plusieurs protocoles comme **WebSockets** (premier choix), **Server-Sent Events** (meilleur support des navigateurs), et **Long Polling** (meilleure compatibilité avec les anciens navigateurs).



SignalR : les protocoles

- WebSocket (HTML 5)
 - Protocole qui permet de créer un canal de communication full-duplex en TCP.
- Server Sent Events (HTML 5 - **Non compatible avec IE / Edge**)
 - API qui permet à un navigateur de recevoir une mise à jours à partir du serveur.
- Ajax long polling
 - Utilisation de requête ouverte en attente de réponse du serveur.
Lorsque la requête est terminée, une nouvelle requête est envoyé.



Environnement de développement Blazor

Visual studio

Téléchargement

- **Visual Studio Community** (gratuit) ou Professional/Enterprise
- Télécharger depuis :
<https://visualstudio.microsoft.com/fr/>

Composants individuels importants

- .NET Runtime (LTS)
- SDK .NET
- Git pour Windows

Workloads essentiels à sélectionner

- ✓ Développement web et ASP.NET
 Développement web et ASP.NET
Générez des applications web en utilisant ASP.NET Core, ASP.NET, HTML/JavaScript ainsi que des conteneurs pren...☒
- ✓ Développement multiplateforme .NET
 Développement .NET Multi-Platform App UI
Créez des applications Android, iOS, Windows et Mac à partir d'une base de code unique en utilisant C# avec .NE...☒
- ✓ Développement Azure (optionnel)
 Développement Azure
Kits Azure SDK, outils et projets pour le développement d'applications cloud et la création de ressources à l'aide...☒

Vérification de l'environnement

Vérifier la version de .NET installée
dotnet --version
Lister tous les SDKs installés
dotnet --list-sdks
Lister tous les runtimes installés
dotnet --list-runtimes

Notre premier projet Blazor Web App Blazor

Création du projet

- **Via Visual studio**

The image shows two screenshots from the Visual Studio IDE. The top screenshot is the 'Create a new project' dialog. It features a search bar with 'Blazor' entered, a 'Clear all' button, and three filters: 'C#' for language, 'All platforms' for platform, and 'All project types' for project type. Below these, a list of project templates is shown, with 'Blazor Web App' selected and highlighted. The bottom screenshot is the 'Configure your new project' dialog for the 'Blazor Web App' template. It shows the project name 'MaPremierApplicationBlazorApp', the location 'C:\Users\micha\source\repos\I', and the solution name 'MaPremierApplicationBlazorApp'. There is an unchecked checkbox for 'Place solution and project in the same directory' and a summary line at the bottom stating the project will be created in the specified path.

Create a new project

Recent project templates

Console App C#

Class Library C#

Blazor

Clear all

C# All platforms All project types

Blazor Web App

A project template for creating a Blazor web app that supports both server-side rendering and client interactivity. This template can be used for web apps with rich dynamic user interfaces (UIs).

C# Linux macOS Windows Blazor Cloud Web

Configure your new project

Blazor Web App C# Linux macOS Windows Blazor Cloud Web

Project name

MaPremierApplicationBlazorApp

Location

C:\Users\micha\source\repos\I

Solution name ⓘ

MaPremierApplicationBlazorApp

☐ Place solution and project in the same directory

Project will be created in "C:\Users\micha\source\repos\I\MaPremierApplicationBlazorApp\MaPremierApplicationBlazorApp\"

Création du projet

- **Framework :**
Choisir une version LTS (ici dans l'exemple .net 8)
- **Authentication Type :**
 - 1) *None* – Aucun ajout dans le projet pour la sécurité
 - 2) *Individual Account* – Ajoute les dépendances et fonctionnalités (Login, Register,...) permettant d'utiliser Asp Net Identity

Additional information

Blazor Web App

C#

Linux

macOS

Windows

Blazor

Cloud

Web

Framework ⓘ

.NET 8.0 (Long Term Support)

Authentication type ⓘ

None

☒ Configure for HTTPS ⓘ

Interactive render mode ⓘ

Auto (Server and WebAssembly)

Interactivity location ⓘ

Per page/component

☒ Include sample pages ⓘ

☒ Do not use top-level statements ⓘ

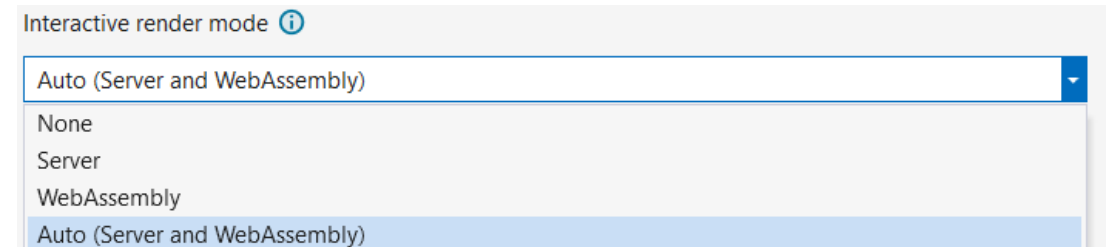
☐ Enlist in .NET Aspire orchestration ⓘ

Aspire version ⓘ

9.0

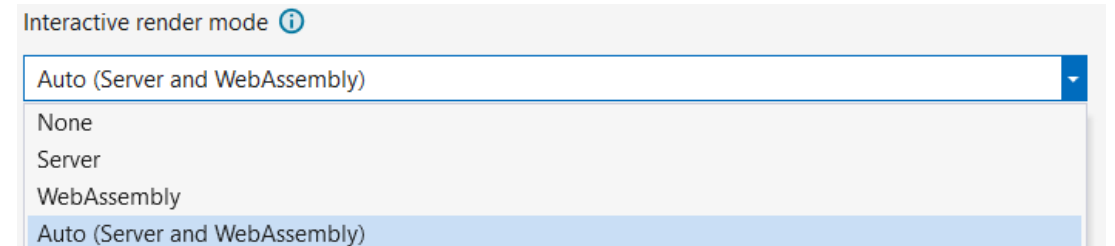
Création du projet

- **Interactive render mode**
 - **None**
 - Aucune interactivité côté client
 - Rendu statique uniquement côté serveur (SSR)
 - Pages HTML générées au moment de la requête
 - Idéal pour des sites principalement informatifs avec peu d'interaction



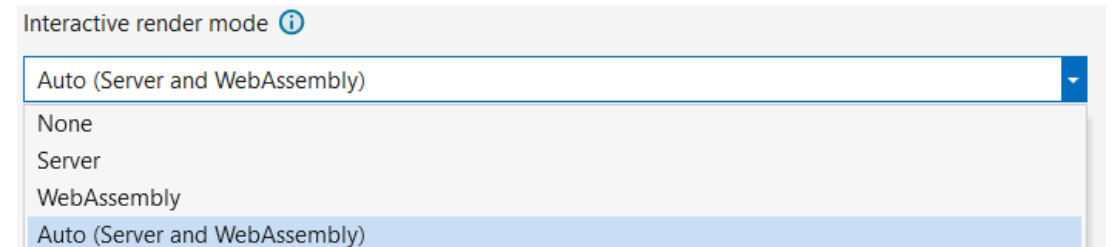
Création du projet

- **Interactive render mode** 
 - **Server**
 - Mode Blazor Server interactif
 - L'interface utilisateur s'exécute sur le serveur via SignalR
 - Les interactions utilisateur sont envoyées au serveur en temps réel
 - Bon pour des applications riches sans téléchargement lourd côté client
 - Nécessite une connexion permanente au serveur



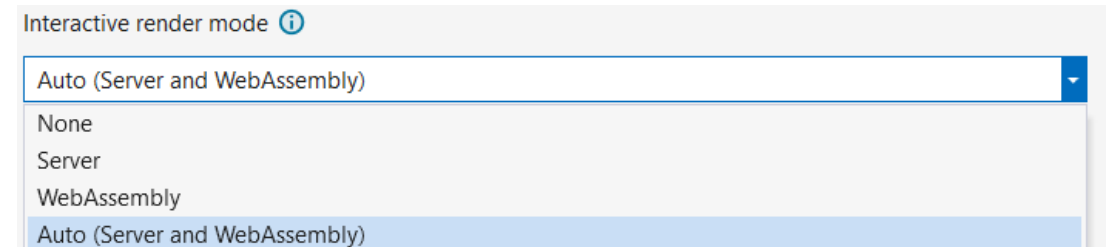
Création du projet

- **Interactive render mode**
 - **WebAssembly**
 - Mode Blazor WebAssembly interactif
 - L'application s'exécute entièrement dans le navigateur
 - Téléchargement initial plus lourd mais fonctionne hors ligne
 - Idéal pour des applications client riches et autonomes



Création du projet

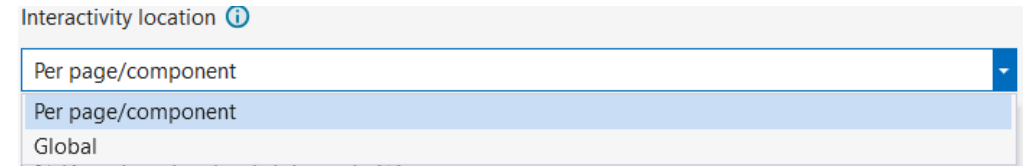
- **Interactive render mode**
 - **Auto (Server and WebAssembly) - Sélectionné**
 - Mode hybride qui combine les avantages des deux approches
 - Démarre en mode Server pour un chargement rapide
 - Passe progressivement en WebAssembly en arrière-plan
 - Offre la meilleure expérience utilisateur mais plus complexe à développer



Le choix "Auto" est recommandé pour la plupart des nouvelles applications car il optimise automatiquement les performances.

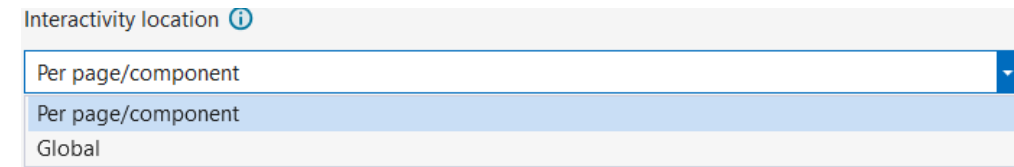
Création du projet

- **Interactivity location**
 - **Per page/component**
 - L'interactivité est définie individuellement pour chaque page ou composant
 - Permet un contrôle granulaire : certaines pages peuvent être statiques, d'autres interactives
 - Flexibilité maximale pour optimiser les performances selon les besoins
 - Idéal pour des applications mixtes avec des sections variées
 - **Global**
 - L'interactivité s'applique à toute l'application de manière uniforme
 - Toutes les pages utilisent le même mode de rendu interactif
 - Plus simple à gérer mais moins flexible
 - Convient aux applications entièrement interactives



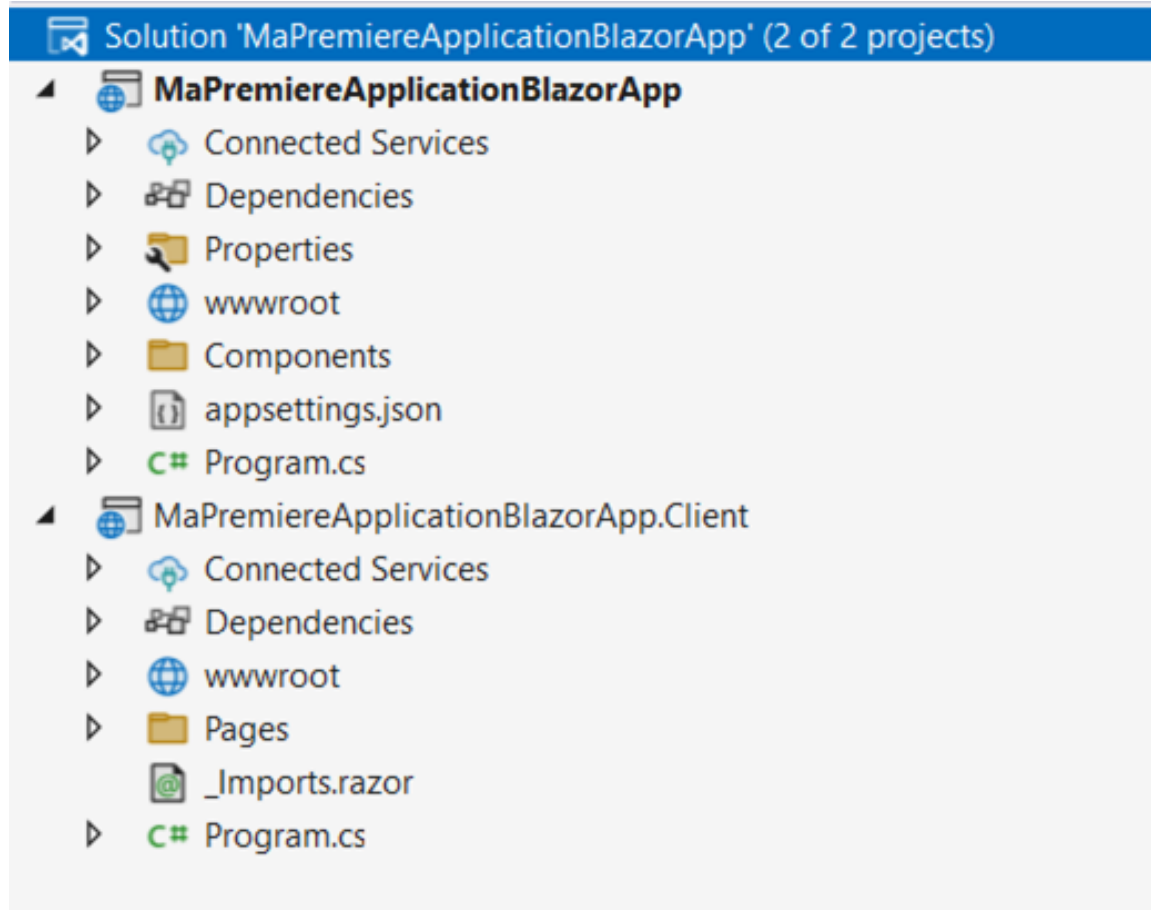
Création du projet

- **Interactivity location**
 - **Impact sur le développement :**
 - **Per page/component** offre plus de contrôle mais nécessite de :
 - Décider du mode de rendu pour chaque composant
 - Utiliser des directives comme @rendermode InteractiveServer ou @rendermode InteractiveWebAssembly
 - Gérer potentiellement différents modes dans la même application
 - **Global** simplifie le développement en appliquant le même comportement partout, mais peut être moins optimal en termes de performances pour des pages qui n'ont pas besoin d'interactivité.



Le choix "Per page/component" est généralement recommandé pour sa flexibilité.

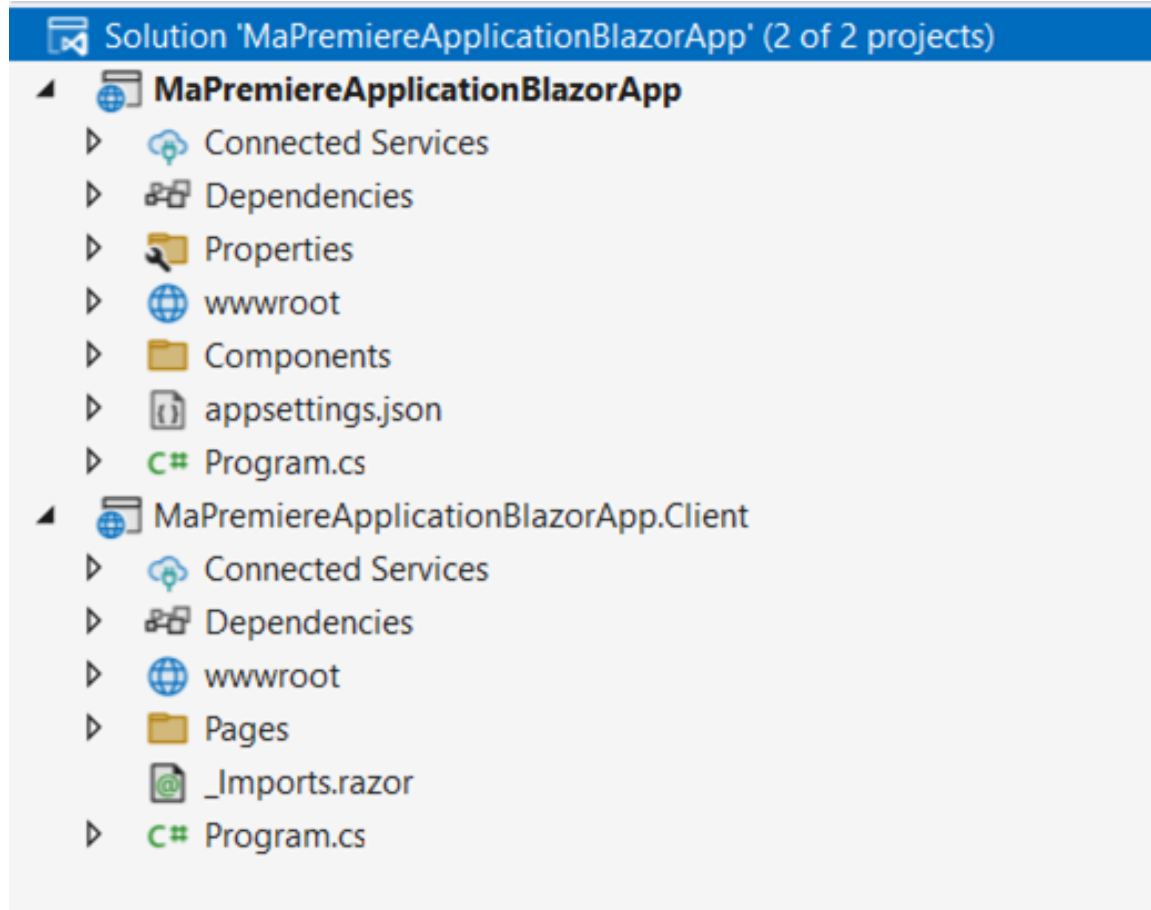
Structure de l'application de base



MaPremiereApplicationBlazorApp

- Projet serveur qui gère le rendu côté serveur
- Contient la logique métier, les services et la configuration
- Éléments clés :
 - Components/ : Composants partagés
 - appsettings.json : Configuration de l'application
 - Program.cs : Point d'entrée et configuration des services
 - wwwroot/ : Fichiers statiques (CSS, JS, images)

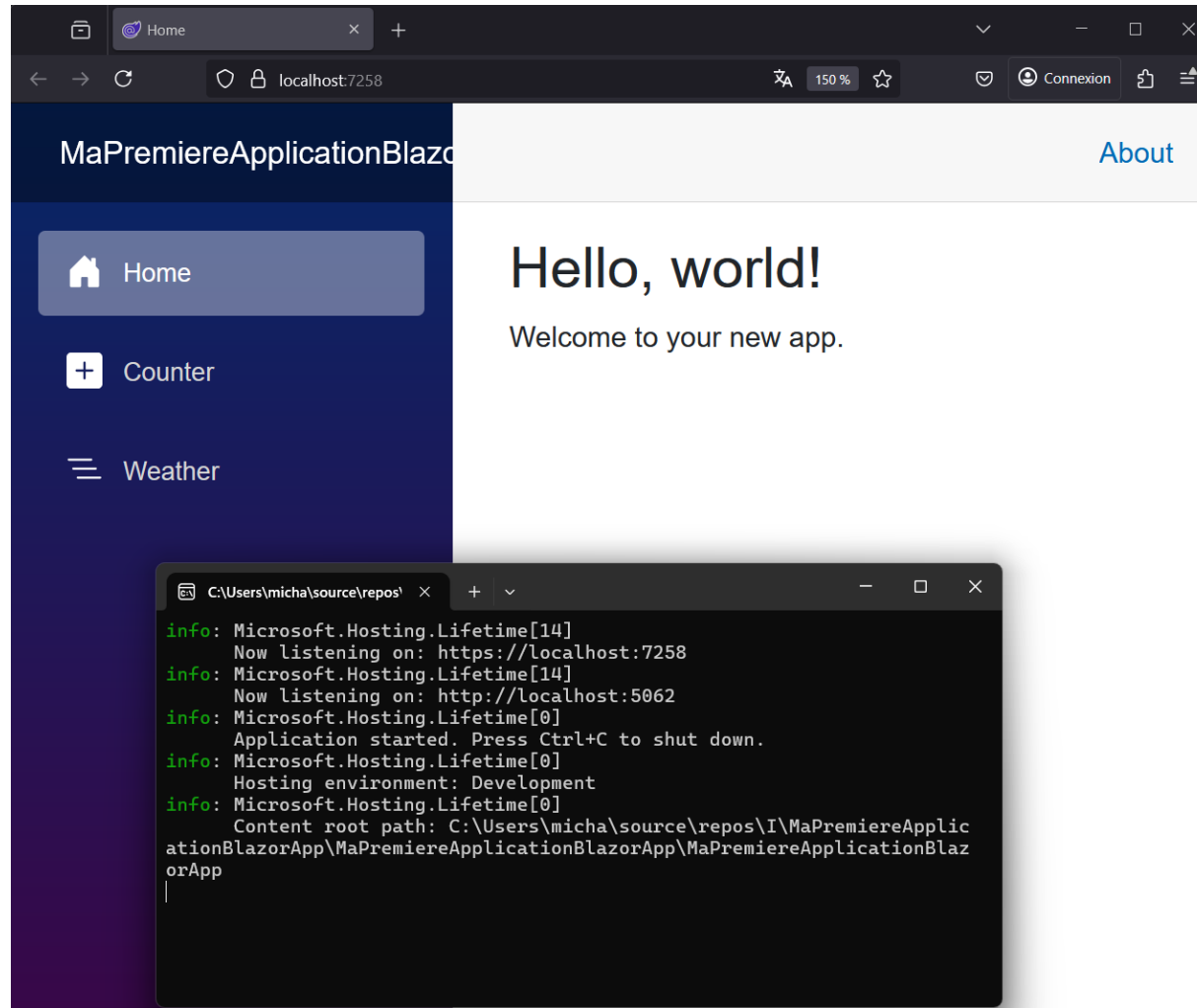
Structure de l'application de base



MaPremiereApplicationBlazorApp.Client

- Projet WebAssembly pour l'exécution côté client
- Contient uniquement ce qui doit s'exécuter dans le navigateur
- Éléments spécifiques :
 - Pages/ : Pages spécifiques au client
 - _Imports.razor : Imports globaux pour Razor
 - Program.cs : Point d'entrée et configuration des services

Lancement de notre APP



Components

Blazor

Qu'est-ce qu'un composant Blazor ?

Un composant Blazor est l'élément fondamental de toute application Blazor, comparable à une pièce de Lego dans votre construction web. Il s'agit d'un bloc réutilisable et indépendant qui représente une partie spécifique de votre interface utilisateur.

Techniquement, c'est un fichier Razor (avec l'extension .razor) qui encapsule :

- Du balisage HTML pour définir la structure
- Du code C# pour la logique métier
- Du CSS pour le style (intégré ou référencé)

Imaginez chaque composant comme une mini-application autonome capable d'être instancié à volonté dans votre projet.



Pourquoi utiliser des composants ?



Réutilisabilité

Créez une fois, utilisez partout !
Un composant bien conçu peut être réutilisé dans différentes parties de votre application, voire dans plusieurs projets.

Cela réduit considérablement la duplication de code et accélère le développement.



Maintenabilité

Lorsqu'un bug survient ou qu'une amélioration est nécessaire, vous n'avez qu'à modifier le composant à un seul endroit.

Cette centralisation facilite la maintenance et réduit les risques d'erreurs.



Modularité

Divisez votre interface en petites unités logiques qui peuvent être développées, testées et déboguées indépendamment.

Cette approche modulaire simplifie la gestion de projets complexes.

Les deux grand types de composants



1

Composants "Page" (Rootable)

Accessibles directement via une URL, ils constituent les points d'entrée de votre application. Ils sont identifiables par la directive **@page** au début du fichier.

2

Composants "Réutilisables" (Non-Rootable)

Ils ne peuvent pas être accédés directement par une URL et sont conçus pour être intégrés dans d'autres composants ou pages. Ils n'ont pas de directive **@page**.

Dans l'écosystème Blazor, on distingue deux catégories principales de composants, chacune ayant son rôle spécifique dans l'architecture de votre application :

Cette distinction fondamentale organise naturellement votre application en niveaux logiques :

les pages définissent la structure globale et le routage, tandis que les composants réutilisables enrichissent ces pages avec des fonctionnalités spécifiques.

C'est comme si les pages étaient les chapitres d'un livre, et les composants réutilisables les paragraphes et illustrations qui composent ces

chapitres.

Bonnes Pratiques

Blazor

Conventions de Nommage



Composants

Utilisez le PascalCase pour nommer vos composants (ex: **UserProfile.razor**, **OrderDetails.razor**). Cette convention est cohérente avec celle des classes C# et facilite l'identification des composants.



Événements

Utilisez le suffixe "Changed" pour les événements de modification de valeur (ex: **ValueChanged**) et "Requested" pour les actions (ex: **SaveRequested**).



Paramètres

Nommez les paramètres des composants en PascalCase, précédés de l'attribut [Parameter] (ex: **public string UserName { get; set; }**). Cela maintient la cohérence avec les propriétés C# standard.



Services

Préfixez les interfaces de services avec "I" (ex: **IUserService**) et utilisez le suffixe "Service" pour les implémentations (ex: **UserService**).

Organisation des Fichiers Blazor



Fichiers .razor

Contient le balisage HTML, les directives et éventuellement du code C# inline avec `@code{}`. Idéal pour les composants simples ou purement visuels.



Fichiers .razor.cs

Contient la logique du composant via le modèle code-behind. Permet de séparer clairement le balisage de la logique pour les composants complexes.



Fichiers .razor.css

Styles isolés spécifiques au composant. Utilise l'encapsulation CSS pour éviter les conflits de style entre composants.



Fichiers de test

Placez les tests dans un projet séparé suivant la structure du projet principal pour faciliter la maintenance et assurer une couverture complète.

Structure des Projets Blazor



Components

Organisez en sous-dossiers par fonctionnalité ou domaine



Services

Interfaces et implémentations pour la logique métier



Models

Classes de données et DTOs pour les échanges

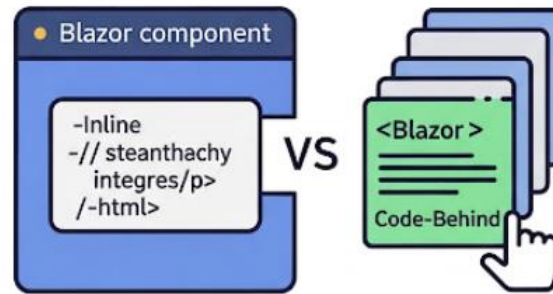
- Préférez une organisation par fonctionnalité plutôt que par type technique. Par exemple, regroupez tous les éléments liés à la "Gestion des Utilisateurs" dans un même dossier
- Pour les projets de taille moyenne à grande, envisagez une structuration en couches ou une architecture modulaire.
- Vous pouvez créer des bibliothèques de classes Razor (.razor) distinctes pour les fonctionnalités réutilisables ou les modules métier spécifiques.

Code-Behind vs Code Inline

Code Inline (@code{})

Le code C# est intégré directement dans le fichier .razor à l'aide du bloc `@code{}`.

- Idéal pour les composants simples
- Tout au même endroit (pratique pour les débutants)
- Accès direct aux éléments du balisage
- Devient difficile à gérer avec la complexité



Code-Behind (.razor.cs)

Le code C# est placé dans un fichier séparé qui partage le même nom que le composant.

- Séparation claire des responsabilités
- Meilleure organisation pour les composants complexes
- Facilite les tests unitaires
- Nécessite une classe partielle

Pour les applications professionnelles, le modèle code-behind est généralement recommandé car il favorise une meilleure séparation des préoccupations et facilite la maintenance

TIPS :
si votre bloc `@code` dépasse 50 lignes, envisagez de migrer vers l'approche code-behind pour améliorer la lisibilité et la maintenabilité.

Le choix d'architecture

Le choix de la bonne architecture est primordiale lorsque nous désirons avoir une application performante et maintenable

Problèmes Courants

- Composants monolithiques avec trop de responsabilités
- Logique métier mélangée au code d'interface
- Absence de réutilisation des composants
- Dépendances directes entre composants

Solutions

- Décomposition en composants plus petits et spécialisés
- Extraction de la logique métier dans des services
- Création de composants paramétrables et réutilisables
- Utilisation de l'injection de dépendances

Résultats

- Code plus lisible et maintenable
- Tests unitaires facilités
- Meilleure séparation des responsabilités
- Réduction de la duplication de code

Razor

Blazor

Qu'est-ce que Razor ?

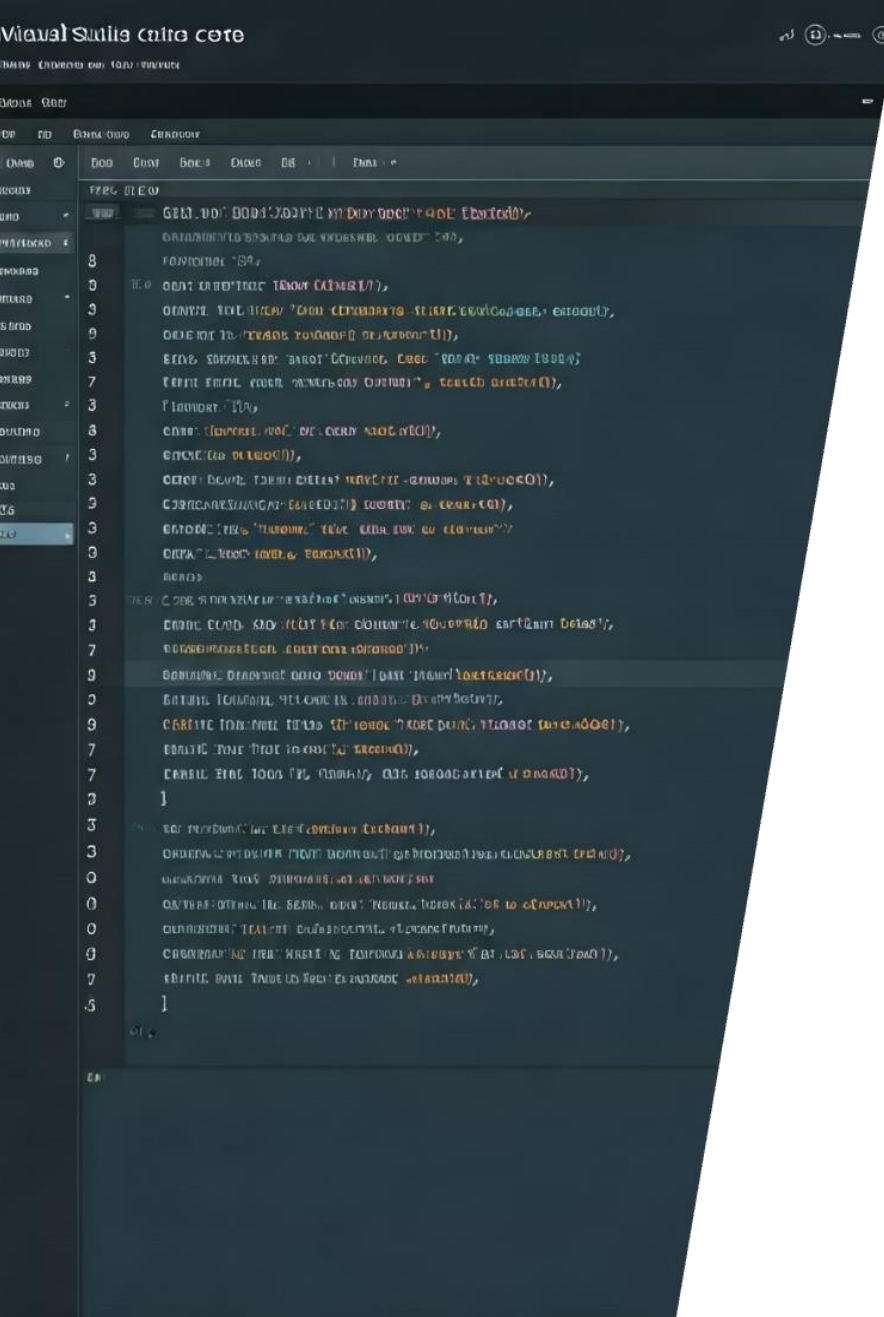
Razor est un moteur de rendu côté serveur permettant de créer des pages web dynamiques.

Il s'agit d'une syntaxe de templating qui combine de façon élégante le **HTML** et le **C#**, facilitant considérablement le développement d'applications web interactives.

Razor utilise le **symbole @** pour basculer entre le HTML et le C#, offrant une transition entre le **balisage** et la **logique d'application**.

Cette syntaxe hybride permet d'intégrer directement des expressions, des variables et des structures de contrôle dans vos pages.

Contrairement au HTML **statique**, Razor permet la génération **dynamique** de contenu, l'utilisation de données provenant de bases de données, la réutilisation de composants, et la mise en œuvre d'une **logique métier**.



Structure d'un fichier Razor



Extension de fichier

Les composants Razor utilisent l'extension **.razor** au lieu de **.html**, indiquant au serveur qu'ils contiennent à la fois du HTML et du code C#.



Zone de balisage HTML

La partie supérieure du fichier contient généralement le balisage HTML standard, avec la possibilité d'insérer des expressions C# à l'aide du symbole **@**.



Directive @code

La section **@code** contient les propriétés, méthodes et événements C# qui définissent le comportement du composant, isolant clairement la logique du balisage.



Directives spéciales

Des directives comme **@page**, **@using** et **@inject** permettent de configurer le routage, les espaces de noms et l'injection de dépendances.

Transformation HTML → Razor

Html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Liste d'utilisateurs</title>
</head>
<body>
  <h1>Utilisateurs</h1>
  <ul>
    <li>Jean Dupont</li>
    <li>Marie Martin</li>
    <li>Paul Durand</li>
  </ul>
</body>
</html>
```

Razor

```
@page "/utilisateurs"
<h1>Utilisateurs</h1>
<ul>
  @foreach (var user in users)
  {
    <li>@user.Nom @user.Prenom</li>
  }
</ul>
@code {
  private List<User> users = new()
  {
    new User("Dupont", "Jean"),
    new User("Martin", "Marie"),
    new User("Durand", "Paul")
  };
}
```

Cette transformation illustre comment passer d'une liste statique codée en dur à une liste générée dynamiquement à partir d'une collection d'objets.

Remarquez l'utilisation du **@foreach** pour itérer sur la collection et de l'interpolation **@** pour afficher les propriétés de chaque utilisateur.

Interpolation de Données avec @



Variables simples

L'opérateur @ permet d'insérer directement la valeur d'une variable dans le HTML.

Exemple : `<p>Bonjour, @userName !</p>` où
userName est une variable C# définie dans la section
@code.



Expressions et formatage

Utilisez des expressions complexes avec des parenthèses :

`<p>Total : @(price * quantity)€</p>`

ou appliquez un formatage :

`<p>Date : @date.ToString("dd/MM/yyyy")</p>`.



Propriétés d'objets

Accédez aux propriétés d'objets complexes avec la notation pointée : `<p>Client : @customer.FirstName
@customer.LastName</p>`.

Razor résoudra automatiquement ces expressions lors du rendu.



Échappement HTML

Par défaut, Razor échappe les valeurs pour prévenir les attaques XSS.

Pour insérer du HTML brut, utilisez `@Html.Raw()` mais avec précaution : `@Html.Raw(htmlContent)`.

Différences Clés HTML vs Razor

Fonctionnalité

HTML Statique

Razor Dynamique

Attributs

Valeurs fixes :
`class="button"`

Valeurs dynamiques :
`class="@buttonClass"`

Classes CSS

Définies à l'avance

Conditionnelles :
`class="@ (isActive ? "active" : "")"`

Événements

Via JavaScript séparé

Intégrés :
`@onclick="HandleClick"`

Visibilité

Toujours visible ou caché via CSS

Conditionnelle :
`@if (isVisible) { ... }`

Répétition

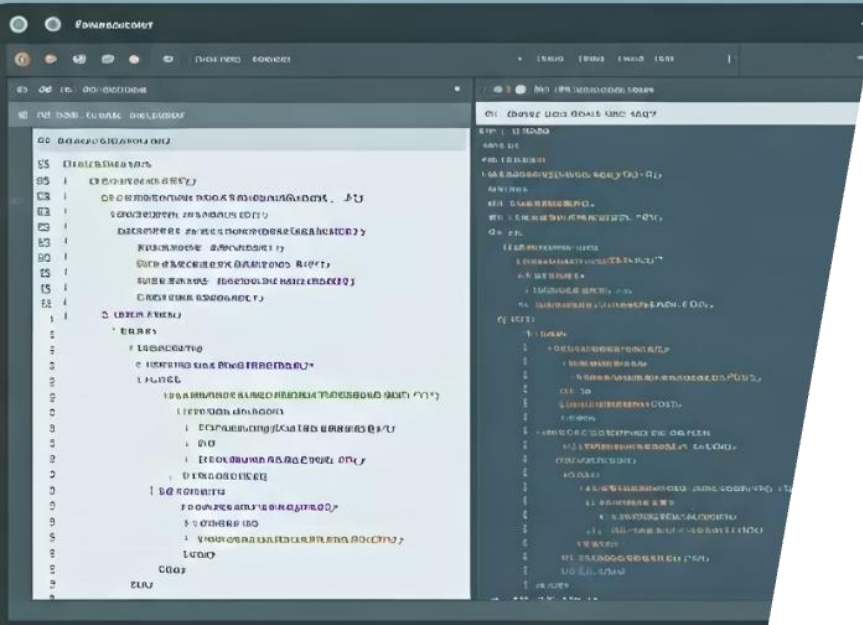
Copier-coller manuel

Boucles : `@foreach`, `@for`

Rendu partiel

Non supporté nativement

Composants réutilisables



Razor – Les directives

Blazor

Razor : Les directives

Qu'est-ce qu'une directive?

Une directive Razor est une instruction spéciale qui modifie la façon dont le moteur Razor traite et génère le contenu HTML.

Toujours précédée du symbole @, elle indique au compilateur d'exécuter une action spécifique plutôt que d'afficher du texte brut.

Syntaxe générale

Les directives suivent généralement le format @directive où "directive" est remplacé par l'instruction spécifique.

Certaines directives nécessitent des accolades `{ }` pour délimiter leur portée, tandis que d'autres utilisent des parenthèses `()` pour leurs paramètres.

Nous sommes donc proche de la syntaxe

C#

Catégories principales

Les directives Razor se divisent en plusieurs catégories :

- conditionnelles (@if, @else)
- boucles (@for, @foreach)
- routage (@page)
- déclaration de modèle (@model)
- inclusion de code (@using).

Chacune répond à des besoins spécifiques dans la création de vues dynamiques.

Directives de rendu conditionnel

</>

Syntaxe des conditionnelles

```
@if (test1)
{ <p>Le test1 est correct</p> }
@else if (test2)
{ <p>Le test1 est incorrect mais le test2 est correct</p> }
@else { <p>Aucun des test n'est correct</p> }
```



Contrôle du flux d'affichage

Ces directives permettent de déterminer quel contenu HTML sera généré en fonction des conditions d'exécution



Applications pratiques

Affichage de messages d'erreur conditionnels, personnalisation du contenu selon le rôle de l'utilisateur, adaptation de l'interface selon les données disponibles ...

Directives de boucles et itération



@for

Idéal pour les itérations avec compteur.

Syntaxe:

```
@for (int i = 0; i < Model.Count; i++) { ... }.
```

Permet un accès facile à l'index et un contrôle précis sur le nombre d'itérations.



@foreach

Parfait pour parcourir des collections entières.

Syntaxe:

```
@foreach (var item in Model.Items) { ... }.
```

Plus simple que @for lorsque l'index n'est pas nécessaire.



@while

Utilisé pour les itérations basées sur une condition.

Syntaxe:

```
@while (condition) { ... }.
```

Moins courant dans Razor mais utile pour certains scénarios spécifiques.

Directive @page et routage



Transformation en endpoint

@page transforme un fichier Razor en point d'accès direct via URL



Routes personnalisées

Paramétrage flexible: @page "/produit/{id:int}"



Navigation structurée

Organisation hiérarchique des pages de l'application

La directive @page constitue le fondement du système de routage dans les Razor Pages.

Elle permet non seulement de rendre une page accessible via une URL spécifique, mais aussi de définir des contraintes sur les paramètres (comme ":int" pour forcer un entier) et de créer une architecture de navigation cohérente.



Autres directives utiles



@model

Définit le type de modèle fortement typé pour la page.

Exemple: @model ProductViewModel.

Cette directive permet l'intellisense et la vérification de type lors de la compilation, réduisant ainsi les erreurs potentielles lors de l'exécution.



@using

Importe des espaces de noms C# dans la vue, similaire à la directive using en C#.

Exemple: @using System.Collections.Generic.

Facilite l'accès aux classes et méthodes sans qualification complète du nom.



@inject

Permet l'injection de dépendances directement dans une vue.

Exemple: @inject IProductService ProductService.

Très utile pour accéder aux services sans passer par le modèle de page.

Routing & Layout

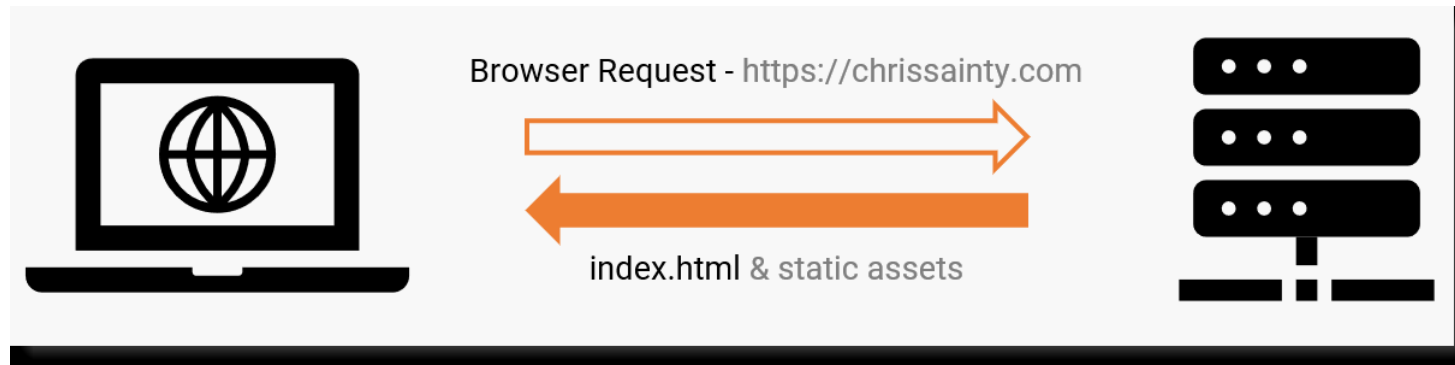
Routing

Blazor prend en charge le routage d'URL.

Le routage d'URL vous permet de configurer une application pour accepter les URL de demande qui ne correspondent pas à des fichiers physiques mais utilisent une route sémantiquement significative.

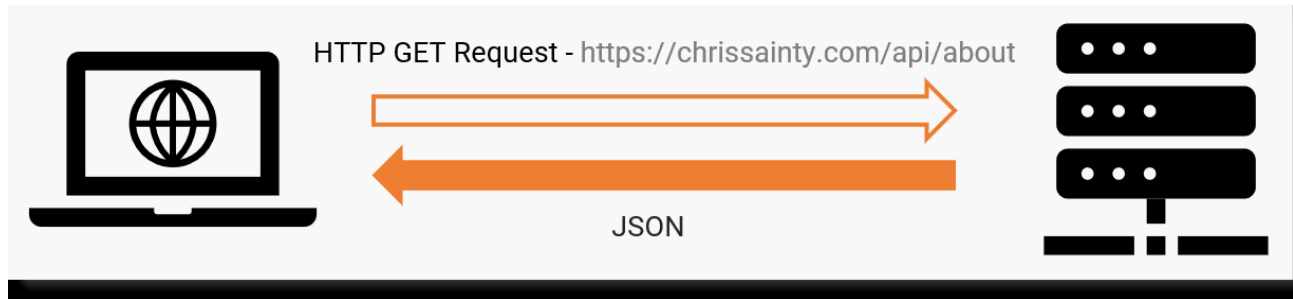
Lors du chargement initial d'un site SPA ce n'est pas très différent des applications Web traditionnelles:

Nous effectuons la requête HTTP GET, puis téléchargeons le HTML, le CSS, le JavaScript et tout autre élément statique.



Routing

La différence apparaît lors des changements de page par la suite :



Cette fois, la SPA ne charge que les données du **serveur**.

En effet, les applications SPA ont tendance à télécharger l'intégralité de l'application lors du premier chargement du site, donc tout est déjà là. Lors du changement de page, le seul contenu supplémentaire requis est les données à afficher.

En bref

Navigation Traditionnelle

Dans le modèle classique, chaque interaction utilisateur déclenche une requête vers le serveur qui retourne une page HTML complète. Ce processus entraîne un rechargement visible et souvent un temps d'attente perceptible. L'utilisateur voit son écran se vider puis se remplir à nouveau, créant une expérience fragmentée et moins fluide.

Navigation SPA

Les SPA chargent une seule page HTML initiale, puis toutes les mises à jour suivantes sont gérées côté client via JavaScript. Les données sont récupérées en arrière-plan, souvent au format JSON.

Cette approche réduit considérablement les requêtes serveur et accélère les transitions entre les vues, offrant une expérience plus réactive.

En bref

Navigation côté serveur

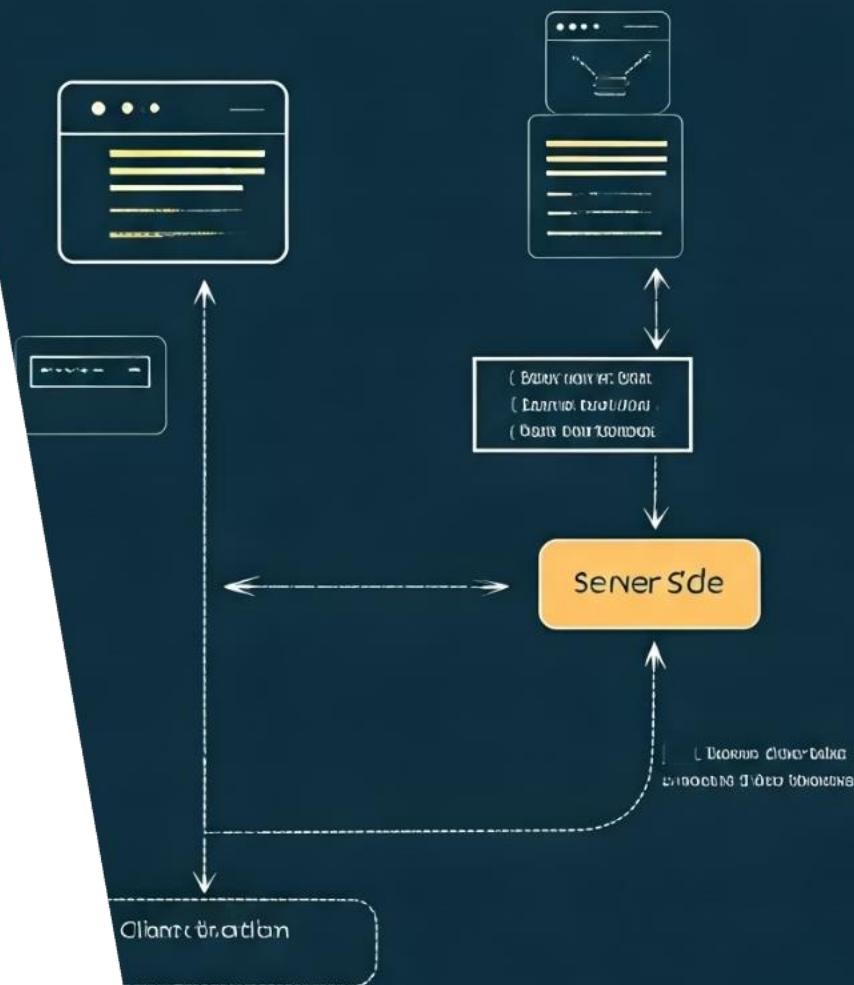
- Chaque navigation génère une requête complète au serveur
- Le serveur traite la requête et renvoie une page HTML entière
- Approche traditionnelle avec rechargement visible

Navigation côté client (SPA/Blazor)

- JavaScript intercepte les clics sur les liens
- Récupération asynchrone des données nécessaires
- Mise à jour partielle du DOM sans rechargement

Avantages de la navigation SPA

- Ressenti applicatif fluide et réactif
- Réduction de la bande passante utilisée
- Expérience utilisateur proche d'une application native

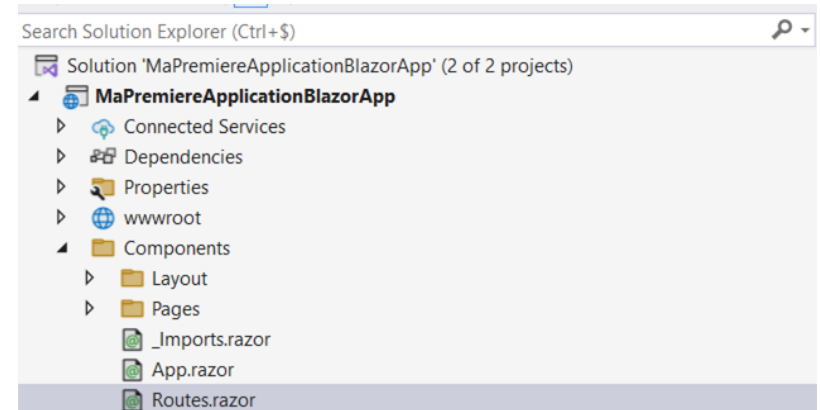


Router Component

Le « Router Component » permet le routage vers les composants Razor de l'application

Expli

```
<Router AppAssembly="typeof(Program).Assembly" AdditionalAssemblies="new[]
{ typeof(Client._Imports).Assembly }">
  <Found Context="routeData">
    <RouteView RouteData="routeData" DefaultLayout="typeof
      (Layout.MainLayout)" />
    <FocusOnNavigate RouteData="routeData" Selector="h1" />
  </Found>
  <NotFound>
    <PageTitle>Not found</PageTitle>
    <LayoutView Layout="@typeof(MainLayout)">
      <p role="alert">Sorry, there's nothing at this address.</p>
    </LayoutView>
  </NotFound>
</Router>
```



Processus de Routage

Le processus se déroule en trois étapes principales :

1. Compilation du Composant

Quand un composant Razor (.razor) contient une directive `@page`, il est compilé. Cette compilation génère une classe de composant à laquelle est automatiquement ajouté un `RouteAttribute`.

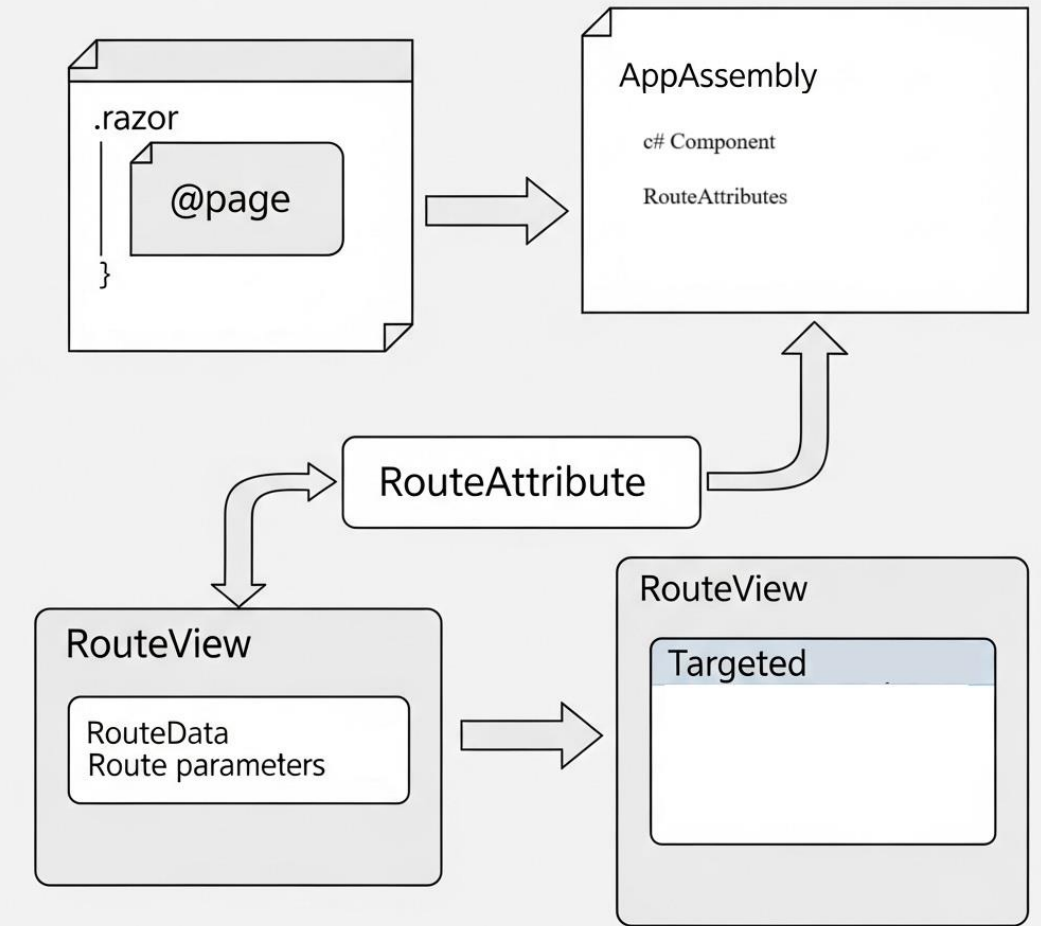
2. Analyse des Itinéraires au Démarrage de l'Application

Au démarrage de l'application, l'assemblage désigné comme `AppAssembly` (celui qui contient le routeur) est scanné.

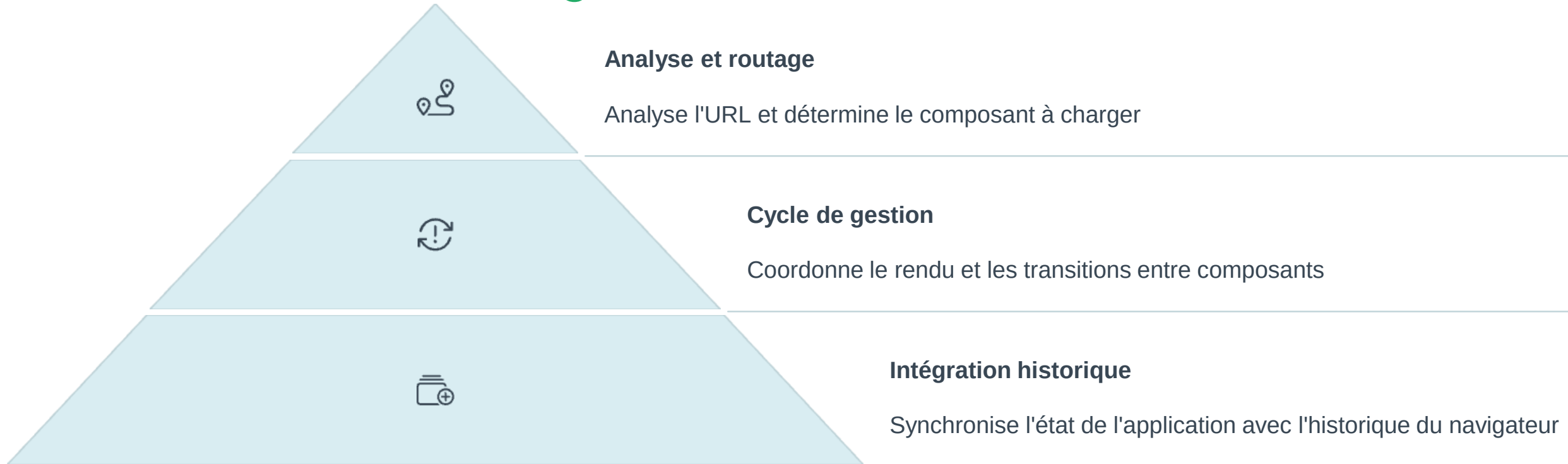
3. Affichage du Composant par `RouteView` à l'Exécution

Au moment de l'exécution, le composant `RouteView` entre en jeu. Il reçoit le `RouteData` (les informations sur l'itinéraire et les paramètres de route) du routeur. Fort de ces informations, `RouteView` est alors capable de rendre le composant approprié

Razor Component and Routing



Processus de Routage



Le composant Router est l'élément central de la navigation dans Blazor. Il joue un rôle essentiel en interceptant les changements d'URL et en coordonnant le chargement du composant approprié, tout en maintenant la synchronisation avec l'historique du navigateur.

Cette architecture permet une séparation claire entre la logique de navigation et la logique métier des composants, facilitant ainsi la maintenance et l'évolution de l'application.

Directive @page



La directive @page définit qu'un composant est une page routable

```
@page "/"

<PageTitle>Home</PageTitle>

<h1>Hello, world!</h1>

Welcome to your new app.
```

Un composant peut répondre à plusieurs URLs :

```
@page "/"
@page "/Home"
@page "/Index"

<PageTitle>Home</PageTitle>

<h1>Hello, world!</h1>

Welcome to your new app.
```



Dans les bonnes pratiques il est recommandé de suivre le principe d'hierarchie pour vos routes

```
<!-- Pages/Clients/Index.razor -->
@page "/clients"

<!-- Pages/Clients/Details.razor -->
@page "/clients/details"

<!-- Pages/Clients/Nouveau.razor -->
@page "/clients/nouveau"
@page "/clients/creer"
```

Directive @page



La directive @page définit qu'un composant est une page routable

```
@page "/"

<PageTitle>Home</PageTitle>

<h1>Hello, world!</h1>

Welcome to your new app.
```

Un composant peut répondre à plusieurs URLs :

```
@page "/"
@page "/Home"
@page "/Index"

<PageTitle>Home</PageTitle>

<h1>Hello, world!</h1>

Welcome to your new app.
```

Dans les bonnes pratiques il est recommandé de suivre le principe d'hierarchie pour vos routes

```
<!-- Pages/Clients/Index.razor -->
@page "/clients"

<!-- Pages/Clients/Details.razor -->
@page "/clients/details"

<!-- Pages/Clients/Nouveau.razor -->
@page "/clients/nouveau"
@page "/clients/creer"
```

Directive @page : Paramètres de route



Les urls statiques ne sont utiles que pour les contenus statiques.

Il est cependant plus qu'intéressant de permettre à un même composant d'avoir un rendu différent basé sur les informations de l'url

Coté Razor, le paramètre est définit entre `{ }` et du côté du Code-behind celui-ci est lié à une propriété via la notation **[parameter]**

```
Client.razor 1 @page "/client/{ClientId}"
2 <h3>Client</h3>
3
4 <b>L'id du client = </b>@ClientId
5
6
7

Client.razor.cs 1 using Microsoft.AspNetCore.Components;
2
3 namespace MaPremiereApplicationBlazorApp.Components.Pages
4 {
5     1 reference
6     public partial class Client
7     {
8         [Parameter]
9         1 reference
10        public string ClientId { get; set; }
11    }
12 }
```

Directive @page : Paramètres de route - Contraintes



Il est également possible de mettre en place des critères sur les paramètres afin de s'assurer du bon passage de celui-ci

Pour définir une contrainte pour un paramètre, il est suffixé de : , puis le type de contrainte.

```
Client.razor* - X
MaPremiereApplicationBlazor -
1 @page "/client/{ClientId:int}"
2 <h3>Client</h3>
3
4 <b>L'id du client = </b>@ClientId
5
6
7

Client.razor.cs* - X
MaPremiereApplicationBlazorApp - MaPremiereApplicationBlazorApp.Components.Pages - ClientId
1 using Microsoft.AspNetCore.Components;
2
3 namespace MaPremiereApplicationBlazorApp.Components.Pages
4 {
5     1 reference
6     public partial class Client
7     {
8         [Parameter]
9         1 reference
10        public int? ClientId { get; set; }
11    }
12 }
```

Directive @page : Paramètres de route - Contraintes



Vous pouvez donc contraindre les paramètres pour respecter les valeurs possibles du type .net qui sera déclaré pour la propriété

:bool true, false

:datetime 2025-05-05 , 02-09-2024,...

:decimal 2.34 0.45

Remarque :
Le séparateur est un .

:double 2.34 0.45

Remarque :
Le séparateur est un .

:float 2.34 0.45

Remarque :
Le séparateur est un .

:int -1 , 256, ...

:guid 99303dc9-8c76-42d9-9430-de3ee1ac25d0

:long -1 , 299792458

Remarque :

- Les contraintes Blazor ne prennent pas en charge la localisation.

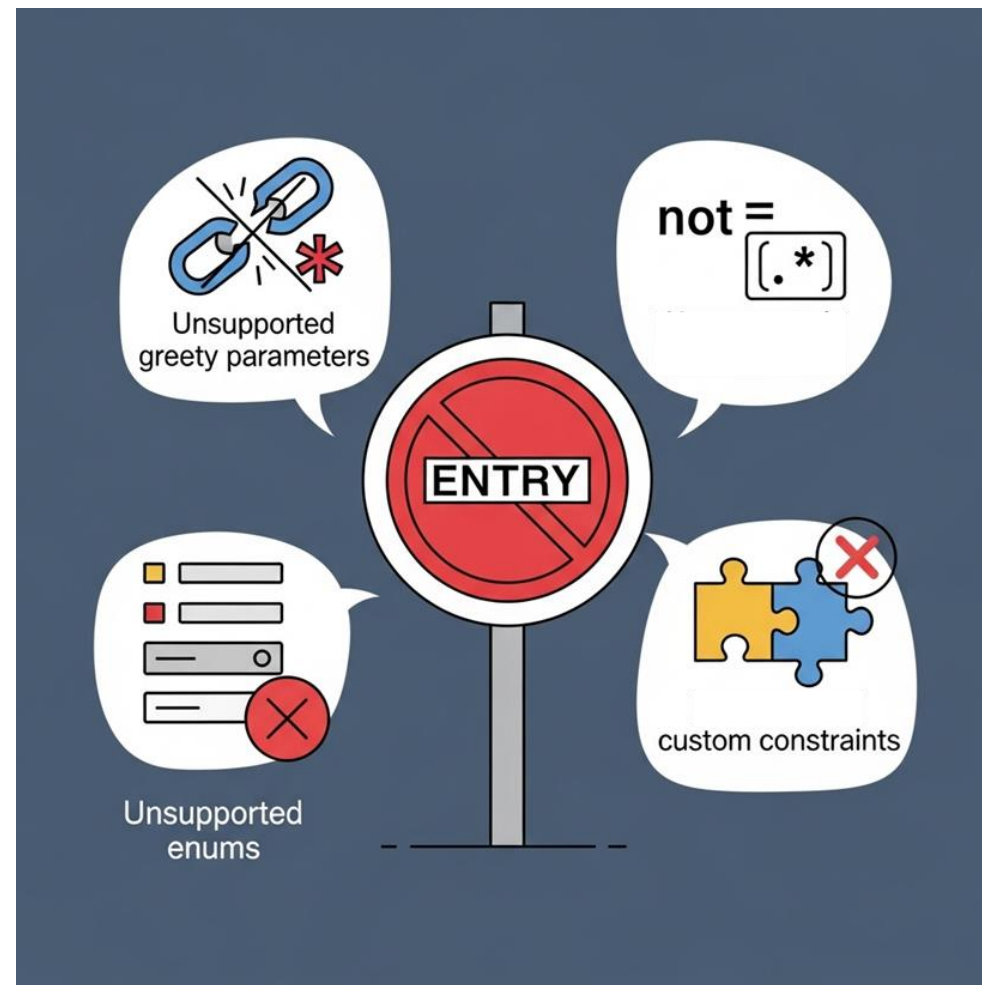
Il faut donc faire attention en ce qui concerne les nombres (Notation Anglaise) et les dates (MM-dd-yyyy, MM-dd-yy ou en format iso yyyy-MM-dd)

Directive @page : Paramètres de route - Contraintes



Blazor ne supportent pas (actuellement) :

- L'utilisation de l'* pour capturer un ensemble de route
- Les expressions régulières pour contraindre un paramètre
- Les enums pour contraindre un paramètre
- Les contraintes personnalisées



Directive @page : Paramètres de route - Exemples



<!-- Contraintes numériques -->

```
@page "/article/{id:int}"           <!-- Entier -->
@page "/prix/{montant:decimal}"     <!-- Décimal -->
@page "/note/{valeur:double}"       <!-- Double -->
@page "/page/{numero:long}"         <!-- Long -->
```

<!-- Contraintes de chaîne -->

```
@page "/profil/{nom:alpha}"         <!-- Lettres uniquement -->
@page "/recherche/{terme:minlength(3)}" <!-- Longueur minimum -->
@page "/code/{valeur:length(5)}"     <!-- Longueur exacte -->
```

<!-- Contraintes de date -->

```
@page "/evenement/{date:datetime}" <!-- Date/heure -->
```

<!-- Contraintes booléennes -->

```
@page "/actif/{statut:bool}"        <!-- true/false -->
```

<!-- Contraintes GUID -->

```
@page "/session/{id:guid}"          <!-- GUID -->
```

Directive @page : Paramètres de route - Optionnel



Les paramètres optionnels ne sont pas explicitement supportés par Blazor.

Pour contourner cette restriction, il nous suffit cependant de déclarer plusieurs routes pour une même page.

Si nous désirons avoir une valeur par défaut pour le paramètre, nous pouvons utiliser la méthode

SetParameterAsync (voir plus loin le cycle de vie)

```
[Parameter]
public int? CurrentCount { get; set; }

void IncrementCount()
{
    CurrentCount++;
}

public async override Task SetParametersAsync(ParameterView parameters)
{
    await base.SetParametersAsync(parameters);
    CurrentCount = CurrentCount ?? 1;
}
```

```
@page "/"counter"
@page "/"counter/{CurrentCount:int}"
@rendermode InteractiveAuto

<PageTitle>Counter</PageTitle>

<h1>Counter</h1>

<p role="status">Current count: @currentCount</p>

<button class="btn btn-primary" @onclick=IncrementCount>Click me</button>

<ul>
    <li><a href="/counter/42">Navigate to /counter/42</a></li>
    <li><a href="/counter/123">Navigate to /counter/123</a></li>
    <li><a href="/counter/">Navigate to /counter</a></li>
</ul>

@code {
    [Parameter]
    public int CurrentCount { get; set; } = 0

    void IncrementCount()
    {
        CurrentCount++;
    }
}
```

Navigation

Blazor

Gestion de l'état de L' URL et du navigateur



Synchronisation URL-Contenu

L'URL reflète constamment l'état actuel de l'application sans nécessiter de rechargement complet. Cette synchronisation automatique garantit que l'URL et le contenu affiché restent cohérents.



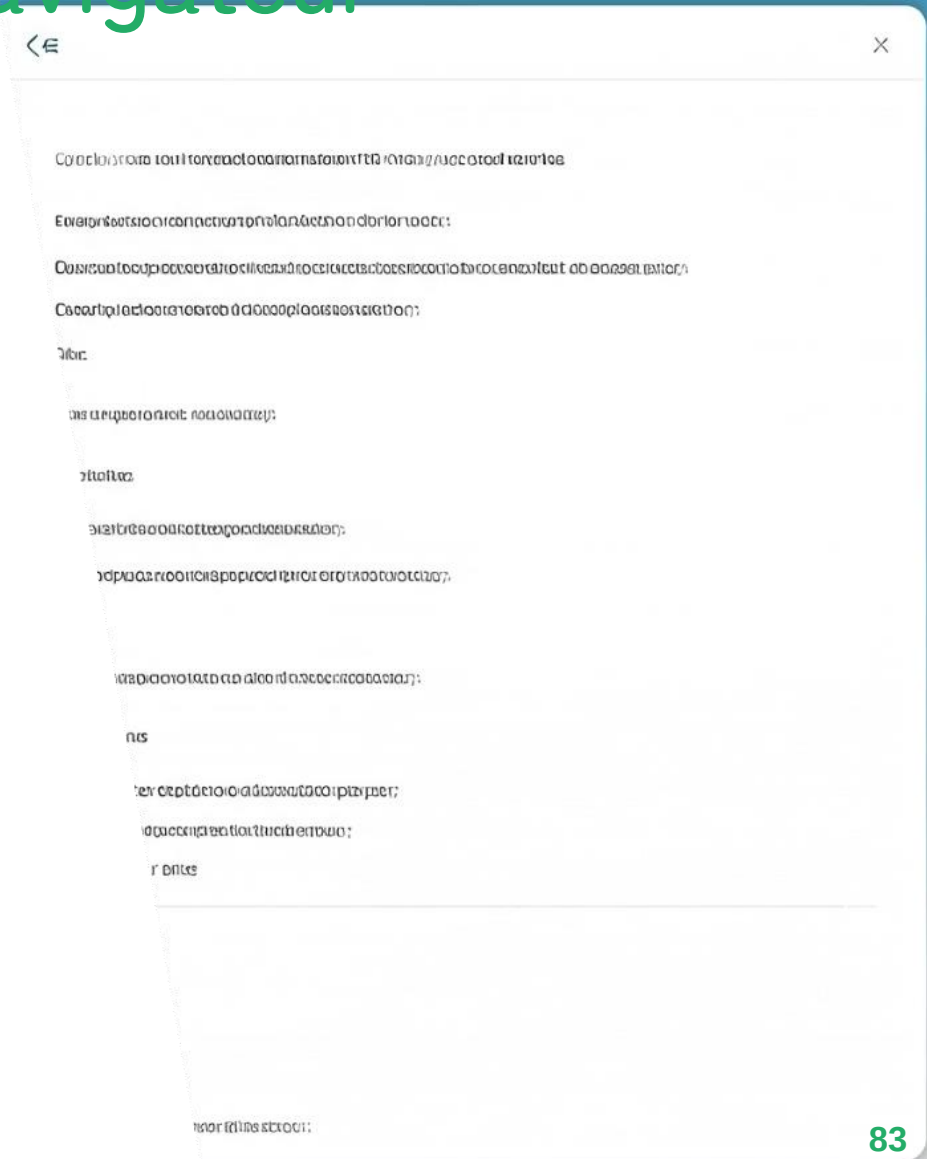
Favoris et partage

Les utilisateurs peuvent mettre en favori ou partager des URL spécifiques, permettant l'accès direct à un état particulier de l'application SPA via le routage client.



Historique préservé

L'historique de navigation est maintenu et respecté, permettant aux utilisateurs d'utiliser les boutons précédent/suivant du navigateur comme ils s'y attendent naturellement.

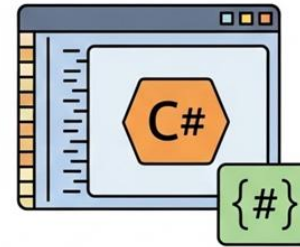


Navigation : Type de navigation

Navigation définie directement dans le markup avec des composants **<NavLink>** ou ****.
Méthode la plus simple et courante pour créer des liens statiques vers des pages spécifiques



Declarative Navigation
(links, buttons)



Programmatic Navigation (C# code)

Navigation contrôlée par le code C# en utilisant **`NavigationManager.NavigateTo("/page")`**.
Permet de naviguer conditionnellement selon la logique métier ou après des opérations asynchrones

Navigation déclenchée par les interactions utilisateur comme les clics, soumissions de formulaires. Combine les événements DOM (**@onclick**, **@onsubmit**) avec la logique de navigation par programmation



User Event Navigation



Automatic Navigation

Navigation automatique basée sur des conditions système (**authentication**, **timeouts**, **redirections**).
Utilise des intercepteurs, middleware ou logique conditionnelle pour rediriger sans action utilisateur.

NavLink

Navigation - Blazor

NavLink

Le composant NavLink est un composant Blazor intégré conçu spécifiquement pour la navigation.
Il est très similaire à une balise <a> HTML classique

Caractéristique	NavLink	
Fonctionnalité	Navigation vers une URL	Navigation vers une URL
Gestion d'état actif	Automatique (ajoute active class)	Manuelle (nécessite JS/CSS personnalisé)
Intégration Blazor	Native, comprend la détection de routage	Standard HTML, aucune intelligence Blazor
Complexité	Simple pour les liens de navigation	Peut devenir complexe pour les états actifs

En bref : NavLink simplifie la création de menus de navigation dynamiques dans Blazor, où l'élément de menu correspondant à la page actuelle est mis en évidence.

NavLink : Avantages et cas d'usage



Mise en évidence Automatique de la Page Active : C'est le principal avantage. NavLink détecte l'URL courante et ajoute une classe CSS (par défaut active) au lien correspondant. Cela permet une navigation intuitive pour l'utilisateur.



Simplicité de Code : Moins de code C# ou JavaScript est nécessaire pour gérer les états actifs des liens.



Performance : Optimisé pour le routage Blazor.

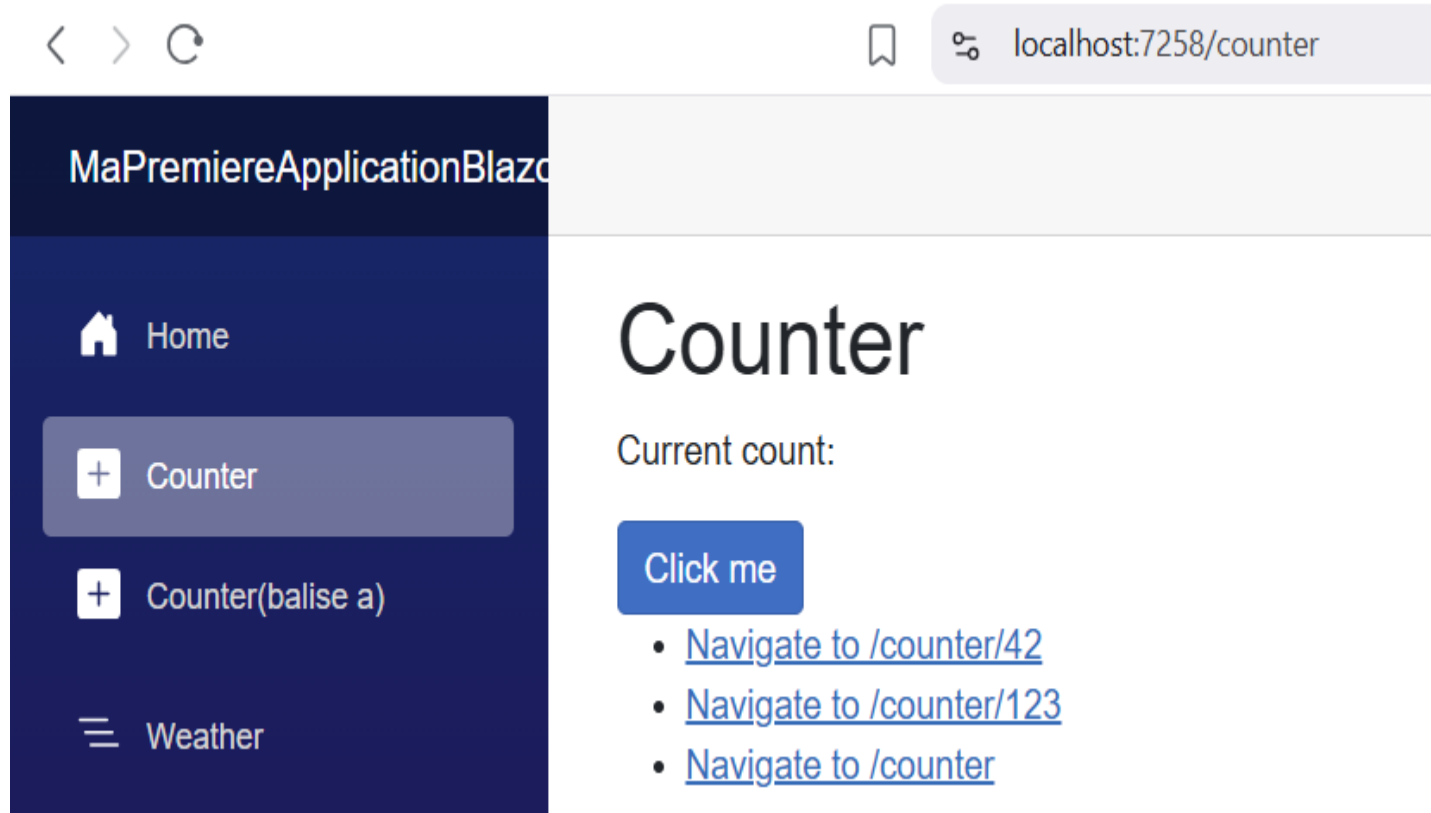


Accessibilité : Améliore l'expérience utilisateur en fournissant un feedback visuel clair sur la page actuellement visitée.

NavLink

Voici un exemple de la gestion automatique de l'état actif

```
<div class="nav-item px-3">  
  <NavLink class="nav-link" href="/counter">  
    <span class="bi bi-plus-square-fill-nav-menu" aria-hidden="true"></span> Counter  
  </NavLink>  
  <a href="/counter" class="nav-link "><span class="bi bi-plus-square-fill-nav-menu"></span>Counter(balise a)</a>  
</div>
```



NavLink - NavLinkMatch

Le NavLink dans Blazor utilise le paramètre **Match** permettant de déterminer quand mettre en surbrillance un élément de menu. Le paramètre **Match** accepte une valeur de l'énumération **NavLinkMatch**, qui a deux options possibles : **Prefix** et **All**.

- NavLink.Match.Prefix :

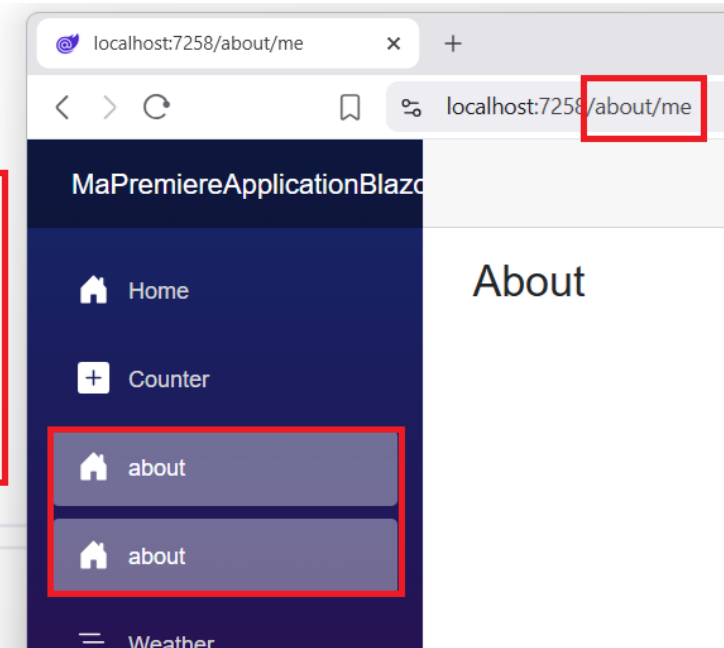
Un élément de menu sera mis en évidence si l'URL du navigateur commence par la valeur du Href de la NavLink.

```
<div class="nav-item px-3">
  <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
    <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span> Home
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" Match="NavLinkMatch.Prefix" href="counter">
    <span class="bi bi-plus-square-fill-nav-menu" aria-hidden="true"></span> Counter
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" Match="NavLinkMatch.Prefix" href="about">
    <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span> about
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" Match="NavLinkMatch.Prefix" href="about/me">
    <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span> about
  </NavLink>
</div>
```



NavLink - NavLinkMatch

- NavLink.Match.All :

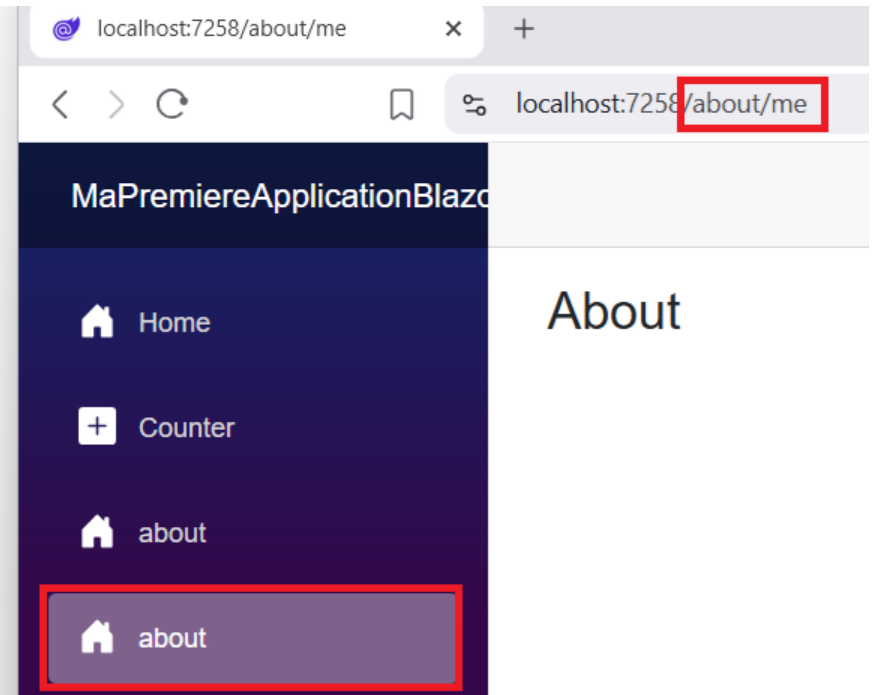
Un élément de menu sera mis en évidence si l'URL du navigateur est exactement la même que la valeur du Href de la NavLink.

```
<div class="nav-item px-3">
  <NavLink class="nav-link" Match="NavLinkMatch.Prefix" href="counter">
    <span class="bi bi-plus-square-fill-nav-menu" aria-hidden="true"></span> Counter
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" Match="NavLinkMatch.All" href="about">
    <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span> about
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" Match="NavLinkMatch.All" href="about/me">
    <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span> about
  </NavLink>
</div>

<div class="nav-item px-3">
  <NavLink class="nav-link" href="weather">
```



NavLink – Passage de paramètres

Pour passer un paramètre de vous devez d'abord le définir dans la directive `@page` de votre composant de destination et prévoir les propriétés de réception.

Exemple :

```
Article.razor  + X
C# MaPremiereApplicationBlazorApp.Client
1  @page "/"article"
2  @page "/article/{id:int}"
3  <h3>Article n° @Id</h3>
4
5  <h3>Article n° @ArticleId</h3>
6
7  @code {
8      [Parameter] //Paramètre de route
9      public int Id { get; set; }
10     [SupplyParameterFromQuery] //Paramètre d'URL
11     public int ArticleId { get; set; }
12 }
13
```

NavLink – Passage de paramètres

Ensuite, il faut construire l'URL de votre NavLink en y incluant la valeur du paramètre

```
<div class="nav-item px-3">
  <NavLink class="nav-link" href="article/1">
    <span class="bi bi-card-text" aria-hidden="true"></span> Article 1
  </NavLink>
</div>
<div class="nav-item px-3">
  <NavLink class="nav-link" href="article?ArticleId=25">
    <span class="bi bi-card-text" aria-hidden="true"></span> Article 25
  </NavLink>
</div>
```



localhost:7258/article/1



localhost:7258/article?ArticleId=25

Article n° 1

Article n° 0

Article n° 0

Article n° 25

Echappement et sécurité des URLs



Pourquoi l'échappement d'URL est-il critique ?



Injection XSS

Scripts malveillants injectés via les paramètres d'URL qui peuvent voler des données, détourner des sessions ou corrompre l'interface utilisateur



Open Redirect

Redirection automatique vers des sites malveillants qui peuvent tromper les utilisateurs et les conduire vers des pages de phishing



Manipulation de paramètres

Accès non autorisé à des ressources sensibles en modifiant les paramètres d'URL pour contourner les contrôles d'accès

Echappement et sécurité des URLs

Exemple :

```
// ❌ DANGEREUX - Paramètre non échappé
<NavLink href="@($"/profile?name={userName})">
    Profil de {userName}
</NavLink>

// Si userName = "admin<script>alert('XSS')</script>"
// L'URL devient : /profile?name=admin<script>alert('XSS')</script>
```

Blazor comprend un système de protection XSS intégré pour aider à prévenir les attaques XSS.

Il code toutes les chaînes définies en standard Razor (le @ directive) et pour afficher le code HTML sous forme de texte, le système remplace les caractères de service (par exemple < et >) avec leur équivalent « entity reference » (< et >').

Il est cependant recommandé de toujours échappé les paramètres via `Uri.EscapeDataString`

csharp

```
// ✅ SÉCURISÉ - Échappement approprié
<NavLink href="@($"/profile?name={Uri.EscapeDataString(userName)})">
    Profil de {userName}
</NavLink>

// Résultat : /profile?name=admin%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E
```

NavigationManager

Navigation - Blazor

Navigation Manager

Le **NavigationManager** est le service central pour gérer la navigation dans les applications Blazor. Il permet de contrôler les déplacements entre pages de manière programmatique.

Caractéristiques principales

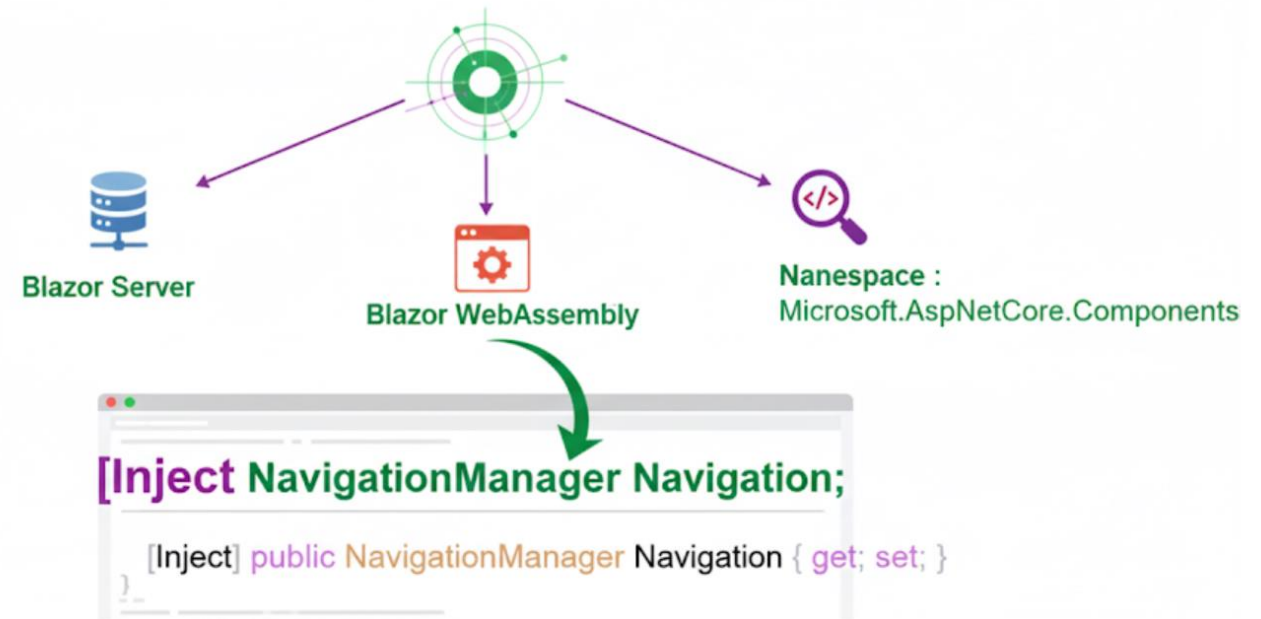
- **Injecté automatiquement** dans le conteneur DI par défaut
- **Compatible** Blazor Server, Blazor WebAssembly
- **Namespace** : Microsoft.AspNetCore.Components

Pour utiliser le navigation manager au sein de notre code, il nous suffit d'utiliser le mot clé [Inject]

```
@inject NavigationManager Navigation
```

```
// ou
```

```
[Inject] public NavigationManager Navigation { get; set; }
```



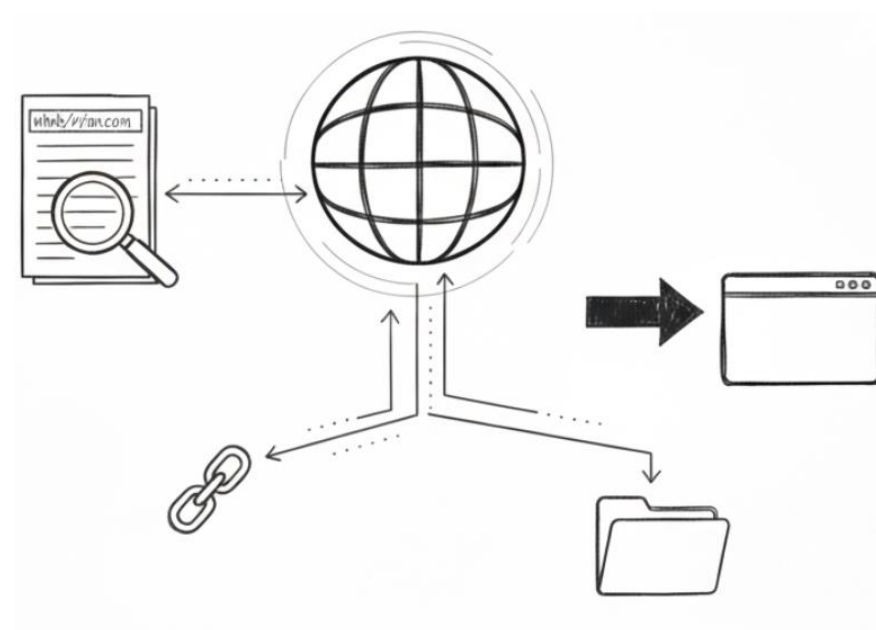
Navigation Manager - Propriétés Essentielles

Propriétés de Navigation

Le NavigationManager expose plusieurs propriétés pour analyser et manipuler les URLs de manière sécurisée.

Propriétés principales

- **BaseUrl** - URL de base de l'application
- **Uri** - URL complète actuelle
- **ToAbsoluteUri()** - Conversion en URL absolue
- **ToBaseRelativePath()** - Extraction du chemin relatif



@code

```
{ protected override void OnInitialized()
{
    var baseUrl = Navigation.BaseUri; // "https://localhost:5001/"
    var currentUrl = Navigation.Uri; // "https://localhost:5001/products/123"
    var relativePath = Navigation.ToBaseRelativePath(Navigation.Uri); // "products/123"
}
```

Navigation Manager – Méthodes de navigation

NavigateTo()

La méthode NavigateTo() offre plusieurs options pour contrôler le comportement de navigation selon les besoins de l'application.

Options disponibles

- **Navigation simple** - Déplacement SPA standard
- **ForceLoad** - Rechargement complet de la page
- **Replace** - Remplacement dans l'historique du navigateur

// Navigation SPA standard

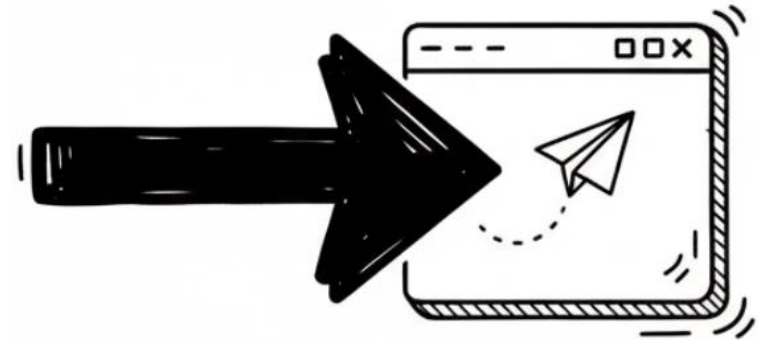
```
Navigation.NavigateTo("/products");
```

// Rechargement complet (perte de l'état)

```
Navigation.NavigateTo("/admin", forceLoad: true);
```

// Remplace l'entrée dans l'historique

```
Navigation.NavigateTo("/dashboard", replace: true);
```

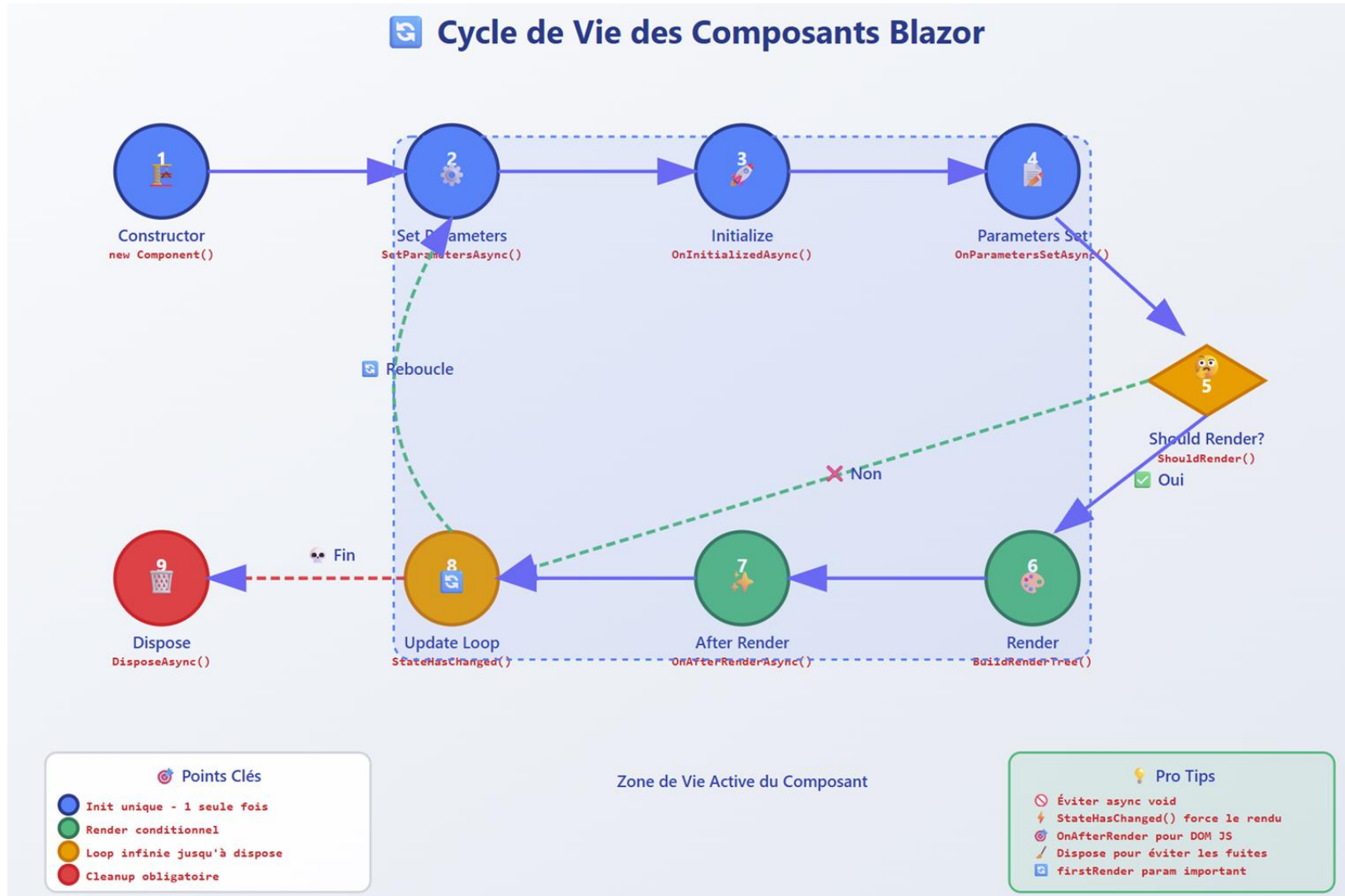


Navigation Manager – Événements

Le cycle de vie d'un composant

Blazor

Le cycle de vie d'un composant - Vue d'ensemble



SetParametersAsync()

Cette méthode est appelée lorsque le composant reçoit des paramètres du composant parent.

```
public override Task SetParametersAsync(ParameterView parameters)
{
    return base.SetParametersAsync(parameters);
}
```

C'est le premier point d'entrée dans le cycle de vie où vous pouvez intercepter et manipuler les paramètres avant qu'ils ne soient assignés aux propriétés.

Exemple :

```
public override async Task SetParametersAsync(ParameterView parameters)
{
    // Capture l'ancienne valeur pour comparaison
    int? oldId = ClientId;
    // Laisse Blazor assigner les paramètres
    await base.SetParametersAsync(parameters);
    // Réagit au changement spécifique d'un paramètre
    if (oldId != ClientId && ClientId > 0)
    {
        await ChargerDonnéesUtilisateur(ClientId);
    }
}
```

Remarque : En .NET 8, `ParameterView` a été optimisé pour réduire les allocations mémoire lors des rendus fréquents.

SetParametersAsync()

Bonnes Pratiques

- Appelez toujours `base.SetParametersAsync()` pour assurer l'affectation correcte des paramètres
- Utilisez cette méthode pour des validations complexes ou des conversions de paramètres
- Évitez d'y placer des logiques métier lourdes qui pourraient être déplacées vers `OnParametersSetAsync()`



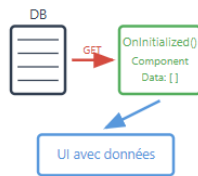
OnInitialized() / OnInitializedAsync()

```
protected override void OnInitialized()
{
    base.OnInitialized();
}

protected override Task OnInitializedAsync()
{
    return base.OnInitializedAsync();
}
```

Ces méthodes sont appelées une seule fois lors de la première instantiation du composant. C'est l'endroit idéal pour initialiser l'état du composant, notamment avec des données provenant d'API ou de services.

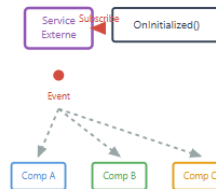
Charger des données initiales



Initialiser le composant avec des données provenant d'une API ou d'une base de données dès le premier rendu.

```
protected override async Task
OnInitializedAsync() { users = await
userService.GetUsersAsync(); }
```

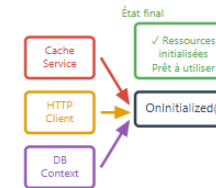
S'abonner à des événements



Configurer les abonnements aux événements externes (SignalR, services de notification, etc.) lors de l'initialisation.

```
protected override void OnInitialized() {
notificationService.OnNotification +=
HandleNotification; }
```

Initialiser des ressources coûteuses



Préparer des connexions coûteuses, initialiser des caches ou configurer des services lourds une seule fois au démarrage.

```
protected override async Task
OnInitializedAsync() { await
cacheService.InitializeAsync();
httpClient.DefaultRequestHeaders
.Add("Authorization", token); }
```

OnInitialized() / OnInitializedAsync()

Exemple :

```
3 references
[Inject] private HttpClient _http { get; set; }

2 references
[Inject] private ILogger _logger { get; set; }
private List<Client> _clients = new();
private bool erreurChargement;

1 reference
private ConfigApp Configuration { get; set; } = new();

0 references
public Client(ILogger logger, HttpClient http)
{
    _logger = logger;
    _http = http;
}

0 references
protected override async Task OnInitializedAsync()
{
    try
    {
        // Paralléliser les appels indépendants
        Task<List<Client>?> tacheClient = _http.GetFromJsonAsync<List<Client>>("api/client");
        var tacheConfig = _http.GetFromJsonAsync<ConfigApp>("api/configuration");
        // Attendre que tout soit terminé
        await Task.WhenAll(tacheClient, tacheConfig);
        _clients = tacheClient.Result ?? new List<Client>();
        Configuration = tacheConfig.Result ?? new ConfigApp();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Erreur lors de l'initialisation");
        erreurChargement = true;
    }
}
```

OnInitialized() / OnInitializedAsync()

Bonnes Pratiques

- Parallélisez les appels API indépendants pour optimiser les performances
- Implémentez une gestion d'erreurs robuste avec fallback UI*
- Utilisez les services injectés plutôt que d'accéder directement aux ressources



* Voir le chapitre gestion des erreurs

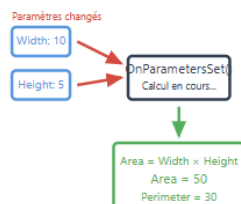
OnParametersSet() / OnParametersSetAsync()

```
protected override void OnParametersSet()  
{  
    // Réaction synchrone aux paramètres  
}
```

```
protected override async Task OnParametersSetAsync()  
{  
    // Réaction asynchrone aux paramètres  
}
```

Ces méthodes sont appelées **après chaque rendu** lorsque le composant reçoit des paramètres, y compris lors de la première initialisation. Elles permettent de réagir aux changements de paramètres et de maintenir l'état interne du composant synchronisé.

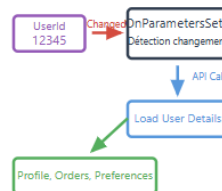
Recalculer des valeurs dérivées



Quand les paramètres changent, recalculer automatiquement les valeurs dérivées comme les totaux, moyennes, ou données formatées.

```
protected override void  
OnParametersSet() { Area = Width  
    * Height; Perimeter = 2 * (Width  
    + Height); }
```

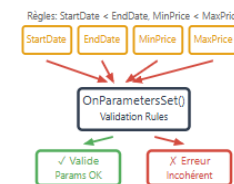
Charger des données supplémentaires



Lorsqu'un paramètre clé change (comme un ID), charger les données associées depuis une API ou base de données.

```
protected override async Task  
OnParametersSetAsync() { if  
(UserId != previousUserId) {  
    userDetails = await userService  
        .GetUserDetailsAsync(UserId);  
    previousUserId = UserId; } }
```

Valider la cohérence des paramètres



Vérifier que les paramètres sont cohérents entre eux (dates valides, plages correctes, combinaisons logiques).

```
protected override void  
OnParametersSet() { if (StartDate  
    >= EndDate) throw new  
    ArgumentException("StartDate must  
    be before EndDate"); if (MinPrice  
    > MaxPrice) MaxPrice = MinPrice;  
    }
```

OnParametersSet() / OnParametersSetAsync()

Exemple

```
[Parameter] public int CategorieId { get; set; }
private List<Client> clientsFiltres = new();
private int dernierCategorieId;
0 references
protected override async Task OnParametersSetAsync()
{
    // Vérifie si le paramètre a vraiment changé
    if (CategorieId != dernierCategorieId)
    {
        dernierCategorieId = CategorieId;
        // Évite les appels inutiles
        if (CategorieId > 0)
        {
            await ChargerClientPourCategorie();
        }
        else
        {
            // Réinitialise si le paramètre n'est pas valide
            clientsFiltres.Clear();
        }
    }
}
```

OnParametersSet() / OnParametersSetAsync()

Pièges à Éviter

- Ne pas vérifier si les paramètres ont réellement changé, causant des rendus en cascade
- Déclencher des effets de bord qui modifient l'état d'autres composants
- Effectuer des opérations trop longues qui bloquent le rendu



OnAfterRender() / OnAfterRenderAsync()

```
protected override void OnAfterRender(bool firstRender)
{
    // Code exécuté après le rendu
}
```

```
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    // Code asynchrone exécuté après le rendu
}
```

Ces méthodes sont appelées **après chaque cycle de rendu** du composant, lorsque le DOM a été mis à jour.

Le package **JS** Bibliothèques JavaScript Tierces

que **DOM** Manipulation Directe du DOM

du **⚡** Mesure des Performances

🏠 DOM Complètement Construit

🔧 Initialisation de Chart.js, FullCalendar, Leaflet après le rendu

- ✓ Initialisation sécurisée après rendu DOM
- ✓ Intégration avec bibliothèques externes
- ✓ Configuration une seule fois (firstRender)
- ✓ Gestion des erreurs JS

Chart.js Initialisé

FullCalendar Initialisé

Leaflet Maps Initialisé

```
protected override async Task
OnAfterRenderAsync(bool firstRender) { if
(firstRender) { // Initialiser Chart.js await
JS.InvokeVoidAsync("initChart", chartElement);
// Configuration une seule fois
isChartInitialized = true; } }
```

🎯 Focus automatique, mesures, animations CSS personnalisées

- ✓ Focus automatique sur les éléments
- ✓ Mesure des dimensions réelles
- ✓ Déclenchement d'animations CSS
- ✓ Observation des changements DOM

Input Focus

Mesures

Animations

```
protected override void OnAfterRender(bool
firstRender) { if (firstRender) { // Focus sur
un élément JS.InvokeVoidAsync("focusElement",
"myInput"); // Animer l'apparition
JS.InvokeVoidAsync("animateIn", ".component");
} }
```

📊 Monitoring temps de rendu, optimisation, métriques utilisateur

- ✓ Mesure du temps de rendu réel
- ✓ Comptage des éléments DOM
- ✓ Détection des goulots d'étranglement
- ✓ Logs de performance

42ms
Temps Rendu

127
Éléments DOM

```
private DateTime renderStart; protected
override void OnInitialized() { renderStart =
DateTime.Now; } protected override void
OnAfterRender(bool firstRender) { var
renderTime = DateTime.Now - renderStart;
Logger.LogInformation("Rendu: {0}ms",
renderTime.TotalMilliseconds); }
```

📦 Opérations nécessitant toutes les ressources chargées

- ✓ Attente du chargement des images
- ✓ Calcul des dimensions finales
- ✓ Initialisation des interactions
- ✓ Validation de l'état complet

DOM Complètement Construit ✓

🖼 Images chargées • 📏 Dimensions calculées • 🎮 Interactions prêtes

```
protected override async Task
OnAfterRenderAsync(bool firstRender) { if
(firstRender) { // Attendre le chargement des
images await
JS.InvokeVoidAsync("waitForImages"); //
Dimensions finales var dims = await
JS.InvokeAsync<object>("getDimensions");
isFullyLoaded = true; StateHasChanged(); } }
```

OnAfterRender() / OnAfterRenderAsync()



Points Clés à Retenir



OnAfterRender

Synchrone, idéal pour des opérations rapides sur le DOM comme le focus ou les mesures simples.



OnAfterRenderAsync

Asynchrone, parfait pour l'initialisation JavaScript et les opérations longues nécessitant await.



Paramètre firstRender

Utilisez toujours cette condition pour éviter les exécutions répétées inutiles lors des re-rendus.



StateHasChanged()

Évitez d'appeler StateHasChanged() sans condition dans ces méthodes pour prévenir les boucles infinies.

OnInitialized() / OnInitializedAsync()

Bonnes Pratiques

- Utilisez **firstRender** pour les initialisations uniques
- Évitez de modifier l'état qui déclencherait un nouveau rendu
- Gérez correctement les erreurs JS qui peuvent survenir



ShouldRender()

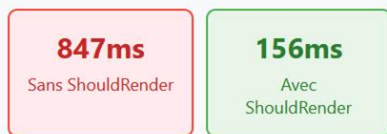
Cette méthode permet de contrôler si un composant doit être rendu ou non.
Elle est appelée avant chaque rendu potentiel et permet d'optimiser les performances en évitant les rendus inutiles.

```
protected override bool ShouldRender()  
{  
    return base.ShouldRender();  
}
```

⚡ Améliorer les Performances

🚀 Éviter des rendus coûteux en bloquant les mises à jour inutiles

- ✓ Éviter les re-rendus inutiles
- ✓ Optimiser les composants lourds
- ✓ Réduire la charge CPU
- ✓ Améliorer la réactivité UI



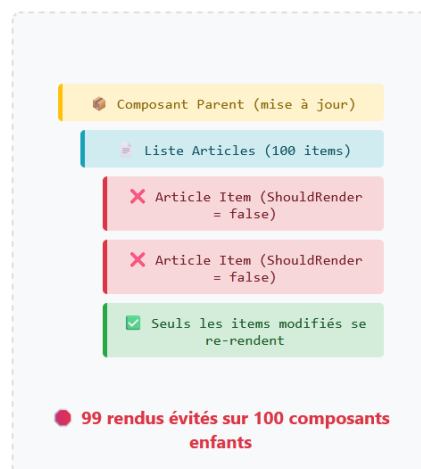
⚡ Amélioration de 82% des performances

```
protected override bool ShouldRender() {  
    // Ne pas re-rendre si données inchangées  
    if (previousData?.Equals(currentData) == true)  
        return false; // Ne pas re-rendre pendant le chargement  
    if (isLoading) return false;  
    previousData = currentData; return true;  
}
```

🚫 Empêcher les Cascades

🛑 Stopper la propagation des rendus dans les hiérarchies complexes

- ✓ Bloquer les cascades parent → enfant
- ✓ Optimiser les listes importantes
- ✓ Contrôler la propagation d'état
- ✓ Isoler les composants critiques



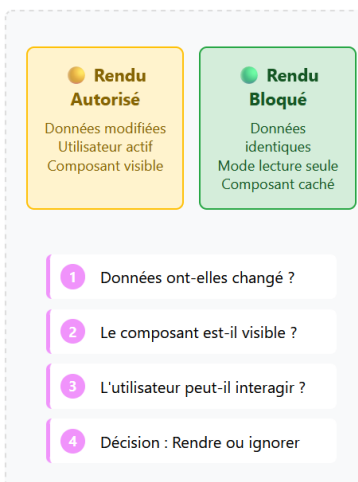
```
// Composant Article dans une liste  
protected override bool ShouldRender() {  
    // Ne re-rendre que si CET article a changé  
    var shouldUpdate = Article.LastModified > lastRendered;  
    if (shouldUpdate) lastRendered = DateTime.Now;  
    return shouldUpdate;  
} // Évite que tous les 100 items se re-rendent quand seul 1 item change
```

🎯 Contrôle Précis

🔧 Maîtriser exactement quand un composant doit se mettre à jour

- ✓ Contrôle conditionnel fin
- ✓ Gestion des états complexes
- ✓ Optimisation contextuelle
- ✓ Debug des cycles de rendu

```
protected override bool ShouldRender() {  
    // Contrôle multi-critères  
    return hasDataChanged && isComponentVisible && !isReadOnlyMode && userCanInteract;  
} // Exemple avancé avec logging  
protected override bool ShouldRender() {  
    var shouldRender = EvaluateRenderConditions();  
    Logger.LogDebug("ShouldRender: {shouldRender} - " + "Data: {hasDataChanged}, Visible: {isVisible}", shouldRender, hasDataChanged, isVisible);  
    return shouldRender;  
}
```



ShouldRender()

Pièges à Éviter

- Sur-optimiser et bloquer des rendus nécessaires
- Implémenter des logiques complexes qui coûtent plus que le rendu
- Oublier de réinitialiser les indicateurs de changement (A expliquer)



StateHasChanged()

```
/// <summary>  
/// Notifies the component that its state has changed. When applicable, this will  
/// cause the component to be re-rendered.  
/// </summary>  
protected void StateHasChanged()
```

Cette méthode permet de demander explicitement au framework de “re-render” le composant.

Quand

l'utiliser?

- Après des mises à jour asynchrones venant de services ou événements
- Lors de l'utilisation de minuteries ou d'intervalles
- Après des réponses à des événements JavaScript

StateHasChanged()

Bonnes Pratiques

- Utilisez **InvokeAsync** pour synchroniser avec le thread UI
- Évitez d'appeler **StateHasChanged()** dans les gestionnaires d'événements UI standards (c'est automatique)
- Utilisez l'invalidation sélective des composants (A Expliquer)



Sécurité et Identité

Blazor

Authentication vs Autorisation

Authentication

L'authentification répond à la question "**Qui êtes-vous ?**". C'est le processus par lequel un système vérifie l'identité d'un utilisateur. Cette vérification peut s'appuyer sur différents facteurs :

- Ce que l'utilisateur sait (mot de passe, réponse à une question secrète)
- Ce que l'utilisateur possède (téléphone, carte à puce)
- Ce que l'utilisateur est (empreinte digitale, reconnaissance faciale)

Autorisation

L'autorisation répond à la question "**Que pouvez-vous faire ?**". Une fois l'identité vérifiée, le système détermine les ressources auxquelles l'utilisateur peut accéder et les actions qu'il peut effectuer, généralement en fonction de :

- Rôles attribués (administrateur, éditeur, lecteur)
- Permissions spécifiques (créer, modifier, supprimer)
- Propriété des ressources

Authentication



vs.

Authorization



L'authentification établit la confiance dans l'identité de l'utilisateur, tandis que l'autorisation établit la confiance dans ses droits d'accès.

L'authentification précède toujours l'autorisation dans le flux de sécurité. Cette séquence est fondamentale : il est impossible d'autoriser un utilisateur dont l'identité n'a pas été préalablement vérifiée et confirmée.

Comparaison

Critère	Authentification	Autorisation
Objectif principal	Vérifier l'identité	Déterminer les droits d'accès
Question	Qui êtes-vous?	Que pouvez-vous faire ?
Mécanismes	Mot de passe, biométrie, certificat, tokens, MFA	Rôles, Claims, policies
Séquence	S'exécute en premier	S'exécute après l'authentification
Transmissions des données	Cookies, http Header, token, JWT	Claims stockés dans les cookies ou tokens
Protocoles	SAML, OAuth 2.0, OpenID Connect	RBAC, ABAC, PBAC
Exemple	Formulaire de connexion, empreinte digitale, Google Authenticator	Contrôle d'accès aux pages admin, blocage de fonctionnalités premium
Implémentation ASP	AuthenticationMiddleware, SignInManager	AuthorizationMiddleware, [Authorize] attribute

Dans un environnement ASP.NET, ces deux aspects sont clairement séparés dans l'architecture. L'authentification est configurée via les services d'authentification dans la méthode `ConfigureServices`, tandis que l'autorisation est généralement appliquée par des attributs sur les contrôleurs ou les pages Razor, ou par des politiques d'autorisation plus granulaires.

Cette séparation des préoccupations permet une gestion plus flexible et plus sécurisée des accès aux applications, particulièrement important dans les architectures modernes comme Blazor.

Windows Identity

Vue d'ensemble de Windows Identity

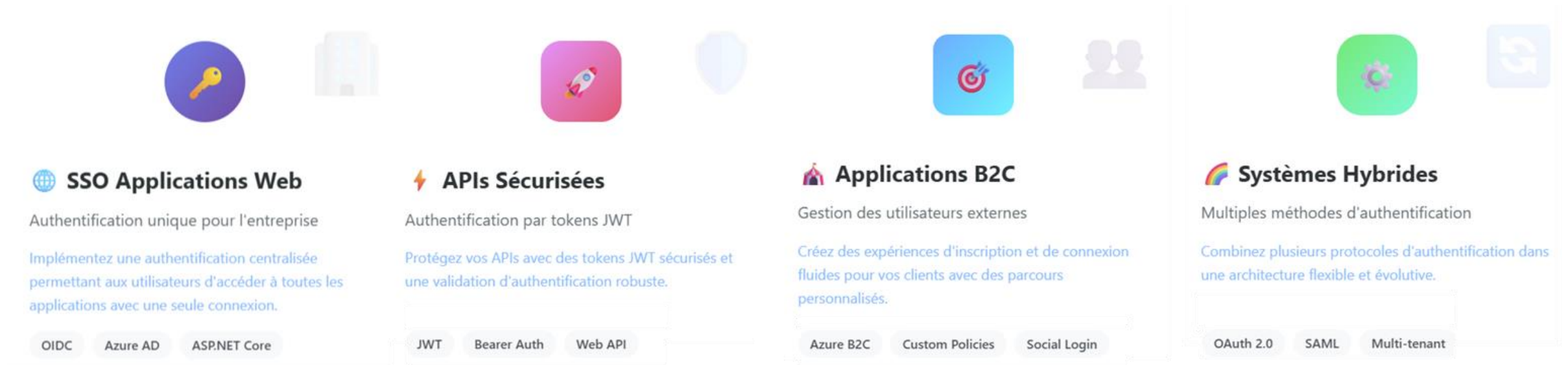
Avant de rentrer dans Blazor proprement dit, nous devons nous arrêter sur la façon dont ASP.NET intègre la sécurité car cela conditionne la mise en place du côté Blazor (surtout en Blazor Web App)

Windows Identity Framework (WIF) est le système d'authentification et d'autorisation moderne intégré à ASP.NET.

- **Sécurité moderne** : Répond aux défis de sécurité actuels des applications web et API
- **Standardisation** : Implémente les standards de l'industrie (OAuth, OpenID Connect)
- **Flexibilité** : S'adapte à différents scénarios d'authentification
- **Évolutivité** : Supporte aussi bien les petites applications que les systèmes d'entreprise

Vue d'ensemble de Windows Identity

Cas d'utilisation typiques :



Avantages



Intégration Native

Intégration .NET 100%

- ✦ Intégration parfaite avec l'écosystème .NET
- ✦ Compatibilité avec l'injection de dépendance
- ✦ Configuration fluide et intuitive
- ✦ Support natif ASP.NET Core

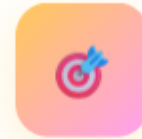


Sécurité

Protection de niveau entreprise

Sécurité par défaut 98%

- ✦ Meilleures pratiques de sécurité par défaut
- ✦ Gestion automatique du hachage des mots de passe
- ✦ Protection intégrée contre CSRF, XSS
- ✦ Support natif de l'authentification 2FA
- ✦ Rotation automatique des tokens



Flexibilité

Adaptable à tous vos besoins

Schémas supportés 92%

- ✦ Support multiple des schémas d'authentification
- ✦ JWT, cookies, certificats compatibles
- ✦ Personnalisation des stores et providers
- ✦ Architecture modulaire extensible



Fonctionnalités Modernes

Technologies de pointe

Standards modernes 96%

- ✦ Support complet OAuth 2.0 et OpenID Connect
- ✦ Gestion des JWT avec validation automatique
- ✦ Claims-based identity intégré
- ✦ Support des rôles et politiques d'autorisation

Inconvénients



Complexité

- Courbe d'apprentissage significative pour les débutants
- Documentation parfois fragmentée entre les différentes versions
- Nombreuses options de configuration pouvant créer de la confusion
- Architecture complexe pour des besoins simples



Surcharge Potentielle

- Framework complet quand parfois seules quelques fonctionnalités sont nécessaires
- Consommation mémoire plus importante que des solutions minimalistes
- Démarrage de l'application légèrement plus lent



Contraintes

- Dépendance forte à Entity Framework par défaut
- Nécessité de suivre les mises à jour de sécurité
- Configuration parfois complexe en environnement distribué
- Complexité accrue pour les architectures microservices
- Personnalisation limitée de certains aspects du pipeline d'authentification

Quand l'utiliser (ou pas ?)

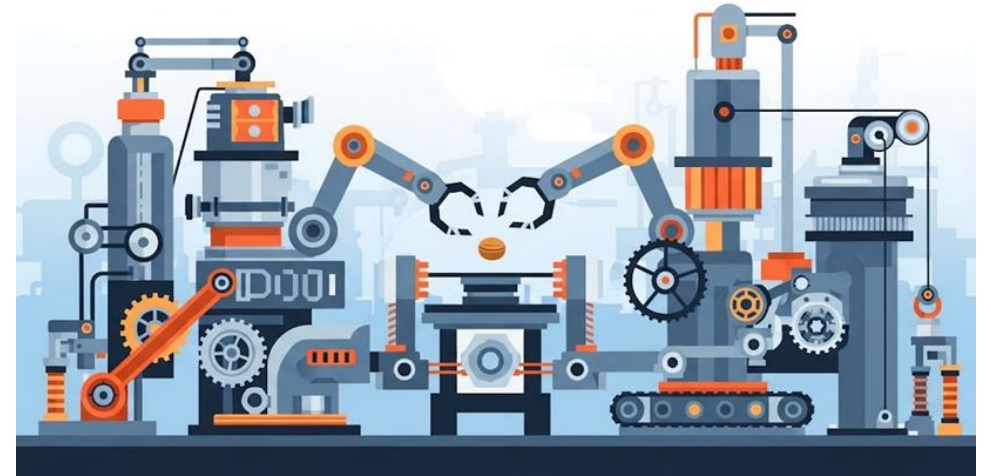
Utiliser WIF quand :

1. Vous avez besoin d'un système d'authentification complet et robuste
2. Vous souhaitez une solution maintenue et sécurisée
3. Vous avez des besoins d'extensibilité futurs
4. Vous développez une application d'entreprise
5. Vous avez besoin de fonctionnalités avancées (2FA, SSO, etc.)



Éviter WIF quand :

1. Vous développez une API très simple avec peu de besoins en authentification
2. Vous avez des contraintes de performance extrêmes
3. Vous avez des besoins très spécifiques incompatibles avec le framework
4. Vous préférez une solution plus légère et personnalisée
5. Vous n'avez pas besoin de la majorité des fonctionnalités



Concepts

Oauth/OIDC

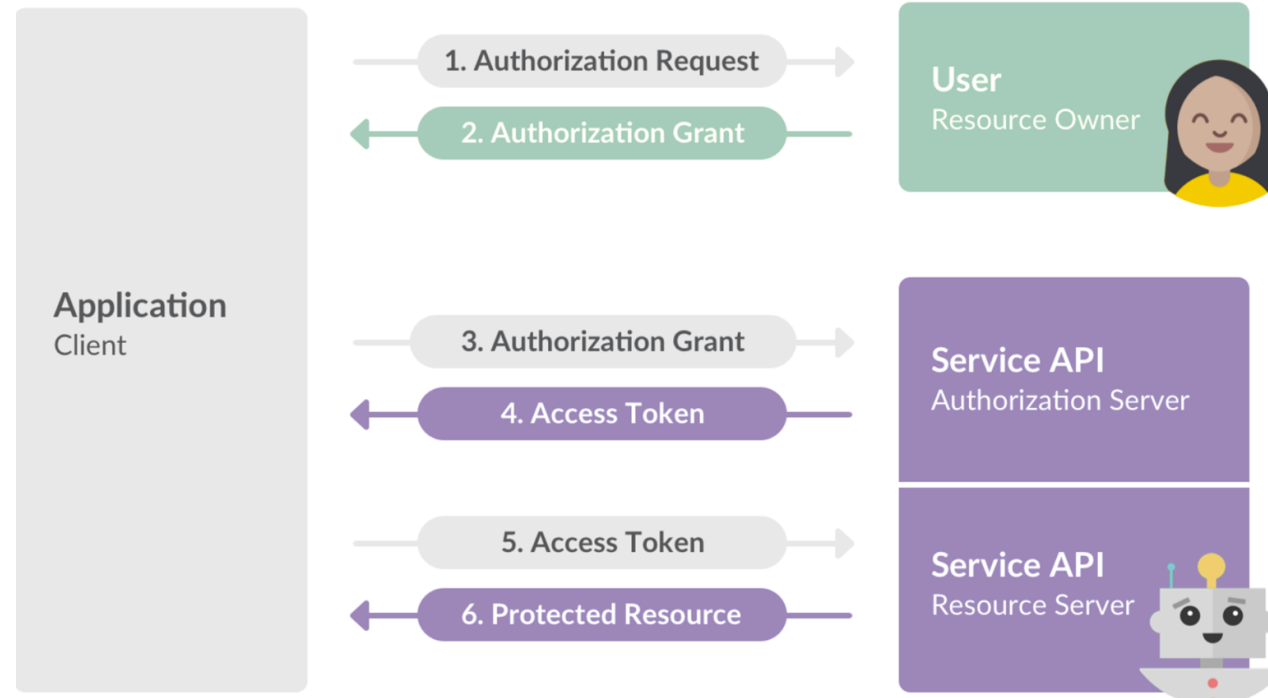
Vue d'ensemble OAuth 2.0

- OAuth 2.0 est un framework d'autorisation qui permet à une application d'obtenir un accès limité à un service
- Ne gère pas directement l'authentification mais l'autorisation
- Concepts clés :
 - Resource Owner : propriétaire des données (l'utilisateur)
 - Client : l'application qui demande l'accès
 - Authorization Server : serveur qui délivre les tokens
 - Resource Server : serveur qui héberge les ressources protégées

OAuth2 – Les types de « Grant »

- Authorization Code Grant
utilisé par les applications Web où le client reçoit un code d'autorisation après l'authentification du propriétaire de la ressource, puis l'échange contre un jeton d'accès
- Implicit Grant
convient aux applications mobiles ou Web où le jeton d'accès est obtenu directement sans avoir besoin d'un code d'autorisation.
- Resource Owner Password Credential
dans les scénarios où le propriétaire de la ressource a une relation de confiance avec le client, les informations d'identification de l'utilisateur sont échangées directement contre un jeton d'accès
- Client Credential Grant
utilisé pour l'authentification de machine à machine où le client lui-même se voit accorder l'accès en fonction de ses propres informations d'identification

Oauth 2.0 - Flow



Source : https://a.slack-edge.com/fbd3c/img/api/articles/oauth_scopes_tutorial/slack_oauth_flow_diagram.png

OAuth2 Vulnérabilités

- Interception du code d'autorisation
Les attaquants interceptent le code d'autorisation pendant le processus de redirection.
L'atténuation consiste à utiliser HTTPS et à garantir un stockage client sécurisé
- Fuite de jetons
si les jetons d'accès sont mal gérés ou divulgués, les attaquants peuvent obtenir un accès non autorisé. La mise en œuvre d'un stockage de jetons sécurisé et de jetons à durée de vie courte permet d'atténuer ce risque
- Validation de portée insuffisante
si le serveur d'autorisation ne parvient pas à valider correctement les portées, les attaquants peuvent accéder à plus de ressources que prévu. Une validation de portée appropriée est cruciale.

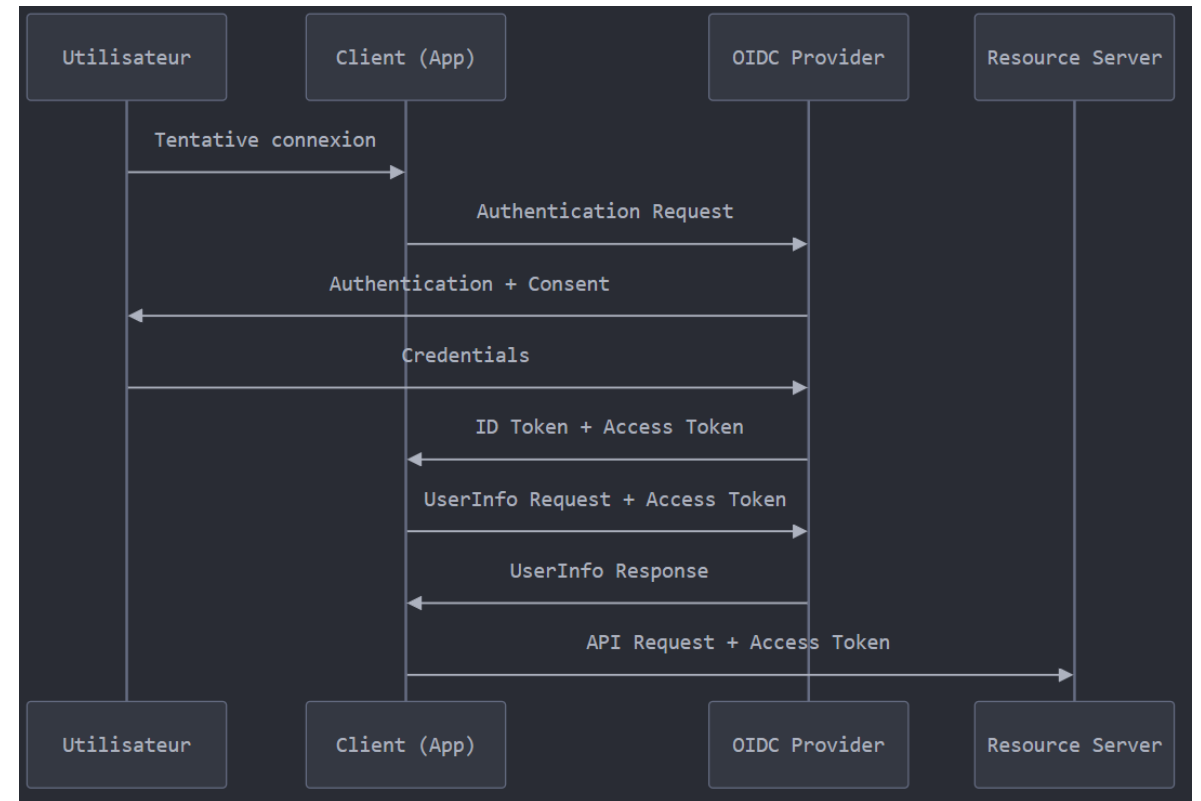
OAuth2 Vulnérabilités

- Usurpation d'identité
si une application malveillante prétend être un client légitime, elle peut tromper les utilisateurs et leur faire accorder un accès non autorisé. La validation de l'enregistrement du client est nécessaire
- Falsification de requête intersites (CSRF)
les attaquants trompent les utilisateurs pour qu'ils effectuent des actions indésirables sans leur consentement. Les jetons anti-CSRF et les pratiques de codage sécurisé peuvent empêcher les attaques CSRF.
- EndPoints non sécurisés
les canaux de communication non protégés ou les points de terminaison mal sécurisés peuvent exposer des informations sensibles. La mise en œuvre de canaux sécurisés et la sécurisation appropriée des points de terminaison sont essentielles.

OpenID Connect (OIDC)

OIDC est une couche d'identité par-dessus OAuth 2.0

- Ajoute :
 - Authentication : Vérifie l'identité de l'utilisateur
 - ID Token : Token JWT contenant les informations d'identité
 - UserInfo Endpoint : Point d'accès aux informations utilisateur
- Standardise le processus d'authentification



Concepts Fondamentaux

- **Types de Tokens**
 - **Access Token** : Pour accéder aux ressources
 - **ID Token** : Contient l'identité de l'utilisateur (OIDC)
 - **Refresh Token** : Pour obtenir de nouveaux access tokens

Différences Clés OAuth vs OIDC

Caractéristique	OAuth 2.0	OpenID Connect
Objectif principal	Autorisation	Authentification
Token principal	Access Token	ID Token + Access Token
Informations utilisateur	Non standardisé	Standardisé (claims)
Scope standard	Custom	openid, profile, email
Vérification identité	Non	Oui

Identity

L'équipe de super-Héros Identity

Auth Man (*Authentication*) :

- Il vérifie l'identité de l'utilisateur via des méthodes comme les mots de passe, OAuth ou OpenID Connect.

Access Guard (*Autorisation*) :

- Il contrôle les accès en fonction des rôles, des claims ou des politiques définies dans l'application.

Info Collector (*Claims & Tokens*) :

- Il stocke et transporte les informations d'identité et d'autorisation sous forme de claims dans des tokens (JWT, Cookies).

Digital Twin (*User Principal*) :

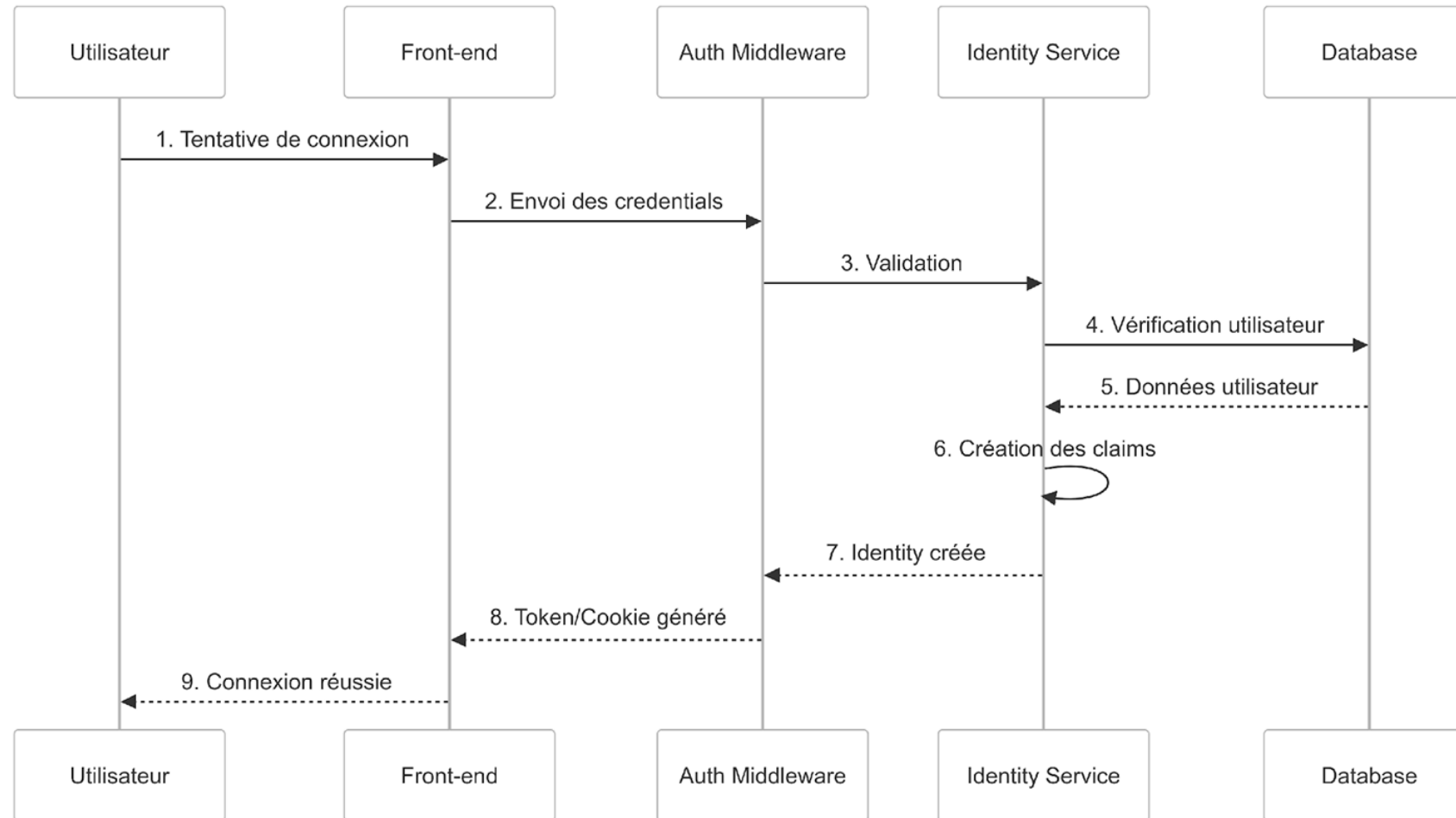
- Il représente l'utilisateur connecté dans l'application via ClaimsPrincipal, accessible dans le code.

Security Chief (*Identity Management*) :

- Il orchestre l'authentification, l'autorisation, la gestion des utilisateurs et la sécurité globale.



Flux et Architecture du système



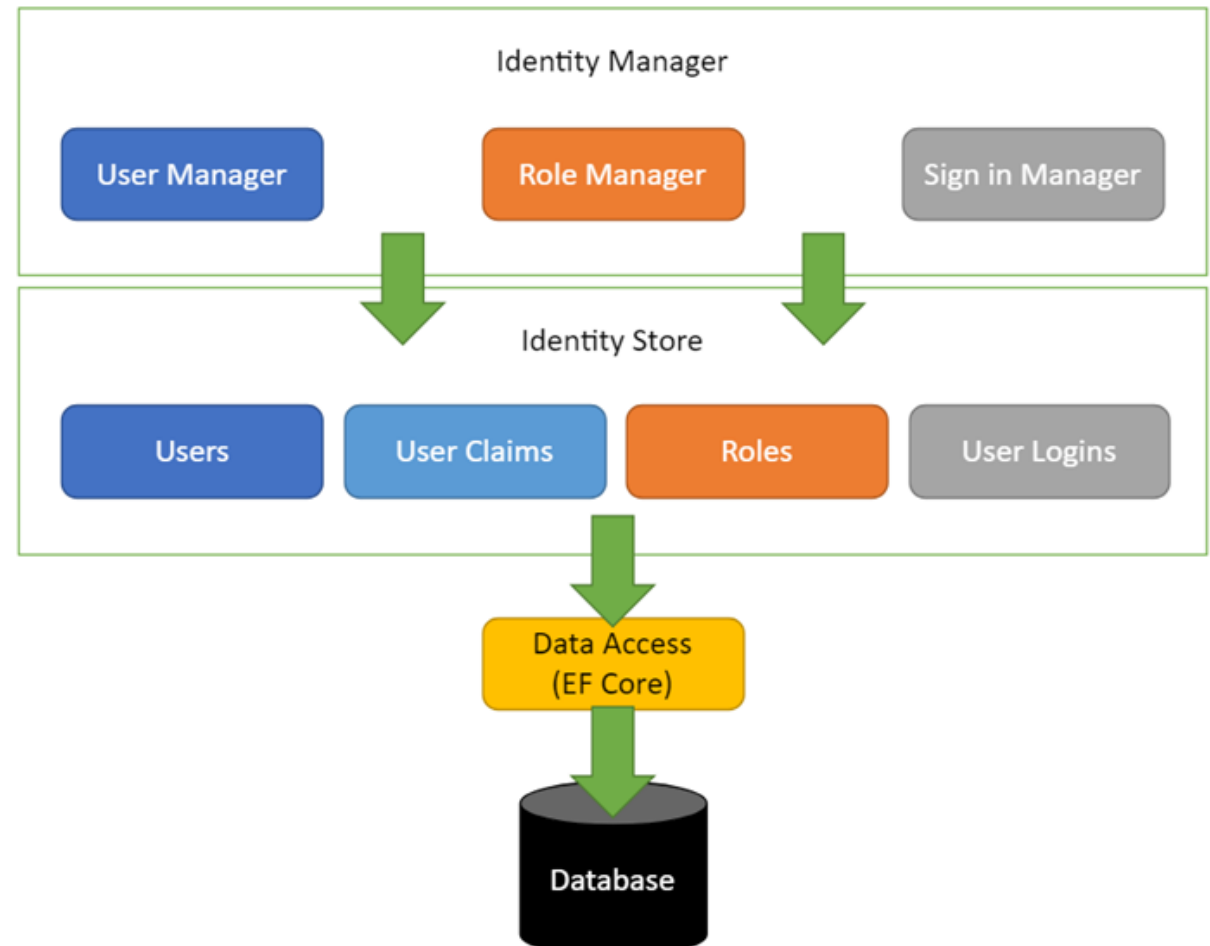
Mise en place

Overview

Voici les principales composantes :

1. Identity Manager

- **User Manager (bleu)** : Gère les utilisateurs (création, mise à jour, suppression).
- **Role Manager (orange)** : Gère les rôles et leur attribution.
- **Sign-in Manager (gris)** : Gère l'authentification des utilisateurs.

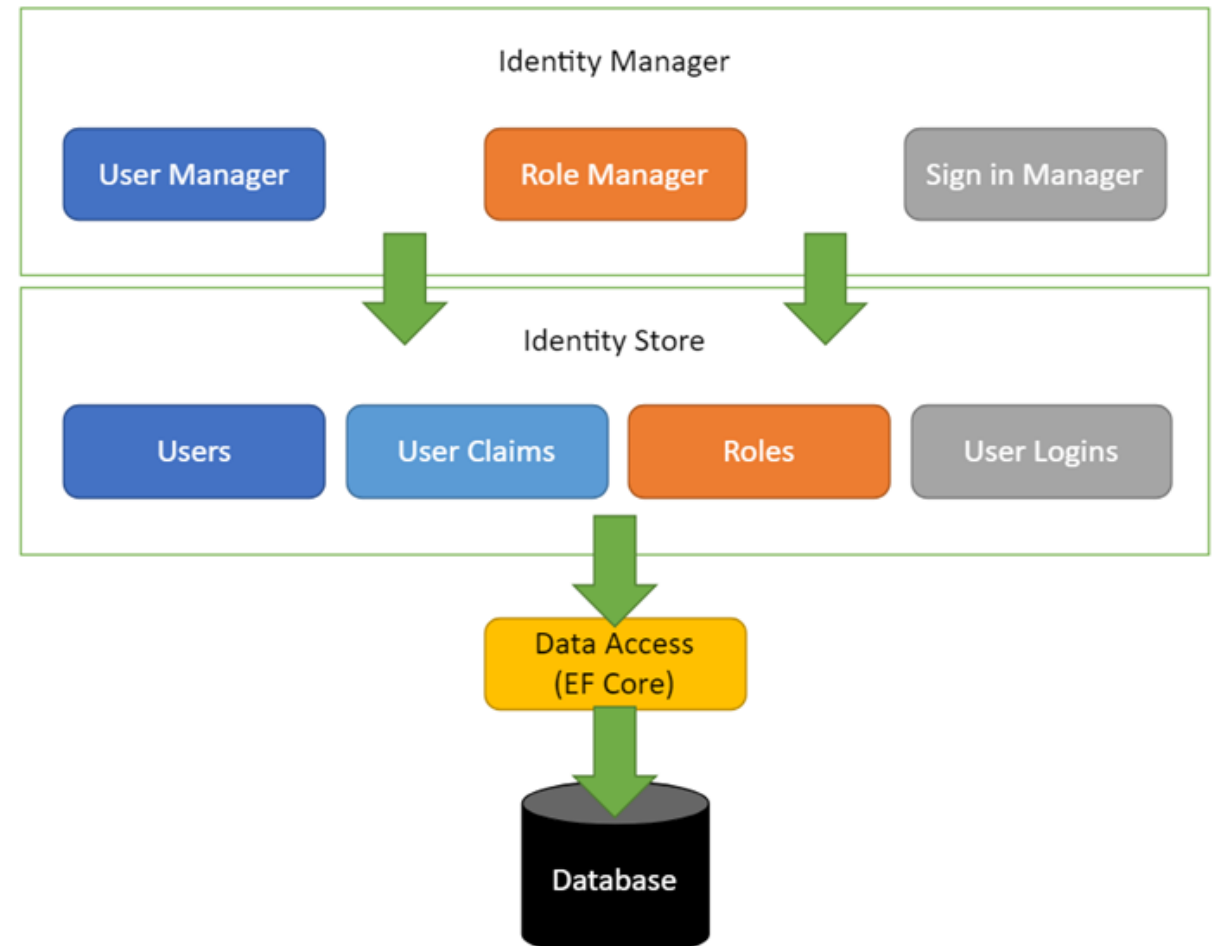


Overview

Voici les principales composantes :

2. Identity Store

- **Users** (bleu) : Stocke les informations des utilisateurs.
- **User Claims** (bleu clair) : Contient les revendications associées aux utilisateurs.
- **Roles** (orange) : Stocke les rôles des utilisateurs.
- **User Logins** (gris) : Gère les informations de connexion externes (OAuth, SSO, etc.).



Overview

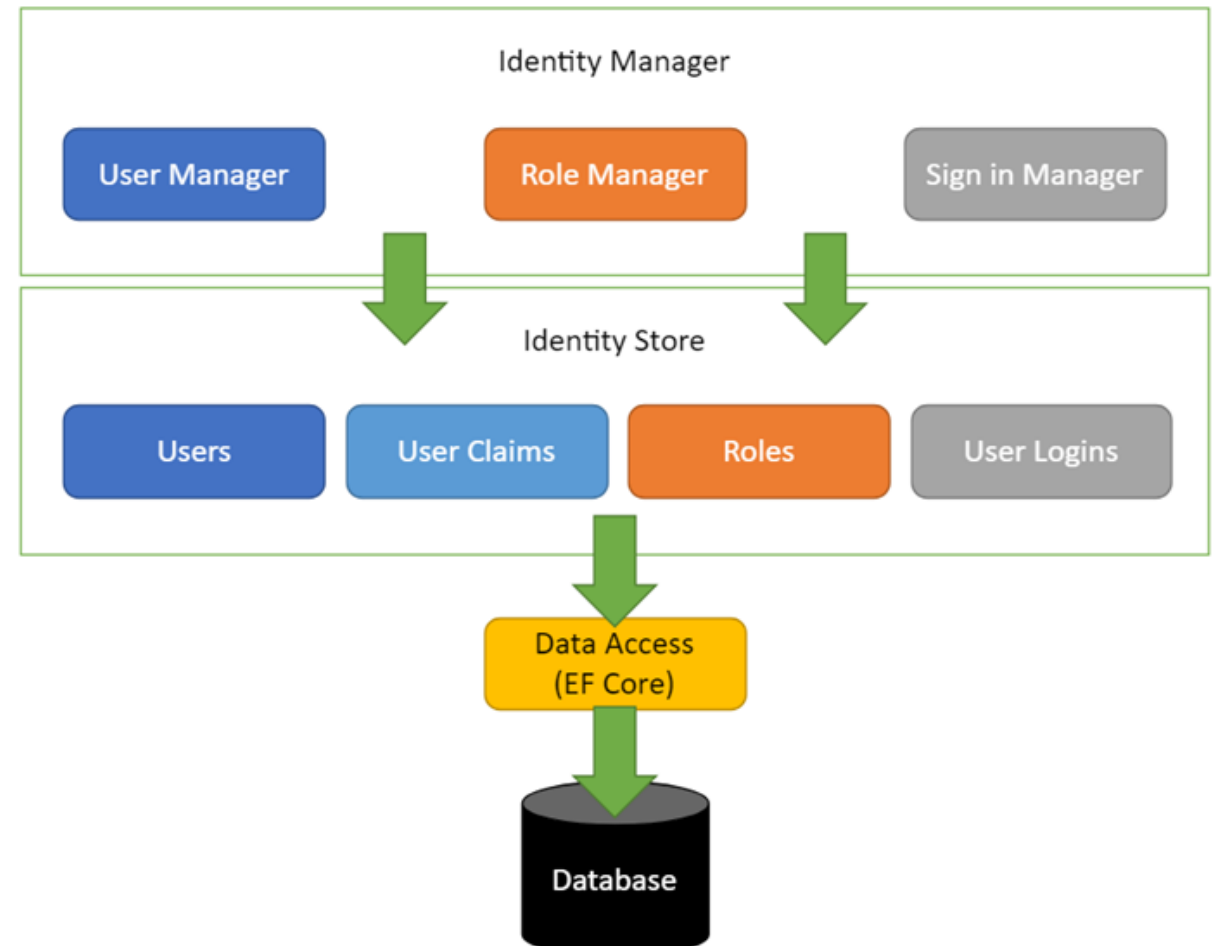
Voici les principales composantes :

2. Data Access (EF Core)

- Fournit un accès à la base de données pour persister et récupérer les informations de l'Identity Store.

3. Database

- Stocke toutes les données relatives aux utilisateurs, rôles et authentifications.



Mise en place

Après avoir créé un projet simple (Asp.net MVC, Asp.net Web Api, ...), Nous devons ajouter les nugget suivants :

- Entity Framework



Microsoft.EntityFrameworkCore ✓ by [aspnet](#), [dotnetframework](#), [EntityFramework](#), [Microsoft](#), **1,37B** downloads

Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL Database, SQLite, Azure Cosmos DB, MySQL, PostgreSQL, and other databases through a provider plugin API.

Microsoft.EntityFrameworkCore.SqlServer ✓ by [aspnet](#), [dotnetframework](#), [EntityFramework](#), [Microsoft](#), **601M** downloads

Microsoft SQL Server database provider for Entity Framework Core.

Microsoft.EntityFrameworkCore.Design ✓ by [aspnet](#), [dotnetframework](#), [EntityFramework](#), [Microsoft](#), **582M** downloads

Shared design-time components for Entity Framework Core tools.

Microsoft.EntityFrameworkCore.Tools ✓ by [aspnet](#), [dotnetframework](#), [EntityFramework](#), [Microsoft](#), **382M** downloads

Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

Mise en place

- Identity



Microsoft.AspNetCore.Identity.EntityFrameworkCore ✓ by [aspnet](#), [dotnetframework](#), [Microsoft](#), **187M** downloads
ASP.NET Core Identity provider that uses Entity Framework Core.

Mise en place

La première phase est de créer un DbContext pour la gestion de la base de données qui contiendra toutes les tables nécessaires.

```
public class IdentitySampleDbContext : IdentityDbContext
{
    0 references
    public IdentitySampleDbContext()
    {
    }
    0 references
    public IdentitySampleDbContext(DbContextOptions options): base(options)
    {
    }
}
```

Mise en place

Et nous configurons également l'injection de dépendance pour que notre DbContext utilise Sql Server avec la connectionString et nous ajoutons la gestion de l'Identity en la liant avec notre DbContext

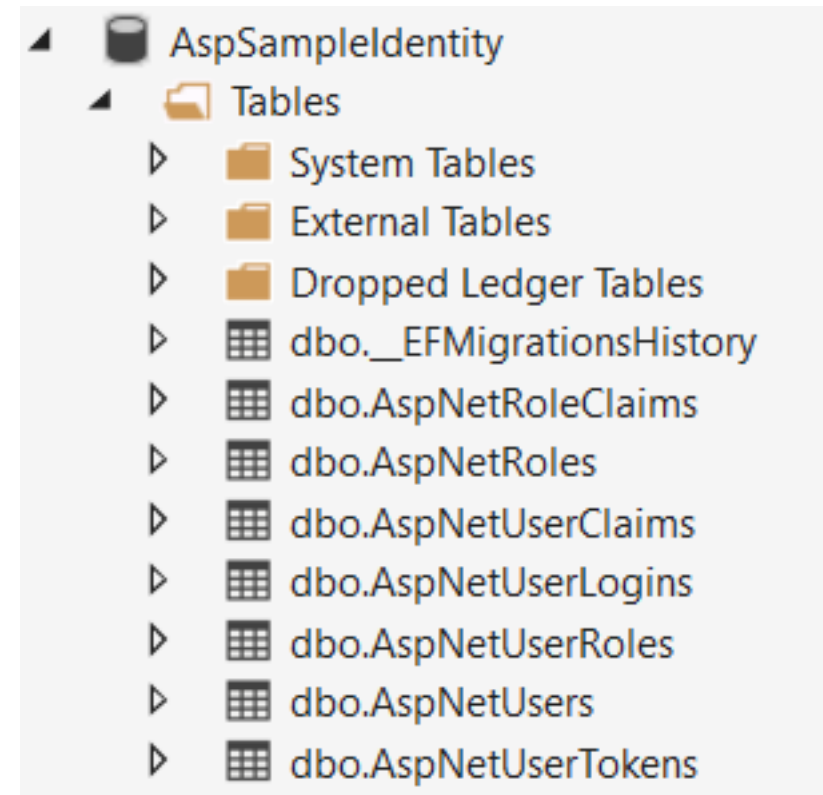
```
/*Récupération des variables à partir de la configuration*/  
string? connectionString = builder.Configuration["ConnectionStrings:Default"];  
/*Pour le DbContext*/  
builder.Services.AddDbContext<IdentitySampleDbContext>(options => { options.UseSqlServer  
    (connectionString); });  
builder.Services.AddIdentity<IdentityUser, IdentityRole>().AddEntityFrameworkStores<IdentitySampleDbContext>  
    ();
```

- Email, PasswordHash...).
- **IdentityRole** : Définit le type de rôle utilisé pour gérer les permissions des utilisateurs.
- **AddIdentity<IdentityUser, IdentityRole>()** : Ajoute les services de gestion des utilisateurs et des rôles à l'injection de dépendances de l'application.

Mise en place

Nous pouvons passer à la migration + Update Database pour voir Identity en action

```
PM> add-migration 'Add Identity'
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20250219174542_Add Identity'.
Done.
```

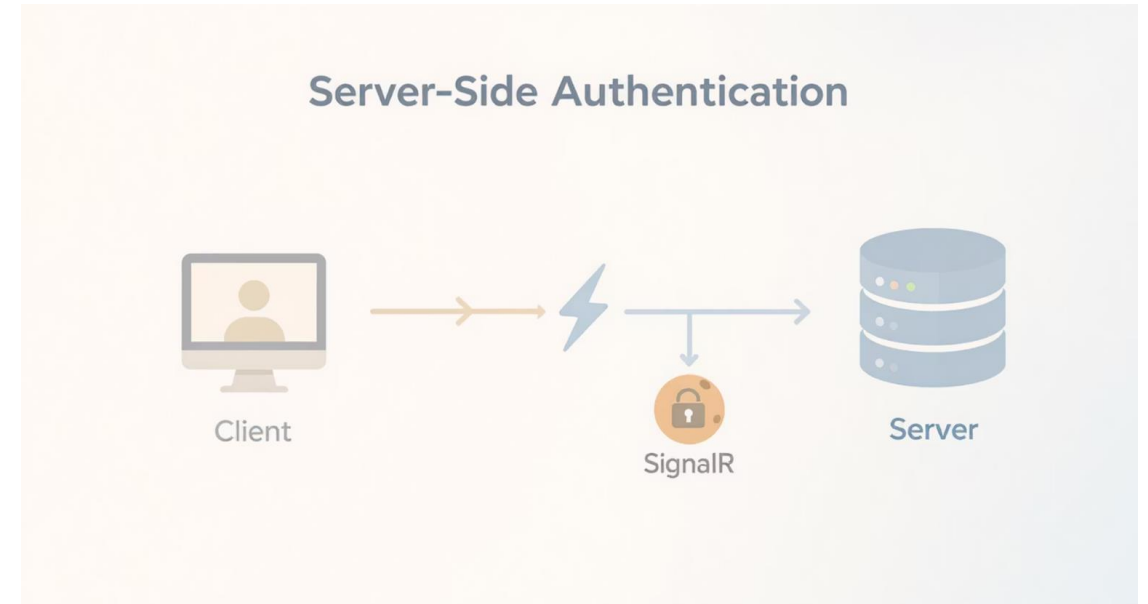


Mécanisme d'Authentification dans Blazor WebApp

Blazor WebApp s'appuie sur un modèle d'authentification centralisé où toute la logique de sécurité est gérée côté serveur.

Cette approche offre plusieurs avantages :

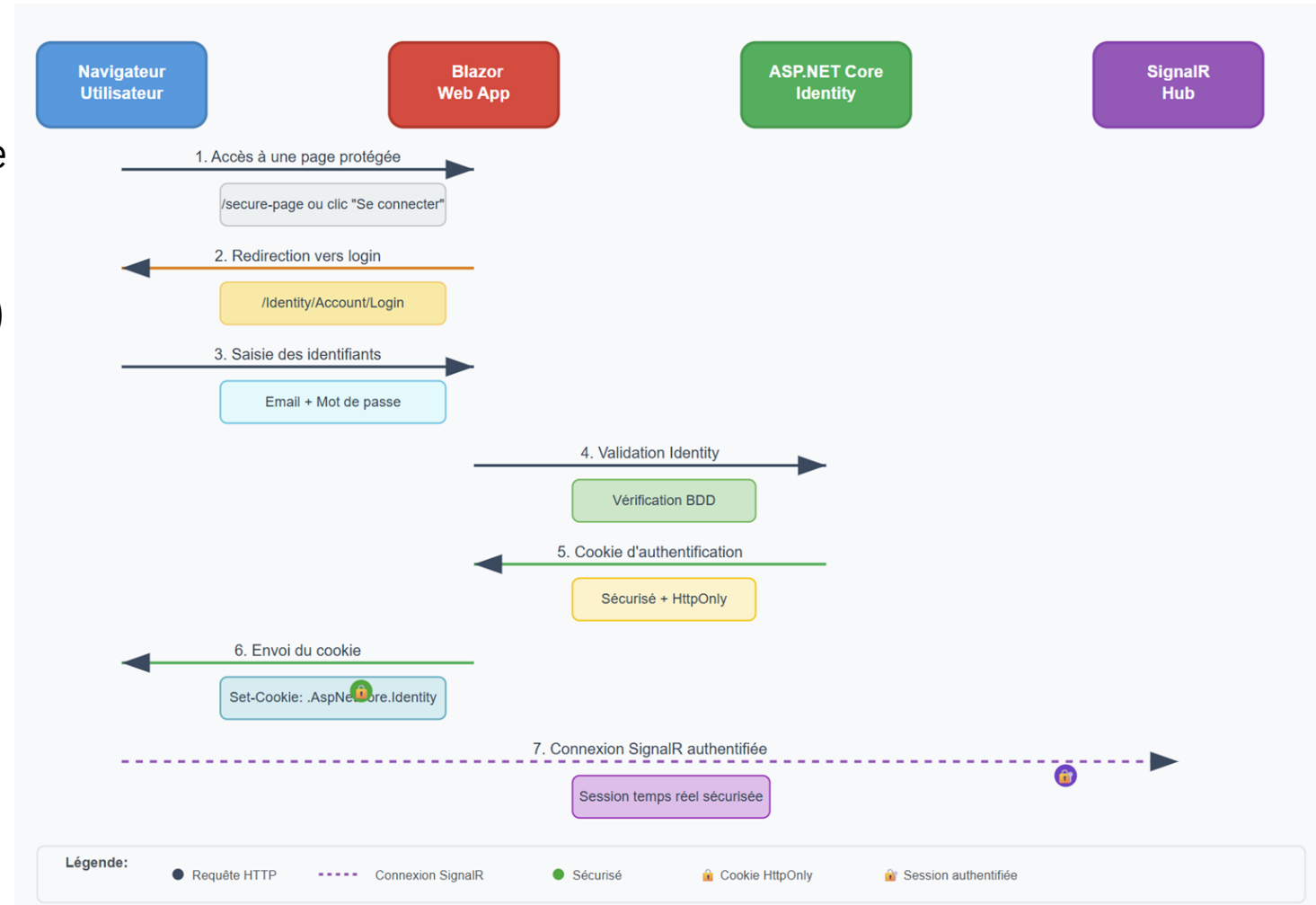
- Intégration native avec ASP.NET Core Identity, le framework d'identité complet de Microsoft
- Protection des secrets d'authentification qui restent toujours côté serveur
- Contrôle total sur le cycle de vie de l'authentification



Mécanisme d'Authentification dans Blazor WebApp

Flux d'Authentification

1. L'utilisateur accède à une page protégée ou clique sur "Se connecter"
2. Redirection vers la page de connexion (généralement /Identity/Account/Login)
3. Saisie des identifiants et validation par ASP.NET Core Identity
4. Création d'un cookie d'authentification sécurisé
5. Établissement d'une session SignalR authentifiée



Mécanisme d'Authentification dans Blazor WebApp : Caractéristiques

○



Demo

Etape 1: Configuration de l'authentification Windows

Nous avons besoin du NuGet suivant : **Microsoft.AspNetCore.Authentication.Negotiate**.

Ce NuGet permet d'avoir accès à *NegotiateDefaults.AuthenticationScheme* qui représente le nom par défaut du schéma d'authentification utilisé pour l'authentification Windows (qui inclut des protocoles comme NTLM et Kerberos).

Nous pouvons dès lors modifier notre *program.cs* pour inclure l'authentification windows

```
builder.Services.AddAuthentication(NegotiateDefaults.AuthenticationScheme)
                .AddNegotiate();
```

Et ensuite ajouter l'autorization

```
builder.Services.AddAuthorization(options =>
{
    // Par défaut, toutes les politiques nécessitent un utilisateur
    // authentifié.
    options.FallbackPolicy = options.DefaultPolicy;
});
```

Etape 2 : Disponibilité de l'authentification pour les composants

```
builder.Services.AddCascadingAuthenticationState();
```

`builder.Services.AddCascadingAuthenticationState();` **Rend disponible** cet état d'authentification à tous vos composants Blazor. Il est également possible d'utiliser le tag `<CascadingAuthenticationState>` dans le `MainLayout` qui aura le même effet

```
<CascadingAuthenticationState>
  @Body
</CascadingAuthenticationState>
```

Etape 3 : Gestion de l'état d'authentification

```
builder.Services.AddScoped<AuthenticationStateProvider, ServerAuthenticationStateProvider>();
```

- *AuthenticationStateProvider* est l'interface que Blazor utilise pour déterminer l'état d'authentification de l'utilisateur.
- *ServerAuthenticationStateProvider* est l'implémentation par défaut fournie pour les applications Blazor Server (et les Web Apps avec un mode interactif Server) qui est capable de lire les informations d'authentification de l'utilisateur à partir du `HttpContext.User` de la requête HTTP côté serveur.

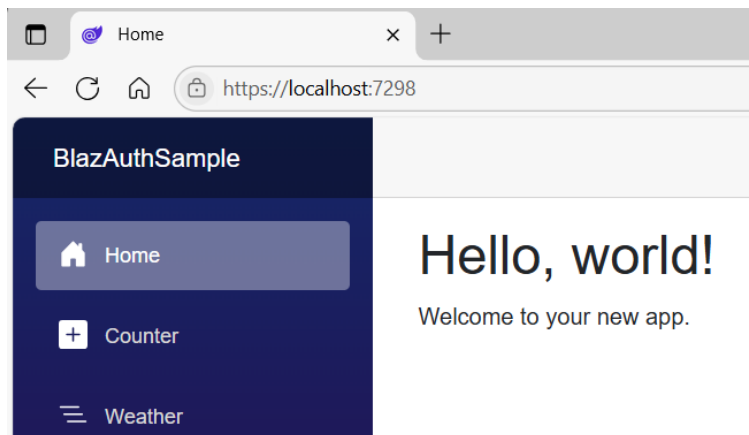
Etape 4 : Ajouter les middlewares

Comme pour Asp.net, nous ajoutons deux directives pour utiliser les middleware d'authentification et d'autorization

```
app.UseAuthentication();  
app.UseAuthorization();
```

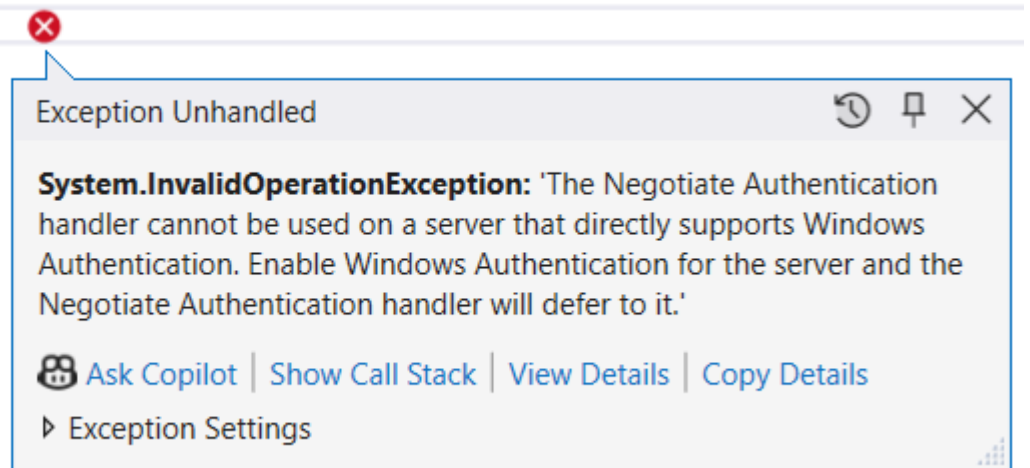
Etape 5 : On Lance ?

En https :



En IISExpress :

```
app.Run();
```



Explication

Pour Kestrel directement sur une machine Windows (sans IIS) :

L'authentification Windows via Negotiate fonctionnera directement avec Kestrel si l'application est exécutée sur une machine Windows et que le processus Kestrel a les permissions nécessaires.

Pour IIS Express (en développement) :

L'activation de `EnableIISExpressWindowsAuthentication` dans le fichier `.csproj` (ou via les propriétés du projet dans Visual Studio) permet l'authentification Windows pour notre application locale. IIS Express agit alors comme un proxy inversé devant Kestrel, gérant l'authentification Windows au niveau d'IIS et transmettant les informations d'identité authentifiées à votre application Kestrel.

Pour le déploiement sur IIS (en production) :

Si vous déployez votre application Blazor Web App sur IIS, ce n'est plus Kestrel qui gère directement l'authentification Windows, mais IIS lui-même.

Dans ce cas, vous devrez **activer l'authentification Windows dans le gestionnaire des services Internet (IIS Manager)** pour votre site ou votre application, et désactiver l'authentification anonyme.

IIS authentifiera l'utilisateur, puis passera son identité à votre application ASP.NET Core exécutée par Kestrel, et `HttpContext.User` contiendra alors les informations de l'utilisateur Windows.

Etape 6 : Création d'une page d'autorisation

```
@page "/authenticated"
@using Microsoft.AspNetCore.Authorization
@using System.Security.Claims
@attribute [Authorize]

<h3>Page Authentifiée</h3>

<p>Bienvenue sur cette page protégée. Vous êtes authentifié !</p>

@if (user != null && user.Identity != null && user.Identity.IsAuthenticated)
{
    <h4>Informations utilisateur :</h4>
    <p>Nom d'utilisateur : <strong>@user.Identity.Name</strong></p>

    <h5>Revendications (Claims) :</h5>
    <ul>
        @foreach (var claim in user.Claims)
        {
            <li><strong>@claim.Type</strong>: @claim.Value</li>
        }
    </ul>
}
else
{
    <p>Vous n'êtes pas authentifié.</p>
}
```

```
using Microsoft.AspNetCore.Components.Authorization;
using System.Security.Claims;

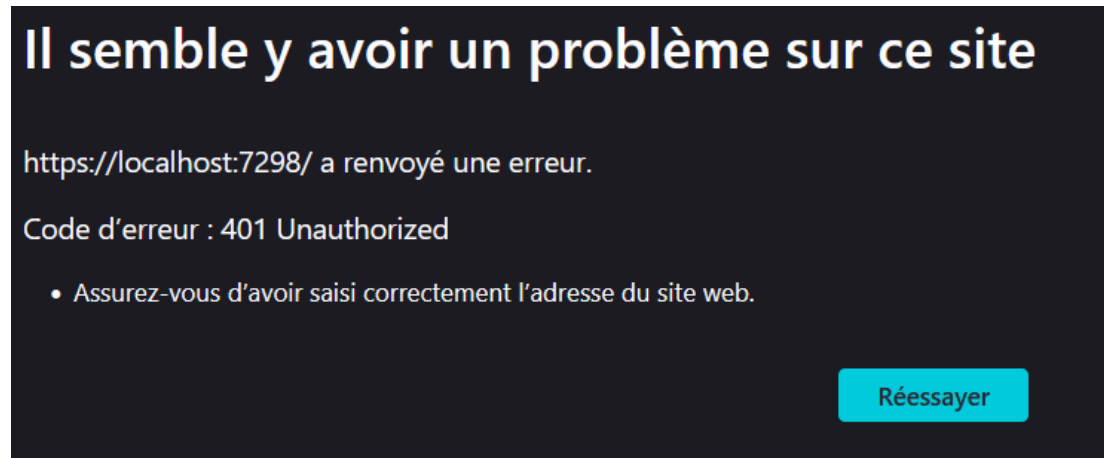
namespace BlazAuthSample.Client.Pages
{
    1 reference
    public partial class AuthenticatedPage
    {
        [CascadingParameter]
        2 references
        private Task<AuthenticationState>? AuthenticationState { get; set; }

        private ClaimsPrincipal? user;

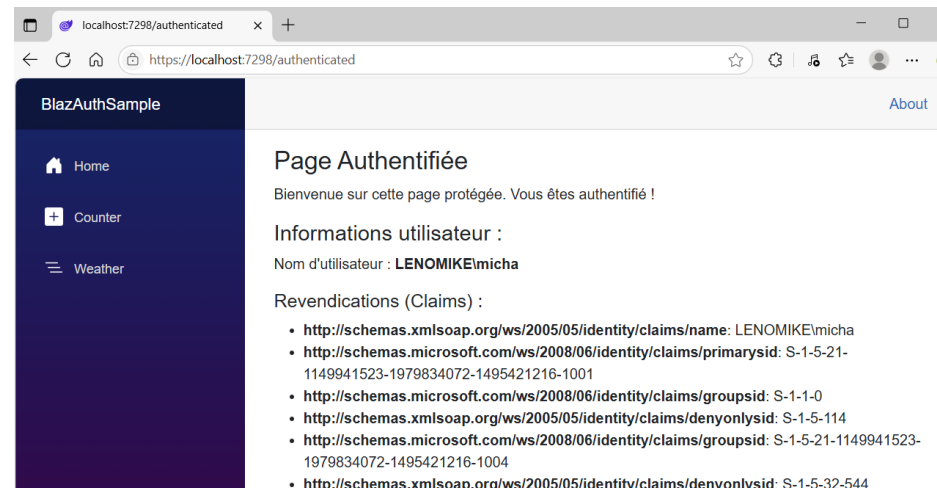
        0 references
        protected override async Task OnInitializedAsync()
        {
            if (AuthenticationState != null)
            {
                var authState = await AuthenticationState;
                user = authState?.User;
            }
        }
    }
}
```

Etape 7 : Navigateur

Si vous lancez votre application sur Firefox , vous risquez d'obtenir un status code 401



alors que sur un navigateur basé sur Chromium (Edge, Chrome, Brave,...)



Etape 7 : Navigateur



Chrome, Edge, IE

Authentification transparente

Transmettent automatiquement les identifiants
Windows dans les zones de confiance
(Intranet, Sites de confiance).



Firefox

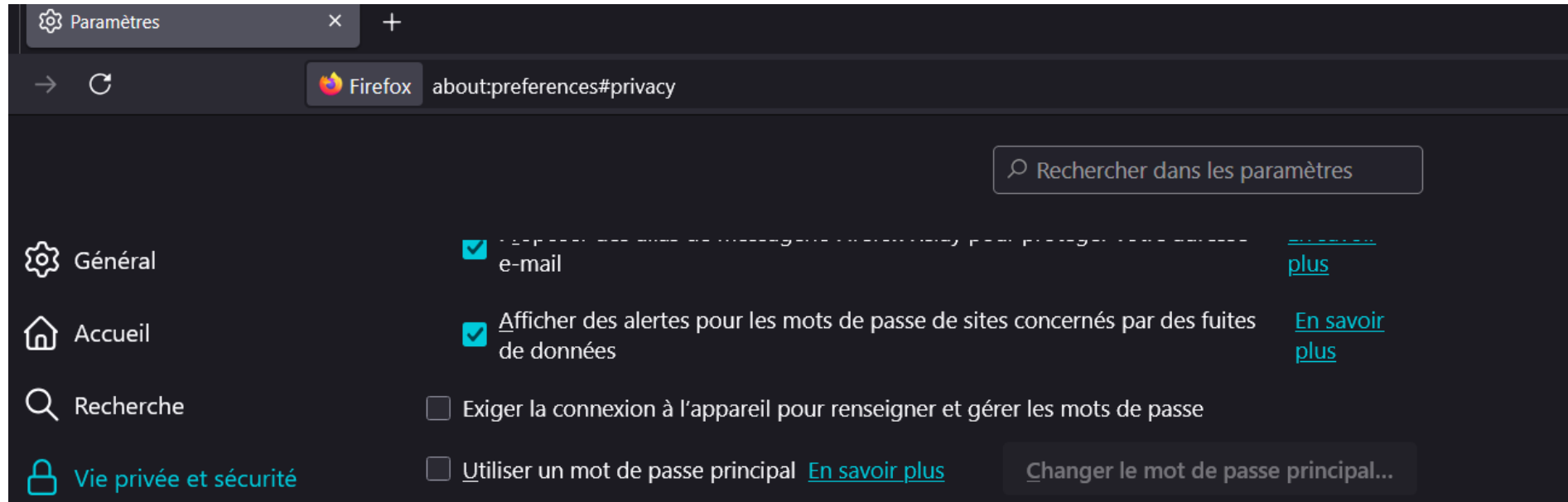
Erreur 401 / Demande d'authentification

Par défaut, ne transmet pas les identifiants.
Nécessite une configuration manuelle
(`about:config`) pour l'authentification
automatique ou affiche une boîte de dialogue.

Etape 7bis : Configurer FireFox

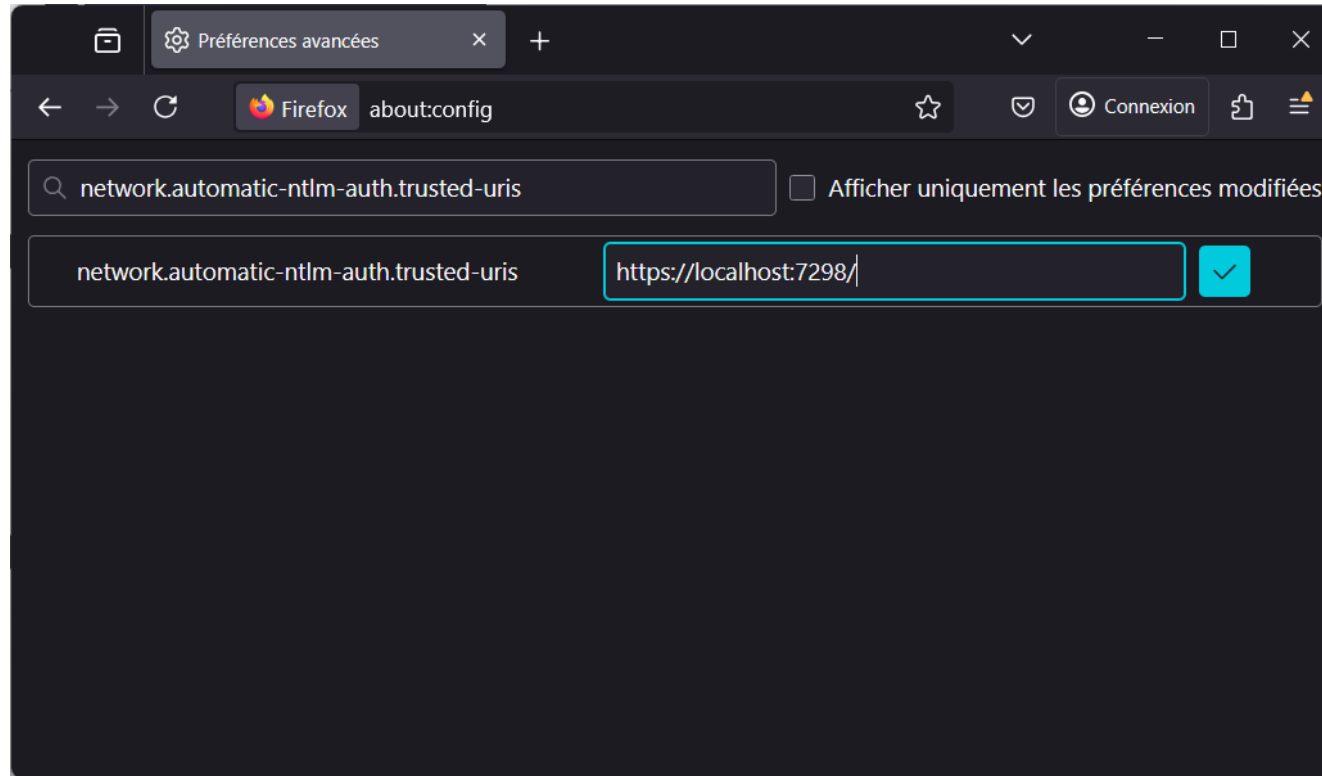
Pour que Firefox vous demande de vous authentifier ou utilise l'authentification Windows, vous devez le configurer manuellement :

- 1) ouvrez Firefox et tapez about:preferences dans la barre d'adresse, puis appuyez sur Entrée.
- 2) cochez la case à côté d'*Autoriser l'authentification unique de Windows pour les comptes Microsoft, professionnels et scolaires*



Etape 7bis : Configurer FireFox

- 3) Dans la barre de recherche, tapez ***network.automatic-ntlm-auth.trusted-uris***.
- 4) Entrez l'URL ou les Urls de l'applications Blazor

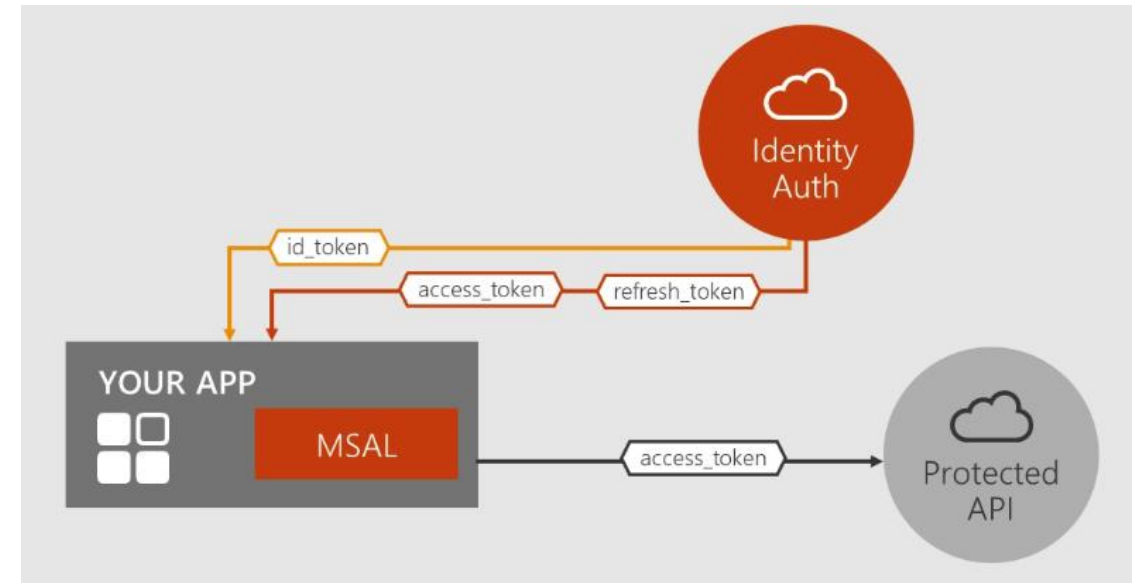


Active Directory

MSAL Sous Blazor

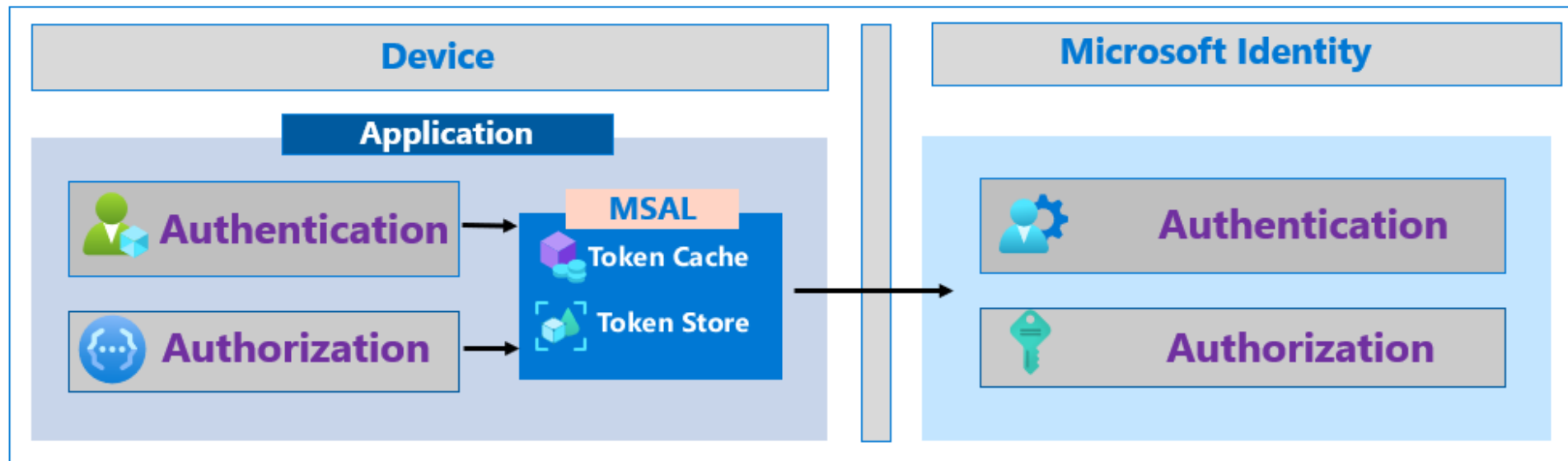
Qu'est-ce que MSAL ?

- Microsoft Authentication Library (MSAL) est une bibliothèque d'authentification développée par Microsoft
- Permet l'authentification des utilisateurs via Azure AD et Microsoft Identity Platform
- Gère l'acquisition, le stockage et le renouvellement des tokens d'accès
- Supporte plusieurs flux d'authentification (Authorization Code, Implicit Flow, etc.)



Architecture MSAL

- Public Client Application : Applications côté client (SPA, applications mobiles)
- Confidential Client Application : Applications serveur avec secrets
- Authority : Point de terminaison d'authentification (Azure AD)Token
- Cache : Stockage sécurisé des tokens
- Middleware : Intercepte et gère les requêtes authentifiées



Avantages de MSAL

- Intégration native avec l'écosystème Microsoft (Azure AD, Office 365)
- Gestion automatique du renouvellement des tokens
- Support multi-comptes et multi-tenants
- Implémentation des meilleures pratiques de sécurité
- Large communauté et support Microsoft

Points d'attention

- Complexité initiale de configuration
- Courbe d'apprentissage pour les développeurs
- Dépendance à l'infrastructure Microsoft
- Nécessite une bonne compréhension des concepts OAuth/OIDC

Exemple d'implémentation

MSAL - Blazor

Azure Entra



Microsoft Entra

Le première étape consiste à configurer notre Azure Entra.

En cliquant sur « Inscription d'applications », nous pouvons créer une nouvelle application et choisir

- son nom
- son type
- son Url de redirection

Remarque : L'url de redirection doit cibler votre application sous VS et se terminer par /authentication/login-callback

Vue d'ensemble

- Fonctionnalités d'évaluation
- Diagnostiquer et résoudre les problèmes
- Gérer
 - Utilisateurs
 - Groupes
 - External Identities
 - Rôles et administrateurs
 - Unités administratives
 - Partenaires d'administration délégués
 - Applications d'entreprise
 - Appareils
 - Inscriptions d'applications 1

Nouvelle inscription 2

Points de terminaison

Nouvelle inscription

Depuis le 30 juin 2020, nous n'ajoutons plus de no technique et des mises à jour de sécurité, mais noi Microsoft Graph. [En savoir plus](#)

Inscrire une application

* Nom

Nom d'affichage côté utilisateur pour cette application (il peut être modifié ultérieurement).

MSALBlaz 3

Types de comptes pris en charge

Qui peut utiliser cette application ou accéder à cette API ?

- ☒ Comptes dans cet annuaire d'organisation uniquement (uniquement - Locataire unique)
- ☐ Comptes dans un annuaire d'organisation (tout locataire Microsoft Entra ID – Multilocataire)
- ☐ Comptes dans un annuaire d'organisation (tout locataire Microsoft Entra ID – Multilocataire) et comptes Microsoft personnels (par exemple, Skype, Xbox)
- ☐ Comptes Microsoft personnels uniquement

[Aidez-moi à choisir...](#)

URI de redirection (facultatif)

Nous retournerons la réponse d'authentification à cet URI une fois l'utilisateur authentifié. Fournir ceci maintenant est facultatif et cela peut être modifié ultérieurement, mais une valeur est requise pour la plupart des scénarios d'authentification.

Web [https://localhost:7020/authentication/login-callback](#) 4

En continuant, vous acceptez les stratégies de la plateforme Microsoft

S'inscrire 5



Nous pouvons ensuite personnaliser l'application via ses propriétés

MSALBlaz | Personnalisation et propriétés ✕ ...

Rechercher ◊ ◀ Des commentaires ?

- Vue d'ensemble
- Démarrage rapide
- Assistant Intégration
- Diagnostiquer et résoudre les problèmes
- Gérer
 - Personnalisation et propriétés**
 - Authentification
 - Certificats & secrets
 - Configuration du jeton
 - API autorisées
 - Exposer une API
 - Rôles d'application
 - Propriétaires
 - Rôles et administrateurs
 - Manifeste
- Support + dépannage

Nom * ⓘ MSALBlaz

Logo 

Charger un nouveau logo ⓘ "blazentra.png" 

URL de la page d'accueil ⓘ ✓

URL des conditions de service ⓘ ✓

URL de la déclaration de confidentialité ⓘ ✓

Informations de référence sur le management des services ⓘ

Notes internes ⓘ

Enregistrer Abandonner

Enfin, il est important de compléter la configuration de l'authentification

Enregistrer Abandonner

Remarque

Pour Blazor, il faut utiliser une plateforme de type SPA

Configurer des plateformes

Applications web

**Web**
Générez, hébergez et déployez une application de serveur web. .NET, Java, Python

**Application à page unique**
Configurez des applications clientes de navigateur et des applications web progressives. JavaScript.

Applications de bureau et mobiles

**iOS / macOS**
Objective-C, Swift, Xamarin

**Android**
Java, Kotlin, Xamarin

**Applications de bureau et mobiles**
Windows, UWP, Console, IoT & appareils à entrée limitée, iOS classique + Android

Configurer une application à page unique

[← Toutes les plateformes](#)

[Démarrage rapide](#) [Documents](#)

i La dernière version de MSAL.js utilise le flux du code d'autorisation avec PKCE et CORS. [En savoir plus](#)

* URI de redirection

Les URI que nous accepterons comme destinations lors du retour des réponses d'authentification (jetons) après l'authentification ou la déconnexion des utilisateurs. L'URI de redirection que vous envoyez dans la demande au serveur de connexion doit correspondre à celui répertorié ici. Également appelées URL de réponse. [En savoir plus sur les URI de redirection et leurs restrictions](#)

✓

Types d'octroi

MSAL.js 2.0 ne prend pas en charge l'octroi implicite. Activez les paramètres d'octroi implicite uniquement si votre application utilise MSAL.js 1.0. [En savoir plus sur le flux de code d'authentification](#)

✓ Votre URI de redirection est éligible pour le flux du code d'autorisation avec PKCE.

Azure Entra



Microsoft Entra

Et surtout configurer les claims qui
seront transportées dans le token
(<https://learn.microsoft.com/fr-fr/entra/identity-platform/id-tokens>)

Ajouter une revendication facultative

Une fois que vous avez sélectionné un type de jeton, vous pouvez choisir dans une liste de revendications facultatives disponibles.

*** Type de jeton**
Les jetons d'accès et d'ID sont utilisés par les applications pour l'authentification. [En savoir plus](#)

☒ ID
☐ Accéder
☐ SAML

Revendication	Description
☐ acct	État du compte utilisateur dans le locataire
☐ acrs	ID de contexte d'authentification des opérations que le ...
☐ auth_time	Heure de la dernière authentification de l'utilisateur ; vo...
☐ cty	Pays/région de l'utilisateur
☒ email	Adresse e-mail adressable pour cet utilisateur, si l'utilisa...
☐ family_name	Fournit le nom de famille ou le nom de l'utilisateur, tel ...
☐ tid	Adresse IP
☒ given_name	Fournit le prénom ou le nom « donné » de l'utilisateur, t...

Modifier la revendication de groupes

L'ajout de la revendication de groupes s'applique aux types de jetons Accès, ID et SAML. [En savoir plus](#)

Sélectionnez les types de groupes à inclure dans les jetons Accès, ID et SAML.

☐ Groupes de sécurité
☒ Rôles d'annuaire
☐ Tous les groupes (inclut 3 types de groupes : groupes de sécurité, rôles d'annuaire et listes de distribution)
☐ Groupes affectés à l'application (recommandé pour les grandes entreprises pour ne pas dépasser la limite du nombre de groupes qu'un jeton peut émettre)

Personnaliser les propriétés de jeton par type

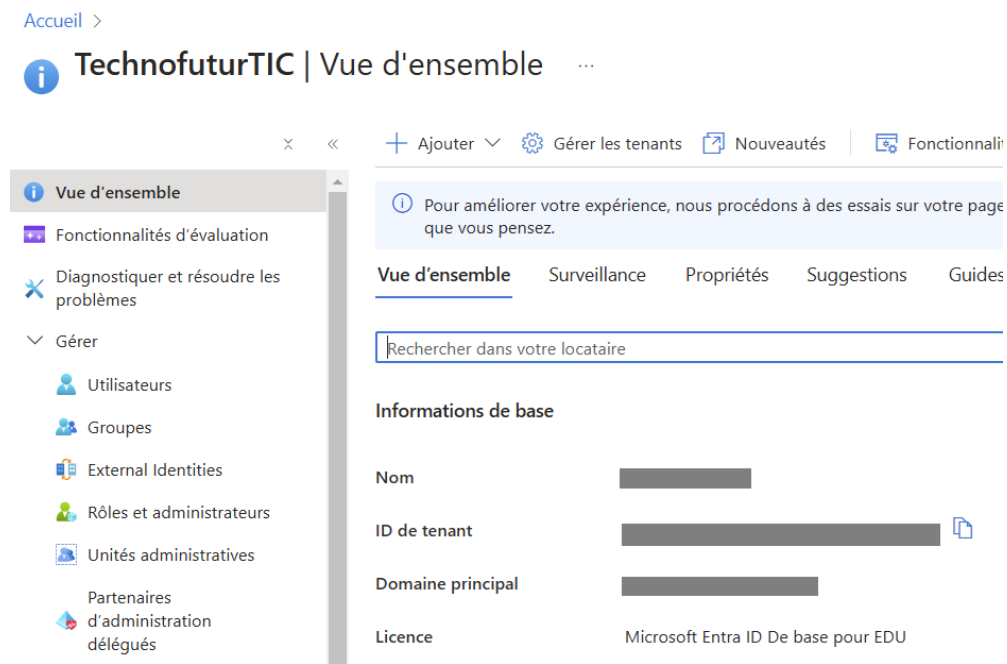
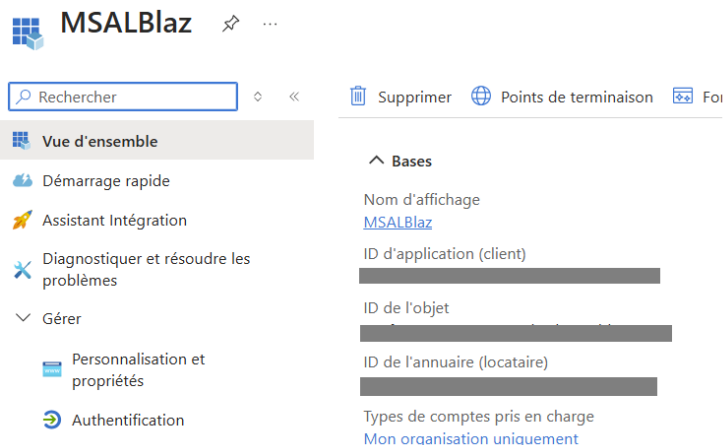
▼ ID
▼ Accéder
▼ SAML

Informations nécessaires pour notre App

Afin de configurer notre application Blazor, nous aurons besoins des données suivantes

:

- Dans la vue d'ensemble d'Azure Entra
 - ID du tenant
 - URI de l'autorité
- Dans la vue d'ensemble de l'application créée
 - ID de l'application (Client ID)



Passons au Blazor

La mise en place de MSAL passe par l'installation du NuGet :

Microsoft.Authentication.WebAssembly.Msal



Microsoft.Authentication.WebAssembly.Msal ✓ by aspnet, dotnetframework, Microsoft, **5,3M** downloads
Authenticate your Blazor webassembly applications with Azure Active Directory and Azure Active Directory B2C

Une fois ce NuGet installé, nous devons inclure un script dans le fichier index.html.

Ce script contient les méthodes pour les opérations d'authentification

```
<script src="_framework/blazor.webassembly.js"></script>
<script src="_content/Microsoft.Authentication.WebAssembly.Msal/AuthenticationService.js"></script>
<script>navigator.serviceWorker.register('service-worker.js');</script>
</body>
</html>
```

Remarque : Assurez-vous d'inclure le script **après** le script blazor.webassembly.js

MsalAuthentication Service

L'étape suivante consiste à ajouter le service MsalAuthentication et à le configurer.

```
builder.Services.AddMsalAuthentication(  
    options =>  
    {  
        builder.Configuration.Bind("AzureAd", options.ProviderOptions.Authentication);  
        options.ProviderOptions.DefaultAccessTokenScopes  
            .Add("https://graph.microsoft.com/User.Read");  
        options.ProviderOptions.LoginMode = "redirect";  
        options.ProviderOptions.Authentication.PostLogoutRedirectUri = "/";  
    });
```

Remarque :

le loginMode « redirect » remplace le paramètre « popup » par défaut. Au lieu d'avoir un écran contextuel, le site Web sera redirigé vers le fournisseur SSO spécifié dans Azure AD. Une fois l'autorisation réussie, elle sera redirigée vers le callback spécifié.

appsettings

Les informations de configuration de la connexion avec Azure Entra seront à placer dans appsettings.json.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",

  "AzureAd": {
    "Authority": "https://login.microsoftonline.com/TenantId",
    "ClientId": "ClientId (Application Id)",
    "ValidateAuthority": true
  }
}
```

Remarque : Il est préférable de stocker ces informations sensibles dans un secret

Nos composants

Nous devons encore créer nos pages (login, login-callback, logout, logout-callback) et un composant permettant de rediriger vers l'authentification:

Pour faciliter la mise en place, le code utilise un « placeholder » pour l'action, ce qui permettra de Réagir aux différents états d'authentification

```
@page "/authentication/{action}"

@using Microsoft.AspNetCore.Components.WebAssembly.Authentication

<RemoteAuthenticatorView Action="@Action" />

@code {
    [Parameter]
    public string Action { get; set; }
}
```

Le composant RemoteAuthenticatorView inclut des éléments d'interface utilisateur par défaut pour les différents états d'authentification. Il utilise RemoteAuthenticationState comme état persistant dans les opérations d'authentification.

LoginCallbackPath	Gets or sets the path to the login callback page.
LoginFailedPath	Gets or sets the path to the login failed page.
LoginPath	Gets or sets the path to the login page.
LogoutCallbackPath	Gets or sets the path to the logout callback page.
LogoutFailedPath	Gets or sets the path to the logout failed page.
LogoutPath	Gets or sets the path to the logout page.
LogoutSucceededPath	Gets or sets the path to the logout succeeded page.
ProfilePath	Gets or sets the path to the endpoint for modifying the settings for the user profile.
RegisterPath	Gets or sets the path to the endpoint for registering new users.
RemoteProfilePath	Gets or sets the path to the remote endpoint for modifying the settings for the user profile. It might be absolute and point outside of the application.
RemoteRegisterPath	Gets or sets the remote path to the remote endpoint for registering new users. It might be absolute and point outside of the application.

Routing

Le composant ***CascadingAuthenticationState*** gère l'exposition de `AuthenticationState` au reste de l'application et doit être ajouté. Il fait partie de l'espace de noms ***Microsoft.AspNetCore.Components.Authorization*** et doit être ajouté à ***_Imports.razor***.

Lorsque l'utilisateur n'est pas autorisé, il passera au nœud ***NotAuthorized***.

Nous changeons donc notre `App.razor`

```
@inject NavigationManager Navigation
<CascadingAuthenticationState>
  <Router AppAssembly="@typeof(App).Assembly">
    <Found Context="routeData">
      <AuthorizeRouteView RouteData="@routeData"
        DefaultLayout="@typeof(MainLayout)">
        <NotAuthorized>
          <h1>Redirection vers la page de login...</h1>
          @{Navigation.NavigateTo(
            $"/authentication/login?returnUrl={Uri.EscapeDataString(Navigation.Uri)}");}
        </NotAuthorized>
        <Authorizing>
          <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
          <FocusOnNavigate RouteData="@routeData" Selector="h1" />
        </Authorizing>
      </AuthorizeRouteView>
    </Found>
    <NotFound>
      <PageTitle>Not found</PageTitle>
      <LayoutView Layout="@typeof(MainLayout)">
        <p role="alert">Sorry, there's nothing at this address.</p>
      </LayoutView>
    </NotFound>
  </Router>
</CascadingAuthenticationState>
```

Protéger les pages

Pour protéger nos pages, nous n'avons plus qu'à utiliser l'attribut `[Authorize]`
(<https://learn.microsoft.com/en-us/aspnet/core/blazor/security/?view=aspnetcore-7.0&tabs=visual-studio#authorize-attribute>)

Soit dans la partie razor

```
@page "/profil"  
@attribute [Authorize]  
<h3>Profil</h3>
```

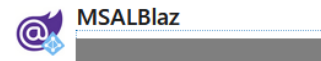
Soit en code-behind

```
using Microsoft.AspNetCore.Authorization;  
  
namespace MSALBlaz.Pages  
{  
    [Authorize]  
    1 reference  
    public partial class Profil  
    {  
    }  
}
```

Let's Go



Autorisations demandées



Cette application n'est pas publiée par Microsoft.

Cette application souhaite :

- ✓ Vous connecter et lire votre profil
- ✓ Conserver l'accès aux données auxquelles vous lui avez donné accès

Accepter ces autorisations signifie que vous autorisez cette application à utiliser vos données comme indiqué dans les conditions d'utilisation du service et la déclaration de confidentialité. Vous pouvez modifier ces autorisations à l'adresse <https://myapps.microsoft.com>. [Afficher les détails](#)

Cette application semble-t-elle suspecte ? [Signaler ici](#)

Annuler

Accepter

Ajout du bouton login et Logout

Afin de permettre de s'authentifier avant d'arriver aux pages protégées, nous pouvons faire un bouton qui pointe vers la page authentication/login

```
<button @onclick="LoginWithAzure" class="ms-login-button">
  <svg class="ms-logo" width="21" height="21" viewBox="0 0 21 21"
  xmlns="http://www.w3.org/2000/svg">
    <rect x="1" y="1" width="9" height="9" fill="#F25022" />
    <rect x="1" y="11" width="9" height="9" fill="#00A4EF" />
    <rect x="11" y="1" width="9" height="9" fill="#7FBA00" />
    <rect x="11" y="11" width="9" height="9" fill="#FFB900" />
  </svg>
  <span>Sign in with Microsoft</span>
</button>
```

```
[Inject]
NavigationManager NavigationManager { get; set; }
private void LoginWithAzure()
{
  NavigationManager.NavigateTo("authentication/login");
}
```

```
.ms-login-button {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  background-color: white;
  color: black;
  padding: 4px 12px;
  border-radius: 4px;
  border: 1px solid #e5e5e5;
  font-weight: 600;
  width: 226px;
  transition: background-color 0.2s;
  box-shadow: 0 1px 2px rgba(0, 0, 0,
0.05);
}

.ms-login-button:hover {
  background-color: #f9f9f9;
}

.ms-logo {
  flex-shrink: 0;
}
```

Et pour le logout ?

MSAL nécessite un état de déconnexion valide avant de procéder à la déconnexion effective, nous devons donc faire appel à la méthode **NavigateToLogout** pour s'en assurer

```
<div class="nav-item px-3">
  <AuthorizeView>
    <NotAuthorized>
      <button @onclick="LoginWithAzure" class="ms-login-button">
        <svg class="ms-logo" width="21" height="21" viewBox="0 0 21 21" xmlns="http://www.w3.org/2000/svg">
          <rect x="1" y="1" width="9" height="9" fill="#F25022" />
          <rect x="1" y="11" width="9" height="9" fill="#00A4EF" />
          <rect x="11" y="1" width="9" height="9" fill="#7FBA00" />
          <rect x="11" y="11" width="9" height="9" fill="#FFB900" />
        </svg>
        <span>Sign in with Microsoft</span>
      </button>
    </NotAuthorized>
    <Authorized>
      <button @onclick="LogoutWithAzure" class="ms-logout-button">
        <svg class="ms-logo" width="21" height="21" viewBox="0 0 21 21" xmlns="http://www.w3.org/2000/svg">
          <rect x="1" y="1" width="9" height="9" fill="#F25022" />
          <rect x="1" y="11" width="9" height="9" fill="#00A4EF" />
          <rect x="11" y="1" width="9" height="9" fill="#7FBA00" />
          <rect x="11" y="11" width="9" height="9" fill="#FFB900" />
        </svg>
        <span>Logout from Microsoft</span>
      </button>
    </Authorized>
  </AuthorizeView>
</div>
```

```
private void LogoutWithAzure()
{
    NavigationManager.NavigateToLogout("authentication/logout");
}
```

```
.ms-logout-button {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  background-color: #ffc978;
  color: black;
  padding: 4px 12px;
  border-radius: 4px;
  border: 1px solid #e5e5e5;
  font-weight: 600;
  width: 226px;
  transition: background-color 0.2s;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.05);
}
```

```
.ms-logout-button:hover {
  background-color: #fc8383;
}
```

Récupération des informations après login

Lorsque nous sommes connecté, nous pouvons utiliser l'**AuthenticationProvider** pour obtenir notre user et parcourir les claims

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Components;
using Microsoft.AspNetCore.Components.Authorization;
using System.Security.Claims;
namespace MSALBlaz.Pages
{
    [Authorize]
    1 reference
    public partial class Profil : ComponentBase
    {
        3 references
        List<string> Infos { get; set; }
        [Inject]
        1 reference
        private AuthenticationStateProvider AuthenticationStateProvider { get; set; }

        0 references
        protected override async Task OnInitializedAsync()
        {
            Infos = new List<string>();
            var authState = await AuthenticationStateProvider.GetAuthenticationStateAsync();
            var user = authState.User;
            foreach(var claim in user.Claims)
            {
                Infos.Add($"{claim.Subject.Name} - {claim.Type} - {claim.Value}");
            }
        }
    }
}
```

```
@page "/profil"

<h3>Profil</h3>
<ul>
    @foreach (string item in Infos)
    {
        <li>@item</li>
    }
</ul>
```

<https://github.com/michaelperson/MSALBlaz.git>